Rebirth: Complete Repository Walkthrough and Architecture Audit

File-by-file, section-by-section explanation of the Dash risk application

OpenAI - evidence-based code review

18 August 2026

Contents

Chapter 1

Scope, snapshot, and how to read this report

This report analyzes the public GitHub repository `streamlitdash/Rebirth` as it existed at commit `7a348c36ddb0e5eed5586a50` dated 18 August 2026. Pinning the commit matters because the repository can change after this document is generated. Every statement about runtime behavior refers to that pinned snapshot.

The repository is a reconstructed, fixture-safe Dash application for a financial risk and P&L dashboard called **Cube**. It is not a Streamlit application despite the GitHub account name. The active web framework is Dash on top of Flask. The project combines strict pandas-based data contracts, a transactional in-memory refresh manager, multi-page UI composition, browser-side interaction code, local persistence, and an official historical archive protocol.

1.1 Coverage promise

The main chapters explain every executable area of the repository: root entry points, configuration, deployment, adapters, feeds, domain code, pages, UI components, callbacks, browser JavaScript, CSS, tools, jobs, tests, data fixtures, archive leaves, and documentation assets. The final appendix accounts for every tracked path in the repository tree.

Review method. This is a static, commit-pinned code audit. The repository tree and source files were read through the GitHub repository interface. A local checkout was not available in the analysis environment, so this report does not claim that I freshly ran the application's pytest suite or launched the Dash server. Test chapters describe what the checked-in tests assert, not a new pass/fail result. The generated PDF itself is rendered and visually inspected.

"Every part" does not mean this document reproduces every source line. Reprinting the entire repository would make the explanation harder to read and would merely duplicate GitHub. Instead, each file is decomposed into its functional sections, public contracts, state transitions, interactions, failure modes, strengths, limitations, and improvement options. The largest files receive multi-section walkthroughs and callback atlases.

1.2 Evidence and intent labels

Three kinds of statements appear throughout:

Observed behavior is directly supported by code, schemas, tests, or checked-in data.

Inferred rationale explains why a design was probably chosen. Source code cannot prove the author's private thought process. These passages are therefore explicitly framed as inference, not fact.

Recommendation proposes a change. A recommendation is not a claim that the current code is wrong. Some current choices are excellent within the repository's stated constraints but would become limiting at larger scale.

1.3 Important boundary: active code versus recovered code

Several adapter files contain large **comment-only recovered connector examples**. They are preserved as historical integration material but are not imported or executed. The active runtime uses visible fake CSV sources marked `FAKE_REPLACE_ME`. This report always separates:

1. active executable contracts;
2. active fixture implementations;
3. comment-only recovered private connector examples; and
4. future production integration points.

That distinction is essential. Treating the commented blocks as live code would produce an incorrect architecture and an incorrect security assessment.

1.4 Executive conclusion

The repository's strongest idea is that **financial data is never committed incrementally**. A refresh builds a complete candidate in private, validates all source schemas and financial invariants, and only then swaps one immutable snapshot pointer from revision N to revision N+1. Readers continue using the last good revision while a refresh is in progress. A failed refresh records an error but does not corrupt the visible dashboard.

The main engineering trade-off is concentration of complexity. The central pipeline, component factory, Risk event module, P&L event module, and browser JavaScript are each very large. They encode many correct safeguards, but their size increases cognitive load and makes local reasoning harder. The most valuable next step is not a rewrite. It is to extract the already-present concepts into smaller state machines, planners, renderers, and connector services while preserving the existing invariants and test coverage.

Chapter 2

A beginner's mental model

This chapter introduces the concepts used everywhere else in the report. A reader who already knows Python, pandas, Flask, and Dash can skim it.

2.1 Python modules and the numbered file convention

Every `.py` file is a Python module. Imports connect modules together. The repository prefixes many files with `s01_`, `s02_`, and so on. The prefix creates a deliberate reading order and makes directory listings stable. It is not a Python requirement.

A module normally contains some combination of:

- constants, which give one name to a repeated value or column name;
- data classes, which bundle related values and can be frozen against mutation;
- protocols, which describe the methods or call signatures another object must provide;
- pure functions, which return an output from inputs without modifying shared state;
- managers or repositories, which own state, locks, caching, or persistence; and
- callback registration functions, which connect Dash components to Python functions.

The project uses `__all__` at the end of many modules to document the intended public API. Python does not fully enforce `__all__`, but it is a useful signal: helpers omitted from it are intended to remain internal.

2.2 pandas DataFrames as contracts

Most financial data moves through pandas `DataFrame` objects. A DataFrame resembles a table: columns have names, rows have an index, and each column can have a data type.

This project treats a DataFrame schema as an API. It often requires:

- an exact ordered list of columns;
- nonblank text in identity columns;
- numeric values that are not booleans, NaN where forbidden, or positive/negative infinity;
- unique keys at a declared grain;
- exact identity agreement between a request and a connector response;
- complete pairs such as `Risk SP01` and `dRisk SP01`; and
- authoritative tenor ranks supplied by market data.

The strictness is intentional. In a financial application, silently accepting a renamed, duplicated, or mis-keyed column can produce a plausible but wrong total. A loud validation error is safer than a quiet calculation on ambiguous data.

2.3 Dash layouts

A Dash page is a tree of components. Examples include:

- `html.Div`, `html.Button`, and `html.Table`, which become HTML;
- `dcc.Dropdown`, `dcc.Store`, `dcc.Interval`, and `dcc.Loading`, which add Dash behavior;
- `dash_table.DataTable`, which provides editable/filterable tables; and
- Plotly graphs, which are rendered by the browser.

Every interactive component has an `id`. A callback names component properties as inputs, state, and outputs.

2.4 Dash callbacks

A server callback can be pictured as:

```
changed Input properties
        +
current State properties
        |
        v
Python function
        |
        v
new Output properties
```

An **Input** triggers the callback when it changes. A **State** value is read when the callback runs but does not trigger it. An **Output** is what the callback updates.

The project also uses:

- `PreventUpdate`, which aborts the callback without changing outputs;
- `no_update`, which preserves selected outputs while changing others;
- `ctx.triggered_id`, which tells a multi-input callback what caused the current invocation;
- pattern-matching IDs such as `ALL` and `MATCH`, which let one callback serve a family of dynamically created controls;
- `Patch`, which updates part of a component property rather than replacing the whole value;
- clientside callbacks, which run JavaScript in the browser; and
- `running` outputs, which temporarily lock controls and mark a callback as active.

A callback is not necessarily a small function in this codebase. The main Risk callback is a state reducer and renderer combined into one request.

2.5 `dcc.Store` as a small state channel

A `dcc.Store` holds JSON-serializable browser state. Rebirth uses stores for several distinct purposes:

- filter state;
- open hierarchy paths;
- selected metric and detail context;
- delegated row/cell/header action envelopes;
- saved-view apply requests;
- date-edit drafts;
- busy and committed revision signals; and
- Stock load intent and loaded-date metadata.

The stores do not contain the full financial dataset. The authoritative data remains on the server in the committed snapshot. This avoids repeatedly serializing large DataFrames to the browser.

2.6 Server state versus browser state

The server owns:

- source configuration;
- validated DataFrames;
- the current immutable snapshot;
- refresh progress;
- the search catalog;
- local adjustment and saved-view repositories;
- archive files; and
- per-process caches.

The browser owns:

- the mounted component tree;
- current control values;
- visual selection;
- small stores and action envelopes;
- theme preference;
- delegated event listeners;
- refresh-progress presentation state; and
- a record of whether one recovery reload has already been attempted.

A revision number links the two. A browser action says, in effect, "apply this action to the view generated from revision N." The server can reject it if revision N is no longer current.

2.7 Flask underneath Dash

Dash uses Flask as its HTTP server. Rebirth adds ordinary Flask endpoints:

- `/healthz` for process and snapshot health;
- `/progressz` for structured refresh progress; and
- `/startz` to request the first background refresh.

The exact path may be prefixed when deployed behind a proxy. `s02_config.py` centralizes that prefix rather than making each link or endpoint guess it independently.

2.8 Immutable snapshots and atomic commit

An immutable snapshot is a bundle of related data that should be read as one version: dashboard rows, market books, dates, thresholds, portfolio metadata, source provenance, search indexes, and settings.

The refresh manager constructs a candidate privately. When all validation succeeds, it takes a short state lock and replaces the committed snapshot reference. That replacement is the atomic commit. Readers see either the old complete snapshot or the new complete snapshot, never a half-refreshed mixture.

2.9 Event delegation in the browser

A naive hierarchical table could create one Dash callback input for every row toggle and every metric cell. Large tables would then have thousands of reactive components. Rebirth instead uses JavaScript event delegation:

1. one document-level listener receives clicks;
2. it finds the nearest row/cell/header button;
3. it packages a tiny action with a sequence number and view-generation token;
4. it writes the action into a `dcc.Store` with `dash_clientside.set_props`; and
5. one server reducer validates and applies the action.

This is a strong performance pattern for large dynamic tables. The cost is that JavaScript and Python must agree exactly on action fields, IDs, tokens, and state semantics.

2.10 What "fail closed" means here

A fail-open design tries to continue when data is missing or ambiguous. A fail-closed design refuses to publish uncertain results. Rebirth generally fails closed:

- invalid connectors raise;
- unknown product identities raise;
- duplicate quote keys raise;
- incomplete required columns raise;
- stale edits raise or are ignored;
- an archive with missing or mismatched files is invisible; and
- a failed refresh preserves the previous committed snapshot.

Fail closed is appropriate for financial correctness, but it requires good error messages, observability, and operational recovery so that strictness does not become unexplained downtime.

Chapter 3

Architecture at a glance

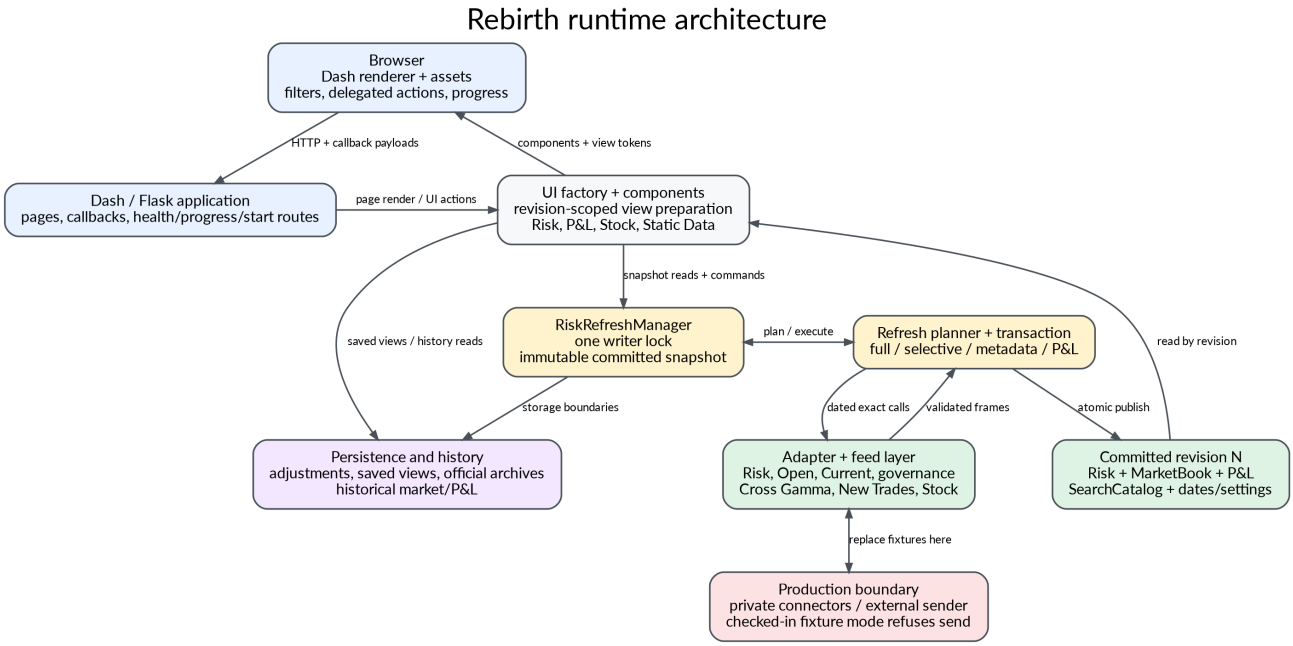

Figure 3.1: Major runtime layers.

3.1 Layer responsibilities

The **browser layer** renders Dash components and handles interactions that are too fine-grained or too latency-sensitive to model as thousands of individual server callback inputs. It owns theme switching, spreadsheet-like selection, delegated table actions, refresh-progress polling, and recovery from certain mount or transport races.

The **Flask/Dash application layer** owns page registration, routing, security headers, HTTP health/progress/start endpoints, the shared page shell, and callback registration.

The **callback layer** translates user intent into domain operations. It does not directly trust browser state. It validates revisions, generation tokens, selected dates, saved-view requests, editor rows, and busy state before asking domain services to act.

The **refresh manager** is the runtime authority. It owns the single-writer rule, dates, settings, source calls, candidate construction, progress, and atomic commit.

The **domain layer** contains pure or mostly pure pandas operations: schema validation, market joins, formula application, thresholding, search indexing, P&L governance, Stock comparison, Cross Gamma calculation, New Trades calculation, and archive validation.

The **feed and adapter layer** binds replaceable source functions to exact schemas. In the checked-in snapshot, those functions read deterministic fake CSVs. The adapter contracts are the stable boundary that a production integration should preserve.

The **persistence layer** contains three distinct protocols:

1. adjustment CSVs for local P&L edits;
2. small saved-view JSON documents; and
3. immutable official daily archive leaves protected by a success manifest and hashes.

3.2 Dependency direction

The healthiest dependency direction in the repository is inward:

```
pages and callbacks
      -> protocols and domain functions
      -> schemas and immutable data
```

The domain code does not import page modules. Connector adapters do not import UI code. Pages ask the current Flask application for builders instead of binding to a global manager. These choices make repeated app construction and unit testing possible.

There are two places where the dependency boundary becomes less clean:

- `ui/s09_factory.py` knows about many page-specific services and caches.
- `ui/s04_components.py`, `ui/s07_events.py`, `ui/s08_plevents.py`, and `assets/s02_app.js` each combine many subdomains in one file.

Those files are not necessarily poorly designed internally. They are simply large enough that their responsibilities should now be named and extracted.

Chapter 4

End-to-end runtime walkthrough

4.1 1. Process import and application construction

`gunicorn` imports `s04_server.py`, which imports `server` from `s01_app.py`. `s01_app.py` reads normalized runtime settings, constructs a refresh manager from `feeds/s01_sources.py`, creates persistence paths and P&L send configuration, and passes these dependencies into `ui/s09_factory.build_app`.

A crucial property is that application construction does **not** call financial sources. It creates callables, locks, routes, layouts, and callbacks. This keeps module import bounded and lets the first HTTP response paint a cold-start shell.

The global `app` and `server` objects exist for WSGI deployment. The same module also exposes `create_app(settings=None)` for tests and alternate embedding. When run directly, command-line host, port, and debug values are put into the environment before the global application is constructed, and Dash's development reloader is disabled so it cannot accidentally construct two refresh managers.

4.2 2. First browser paint and cold-start coordination

The first Risk or P&L page can mount before a committed financial revision exists. The shared shell displays navigation, controls in a disabled/loading state, a progress hero, and a retry path.

The browser-side progress controller can POST `/startz`. The server-side `StartupCoordinator` turns that request into one daemon refresh attempt. Multiple requests are idempotent: they observe the same process-wide writer rather than launching duplicate refreshes. A watchdog can report that a connector has been stuck for too long, but it deliberately does not start a second writer.

The browser polls `/progressz`. The response contains fields such as:

- whether a refresh is running;
- attempt ID and startup attempt ID;
- function and stage names;
- source type, underlying, and product label;
- current and total work units;
- revision;
- timestamps;
- server boot ID; and
- error text.

The JavaScript compares attempt IDs, revision movement, timestamps, and server boot IDs so an old completed response is not mistaken for the new attempt. When revision 1 becomes available, the normal Dash tree takes over. A session-scoped reload is used only as a guarded recovery for a failed mount handoff; the code records the recovery key to prevent loops.

4.3 3. Refresh planning

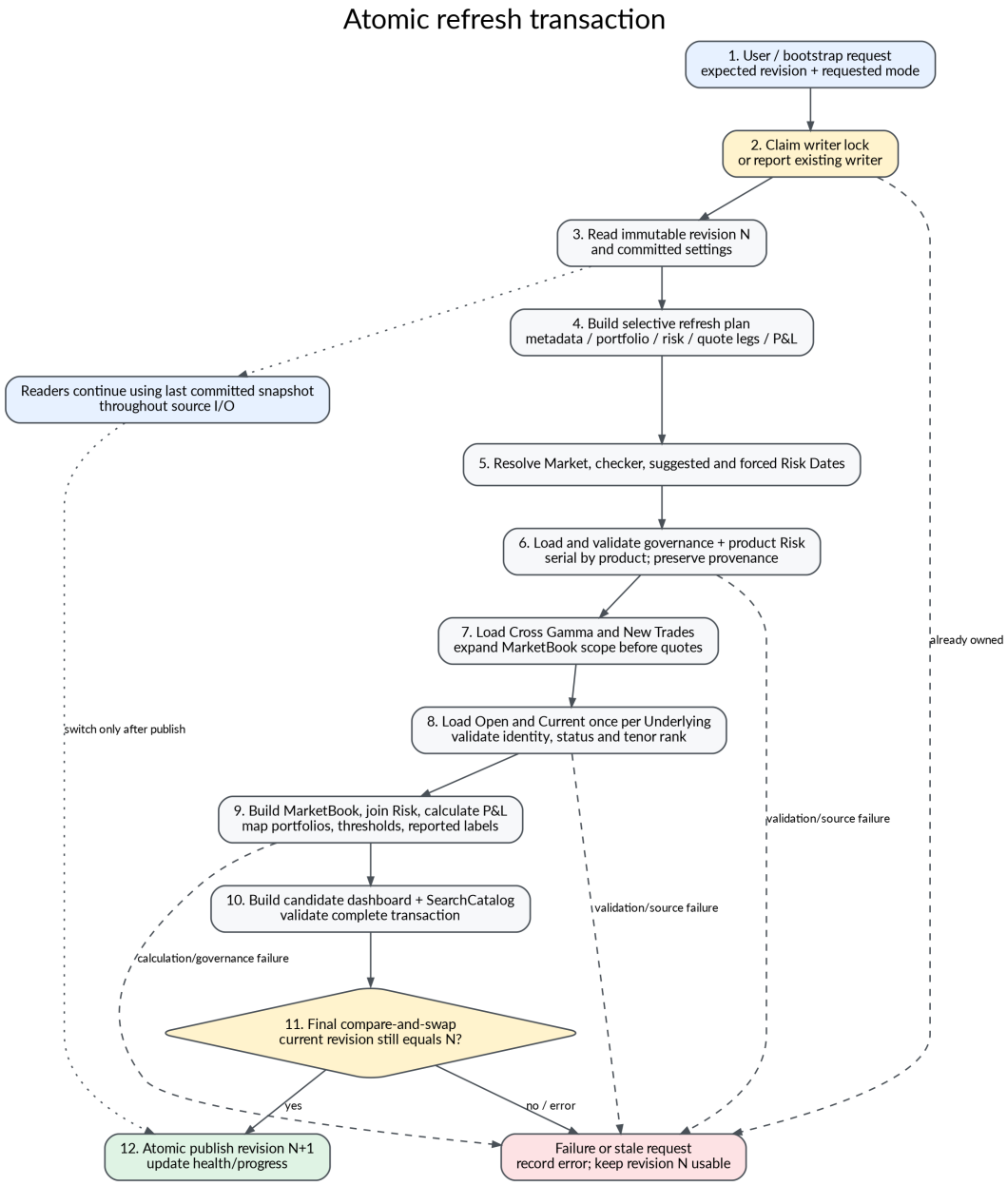

Figure 4.1: The refresh transaction and failure path.

A refresh can be triggered by:

- cold start;
- Refresh Risk;
- Refresh P&L;
- Refresh Portfolios;
- an automatic interval;
- applying forced dates;
- toggling Commodity market loading;
- toggling RiskChecker;
- or a request that follows another active writer.

The callback first normalizes the requested action and checks the browser's base revision. The manager then acquires the single writer lock and calculates a plan. It does not always reload everything.

The planner can select:

- metadata-only commit;
- portfolio-only rebuild;
- P&L-only rebuild from committed risk and market;
- selective risk products whose effective dates changed;
- selective Open or Current market legs;
- supplemental overlay reload;
- setting-driven changes; or
- a complete refresh.

This planning reduces connector traffic, but it is also one of the most complex parts of the code. The correctness is supported by dedicated lifecycle, date, provenance, overlay, and refresh-shell tests.

4.4 4. Authoritative dates

The manager computes one date chain:

1. `market_date_for` chooses the latest weekday. Weekend dates roll to Friday.
2. `checker_date_for` subtracts one pandas business day.
3. RiskChecker readiness assigns an `Age` per product pair.
4. `risk_date_for(checker_date, Age)` subtracts that many additional business days.
5. A valid forced Risk date overrides the suggested product date.
6. A valid forced Market date can override the natural market date under guarded rules.

The manager, not each connector, owns these subtractions. This prevents a production connector from accidentally applying T-1 twice. A limitation is that pandas `BDay` knows weekends but not the institution's holiday calendar. A production deployment should inject a governed business calendar.

4.5 5. Source loading and validation

The manager loads readiness and checker inventory, portfolio configuration, thresholds, reported-underlying mappings, Concerto mappings, risk frames, market Open frames, and market Current frames through configured callables.

For each product, validation checks the ProductSpec contract. Market functions are called once per source type and Underlying. Open and Current are currently loaded serially. The output must contain the exact requested Underlying and canonical axis columns.

Cross Gamma and New Trades are loaded before final market scope is established. Their input and output identities can introduce additional Underlyings or tenors that ordinary aged risk does not contain. The pipeline expands the MarketBook request set so those supplemental rows can be calculated against the same authoritative quote model.

4.6 6. MarketBook construction

Open and Current legs are independently validated, then outer-joined on the product's market keys. The outer join is important: Quick Market should show a market tenor even when no risk position exists at that tenor, and it should explicitly show an incomplete quote leg rather than erase the row.

Market-owned rank columns determine tenor order. Open and Current must agree on ranks for the same identity. The result records:

- Open;
- Current;
- Move = Current - Open;
- exact `Live` or `OFFICIAL` status;

- a boolean availability flag; and
- a human-readable market-data status.

The risk/P&L join is then a many-to-one left join from positions to the MarketBook. This keeps only position keys in the calculated risk view while preserving the full MarketBook for Quick Market.

4.7 7. P&L formulas

The formula is metadata-driven through `ProductSpec`; the engine avoids a long branch on product names.

For an absolute-move product:

```
Move = Current - Open
P&L  = Risk * Move * multiplier
```

For a percentage-move product:

```
Move = (Current - Open) / Open
P&L  = Risk * Move * multiplier
```

If Open is zero, percentage P&L is unavailable rather than infinite.

For Taylor Gamma:

```
scaled_move    = (Current - Open) * gamma_move_scale
developed_delta = GammaRisk * scaled_move / gamma_risk_step
P&L            = 0.5 * developed_delta * scaled_move * multiplier
```

The sourced Gamma row keeps the P&L and authoritative dRisk. A second derived Delta row is emitted only when both market legs exist. That row represents developed exposure, has no fabricated prior-day dRisk, and has P&L zero so the total is not counted twice.

Rows with incomplete market data retain their risk exposure but receive unavailable P&L.

4.8 8. Governance, thresholds, and reporting

Portfolio configuration is joined after source risk is normalized. Rows with no unique portfolio mapping remain visible as unmapped; they are not guessed into a business hierarchy.

Thresholds are keyed by Risk Type and Risk Greek. They support promotion logic for visible hierarchies and status summaries. Reported-underlying mappings are attached as a separate presentation/governance layer after calculations so a display aggregation does not change market identity or P&L.

4.9 9. Candidate validation and atomic commit

Before commit, the manager verifies candidate invariants and constructs immutable readers, including the Quick Search catalog. It then performs an optimistic revision check: if another commit invalidated the base revision used to plan this candidate, the stale candidate is not published.

The state lock is held only for the pointer swap and revision update. The expensive connector and pandas work occurs outside it. Readers can therefore continue to read the old snapshot while the writer works.

Any exception records failure progress and releases the writer lock. The previous revision remains the authoritative dashboard.

4.10 10. Revision propagation to pages

The committed revision appears in the shared shell and `/progressz`. Browser JavaScript writes it to `data-revision-store` only when a financial page capable of consuming the revision is mounted. This avoids firing

page-local callbacks against components that do not exist.

Risk and P&L callbacks then read the manager's immutable snapshot by revision. Stock uses a separate source and cache, but the committed revision tells it when portfolio governance may have changed.

4.11 11. Interactive Risk flow

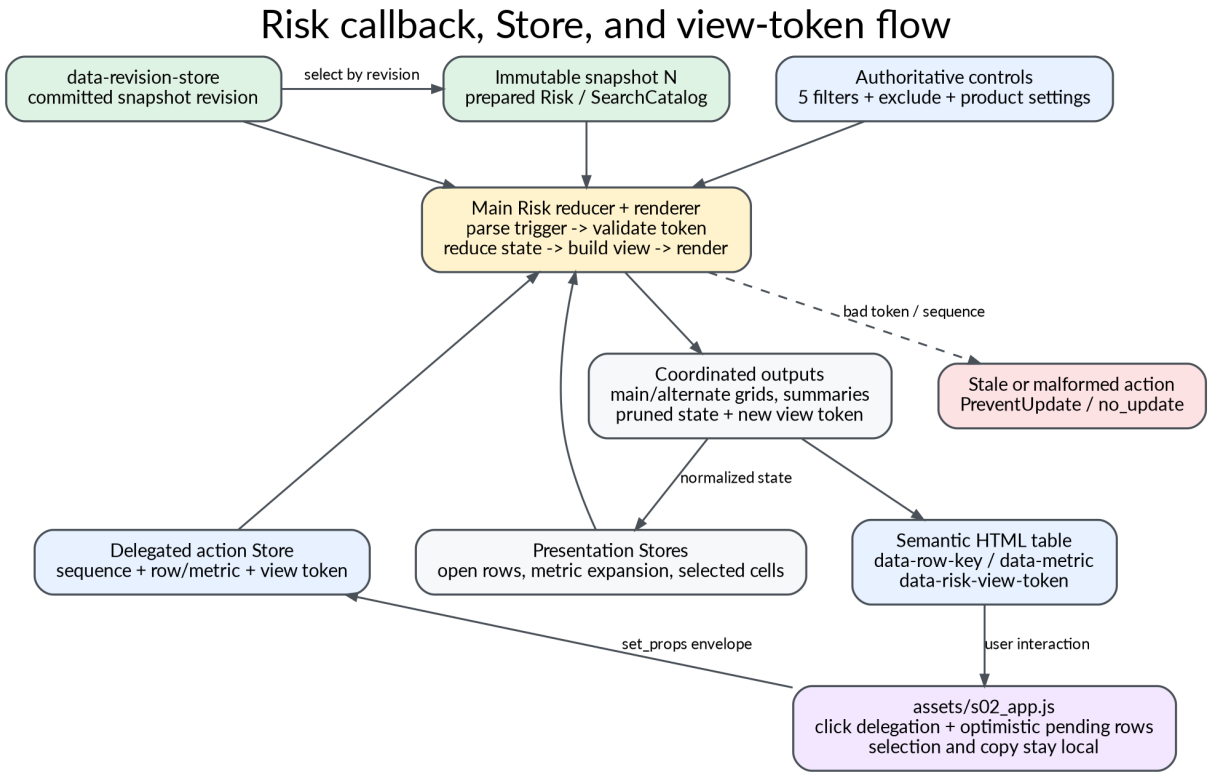

Figure 4.2: Browser actions, stores, reducers, and rendered views.

A row toggle or metric-cell click in the Risk hierarchy is caught by delegated JavaScript. The event envelope includes:

- action kind;
- monotonically increasing browser sequence;
- source;
- row key;
- metric or Credit measure;
- proposed open-row set for row actions; and
- the current view-generation token.

The main Risk callback rejects an envelope generated for an obsolete view. It then updates open paths, selected metric/cell, details context, and display state. It renders the visible hierarchy in the same request. Combining reduction and rendering avoids a second store-triggered round trip.

Filter callbacks normalize the five governed portfolio dimensions. Include mode applies OR within each selector and AND across selectors. Exclude mode removes a row if it matches any populated selector. Saved views use an apply request and acknowledgement protocol so a saved selection is not reported as applied until the visible controls actually match it.

4.12 12. Quick Risk and Quick Market

Quick Search never calls a source. It reads indexes published with the committed snapshot.

Quick Risk uses normalized searchable labels and compact integer row-position arrays. It filters the risk side by exact combined identity, then pivots additive risk metrics. Market quotes used in a combined pivot are independently deduplicated and aggregated so a quote is not repeated once per portfolio position.

Quick Market selects one exact market identity and displays the complete curve or surface, including market-only tenors. Text search is tokenized and case-insensitive for dropdown options, but the eventual selected identity must be exact.

Market history reads completed historical files and presents current and historical views without weakening the live MarketBook's identity rules.

4.13 13. P&L editing and sending

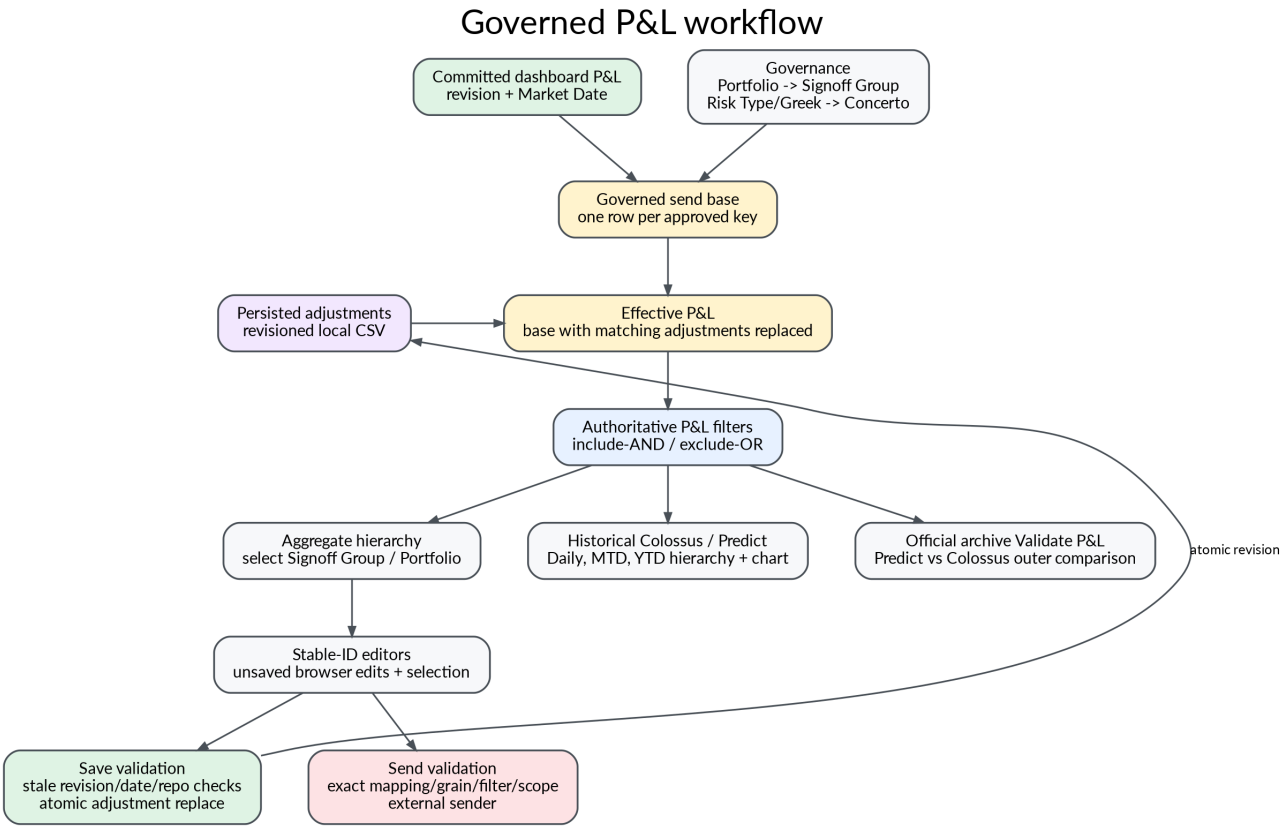

Figure 4.3: Governed P&L preparation, editing, and send boundaries.

Calculated rows are mapped from one `(Risk Type, Risk Greek)` pair to one Concerto field. Portfolio configuration supplies Signoff Group authority. Rows are collapsed to the governed send key.

Saved adjustments do not add another amount on top of the base. They **replace** the base value at the same governed key. This makes an edited value the effective value that will be sent, rather than an opaque delta that must be mentally reconciled.

Two editor scopes, Signoff Group and Portfolio, are generated from common callback factories. The code preserves stable row IDs, captures drafts, applies patches, recovers edits that arrive late relative to a filter render, and checks revision/date/filter context before save or send.

The checked-in sender implementations fail intentionally because production destinations and credentials are not present. The callback layer can report partial destination success when real sender functions are injected.

4.14 14. Stock flow

Stock is not part of the core risk refresh transaction. The Stock page requests two dates, loads exact source snapshots through a Stock adapter, outer-compares current and prior positions, and applies current portfolio governance afterward.

A per-process cache is keyed by committed revision and selected dates. An intent sequence prevents a slow earlier request from overwriting a newer date request. A nonblocking lock coalesces concurrent loads.

The hierarchy is lazy and uses a configurable promotion threshold. Source comparison rows are not rendered until the user explicitly opens them. This reduces initial page weight.

The grouping taxonomy is intentionally labeled provisional. Currency-based grouping is useful for a demo, but it should be replaced by an authoritative business hierarchy before production.

4.15 15. Official archive flow

An eligible OFFICIAL snapshot and Colossus extract are projected into exact archive schemas. Files are first written to a hidden pending directory. Row counts and SHA-256 digests are recorded. `_SUCCESS` is written last, and the directory is atomically renamed to its date.

A reader accepts a leaf only when the manifest, exact file set, schema version, row counts, hashes, OFFICIAL status, quote identities, tenor authority, and arithmetic all agree. A partial or tampered leaf is invisible rather than partly readable.

This is a strong local-filesystem commit protocol. It is not a substitute for distributed object-store transaction semantics, retention policy, encryption, or centrally managed access control.

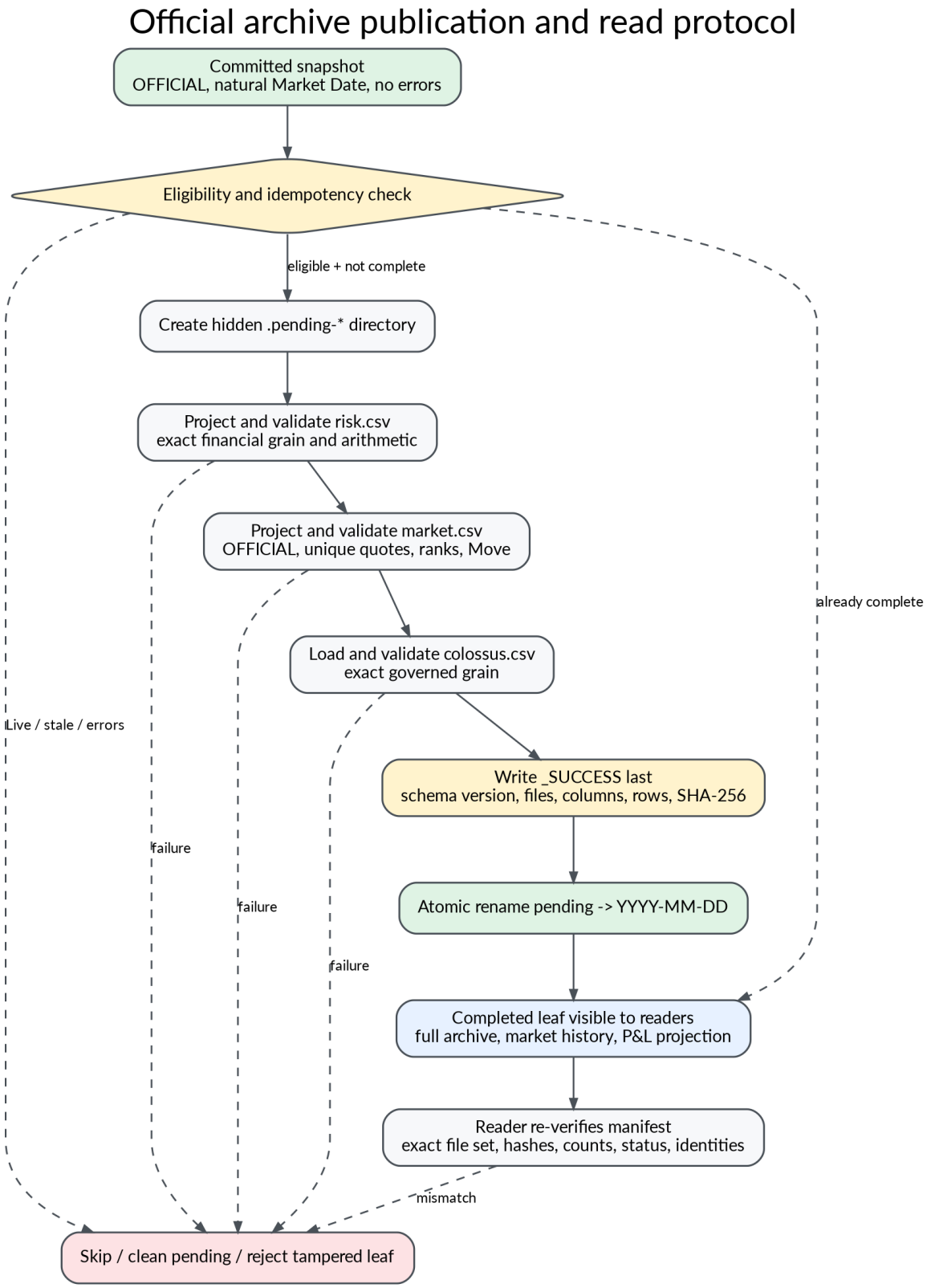

Figure 4.4: Official archive completion protocol.

Chapter 5

Root files: composition, configuration, deployment, and project documentation

5.1 `.gitattributes`

This file normalizes repository text behavior. It lets Git auto-detect text generally, then forces LF endings for CSS, CSV, INI, JavaScript, Markdown, Python, TOML, and text files. It marks PDFs and PNGs as binary.

Why this exists. The project contains generated CSV fixtures, a large stylesheet, JavaScript, notebooks, and cross-platform deployment code. Stable LF endings reduce noisy diffs and prevent shell or parser failures after edits on Windows.

Good choice versus alternatives. Repository-level normalization is better than relying on each contributor's editor. A future improvement would be to add a `.editorconfig` for indentation, final newline, and encoding, because `.gitattributes` governs Git storage but not the editing experience.

5.2 `.gitignore`

The ignore rules cover virtual environments, Python caches, test and lint caches, temporary outputs, logs, environment files, local adjustments, credentials, keys, and private data.

The most important section governs `data/histo`. Runtime `risk.csv`, `colossus.csv`, `market.csv`, and `_SUCCESS` files are ignored. Exact exceptions allow one synthetic completed archive on 2026-08-10 and historical market examples on four later dates to remain tracked. Hidden pending archive directories are ignored.

Inferred rationale. The author wanted the repository to demonstrate the complete archive protocol without creating a habit of committing live financial snapshots. Exact path exceptions make that intent auditable.

Limitation. Ignore rules are not a security boundary. A secret can still be forced into Git. Production should also use secret scanning, pre-commit checks, repository rules, and deployment-time secret stores.

5.3 `README.md`

The README is the primary maintained operating manual. It explains the fixture-only status, the architecture, startup behavior, source replacement boundaries, date rules, market flow, Risk and P&L pages, Stock, persistence, archive operation, tests, publishing, and manual verification.

Its strongest documentation pattern is the **numbered reading order**. A new maintainer can move from schema to pipeline, search, P&L, storage, reporting, Stock, saved views, overlays, archive, UI, feeds, and entry points without guessing which module is foundational.

The README also states important negative guarantees:

- no private credentials are required;
- active fixture data is visibly fake;
- commented recovered connectors do not run;
- app construction should perform no source I/O;
- only one writer should build a financial revision; and
- failed refreshes retain the last good data.

Limitation. The README is very large and `tools/s02_manual.py` treats it as the content source for another PDF. Large manuals tend to drift from code. The repository partly counters that through tests and generated diagrams, but a documentation CI job should verify commands, listed files, environment variables, and callback IDs.

5.4 `rebirth_cohesive_implementation_guide.md`

This guide is an earlier implementation/audit narrative. It records issues and design recommendations such as filter authority, saved views, Credit measure behavior, duplicate quotes, tenor ordering, Stock hierarchy, and initial layouts.

Some recommendations are now implemented in the pinned code. Therefore, the guide is valuable as architectural history but should not be read as the current source of truth.

Improvement. Add a front-matter status field and a table mapping each recommendation to `implemented`, `superseded`, or `open`. Without that, a reader can mistake a historical gap for a current defect.

5.5 `rebirth_full_cold_start_and_performance_guide.md`

This larger guide analyzes cold start and performance. It identifies the first-paint shell, serial per-Underlying market calls, the async bridge in recovered connector examples, missing hard source timeouts, DataFrame copying, and duplicate work.

Several findings still matter:

- market connectors are called serially per product and Underlying;
- the watchdog reports a stuck call but cannot cancel arbitrary synchronous source code;
- snapshot and UI construction copy data deliberately for isolation; and
- the single-process snapshot architecture limits horizontal scaling.

The guide should be retained as a design record. It should also be stamped with the audited commit and linked to current benchmark results.

5.6 `requirements.txt`

The runtime dependency file pins Dash, NumPy, pandas, Plotly, timezone data, and Gunicorn (on non-Windows platforms). Exact pins maximize reproducibility.

The Dash pin is particularly important because this application depends on page registry behavior, pattern-matching callbacks, `Patch`, clientside `set_props`, and renderer details. An unbounded Dash upgrade could change behavior far from the line that changed.

Trade-off. Exact pins improve repeatability but require an intentional upgrade cadence. A production project should combine pins with a lockfile or hashed requirements, dependency scanning, and scheduled compatibility upgrades.

5.7 `requirements-dev.txt`

This file includes runtime requirements and adds Plotly Cloud tooling, pytest, ReportLab, and Ruff. It separates deploy-time minimum dependencies from development/documentation tooling.

ReportLab exists because `tools/s02_manual.py` creates the project manual. Ruff supplies lint and static quality checks. Plotly Cloud supports the publishing wrapper.

5.8 `pytest.ini`

Pytest is configured to discover files named `s*.py` under `tests/` and show extra summary information for skipped, failed, and xfailed tests.

The custom filename pattern matches the repository's numbered convention. It also means that a contributor who creates `test_feature.py` will not have that file collected. The convention should be documented in the contributor workflow or enforced by a repository check.

5.9 `plotly-cloud.toml`

This small deployment descriptor names the application and records cloud identifiers. It is configuration, not runtime application logic.

Security observation. The identifiers are not private credentials, but production repositories should still verify whether environment-specific deployment IDs belong in the default branch or in environment configuration.

5.10 `s01_app.py`: application composition root

5.10.1 Imports and command-line parsing

The module imports runtime settings, the feed-level manager factory, the UI factory, and P&L history/send types. A small parser accepts host, port, and debug options for direct execution.

5.10.2 `create_app(settings=None)`

`create_app` is the composition root. It:

1. resolves or accepts a `RuntimeSettings` instance;
2. constructs the refresh manager from feed composition;
3. resolves the adjustment, saved-view, and historical paths;
4. builds P&L sender configuration;
5. wraps combined historical readers so low-level archive errors become the domain-specific `PLSendValidationError`; and
6. calls `ui.s09_factory.build_app` with explicit dependencies.

This is dependency injection without a framework. The app factory receives services instead of importing hidden singletons.

5.10.3 Global `SETTINGS`, `app`, and `server`

The module creates WSGI-compatible globals for `gunicorn`. Dash exposes its underlying Flask server through `app.server`.

5.10.4 Direct execution

When run as a script, parsed values are written to the environment before the application is built. `use_reloader=False` is deliberate: Flask's reloader can import the module twice and create duplicate background startup behavior.

5.10.5 Why this design is strong

A composition root is the right place to choose implementations. Domain modules can depend on protocols and callables rather than the fixture feed module. Tests can call `create_app` with different settings.

5.10.6 Limitations and improvements

The global app is still constructed at import, which is normal for WSGI but can make some test imports expensive. Construction is currently source-I/O-free, so the cost is acceptable. For more environments, expose a `create_server()` WSGI function or an application loader rather than adding branches to the module.

5.11 `s02_config.py`: normalized runtime settings

5.11.1 Truth-value parsing

A shared truthy set prevents each environment flag from inventing its own interpretation. `env_flag` reads a name and falls back to a default.

5.11.2 Path-prefix normalization

`normalize_path_prefix` rejects full URLs, query strings, and fragments. It guarantees exactly one leading slash and one trailing slash. This prevents malformed Dash routes and open redirect-like mistakes when building internal links.

5.11.3 Data-path resolution

`resolve_data_path` expands user paths and resolves relative paths against the project root, not whichever working directory launched the process. That is essential for schedulers and WSGI servers.

5.11.4 `RuntimeSettings`

The frozen data class collects host, port, debug mode, request prefix, data paths, and related runtime options. `from_env` validates the port and supports normal, JupyterHub proxy, and service-prefix deployment modes. `dash_kwargs` converts settings into the exact arguments needed by Dash.

5.11.5 Why frozen settings are appropriate

A runtime configuration object should not change halfway through app construction. Freezing it turns accidental mutation into an error and makes tests compare settings reliably.

5.11.6 Limitations

Environment parsing is hand-written. As the option set grows, a typed configuration library could provide richer validation and generated documentation. The current implementation is small enough that its directness is a strength.

5.12 `s03_publish.py`: controlled Plotly Cloud staging

5.12.1 Staging model

The publisher does not upload the working tree directly. It creates a temporary directory, copies an allowlisted runtime bundle, renames numbered entry points to conventional cloud names where required, and invokes the Plotly Cloud CLI.

5.12.2 Archive filtering

The staging logic includes the explicit synthetic official archive and selected legacy demo history, while excluding runtime official leaves and hidden pending directories.

5.12.3 Ignore rules

Caches, development artifacts, credentials, tests not needed at runtime, and local outputs are excluded. The staged bundle is therefore smaller and less likely to leak local files.

5.12.4 External process handling

The CLI call has a finite timeout and propagates failure. Temporary staging is removed automatically.

5.12.5 Strengths

A staged allowlist is safer than "upload the repository." It also creates a deterministic deployment shape and lets local file names retain the numbered reading convention.

5.12.6 Improvements

Generate and log a manifest of staged files with hashes. Add a dry-run mode that prints the bundle. Run import and smoke checks inside the staged directory before invoking the cloud CLI. Treat deployment metadata as environment-specific when multiple environments are introduced.

5.13 `s04_server.py`: Gunicorn policy

The Gunicorn configuration uses:

- one worker process;
- the `gthread` worker class;
- four threads; and
- a configurable timeout.

The comment explains the critical reason: the committed financial snapshot is process-local. Two worker processes would independently refresh and could serve different revisions.

5.13.1 Why one process is correct here

Threads in one process share the same manager and committed snapshot. The manager's locks coordinate them. This is safer than pretending multiple processes share memory.

5.13.2 Scalability ceiling

One process limits CPU parallelism, restart availability, and horizontal scale. The production path is not "increase `workers`." It is to move the authoritative snapshot, revision, refresh lease, and progress into a shared service or datastore, then make web workers stateless readers.

Chapter 6

Adapter and feed layer

6.1 `adapters/__init__.py`

The package initializer contains only a docstring. That is intentional: importing `adapters` does not import any recovered private connector block or create source clients. The active feed module imports only the builders it needs.

6.2 `adapters/s01_common.py`: shared exact-contract helpers

6.2.1 Comment-only recovered section

The top portion preserves an earlier helper implementation, including a `run_async` bridge. Because every line is commented, none of its imports, executor state, or async behavior exists at runtime.

The recovered bridge appears to use a single worker to run asynchronous private APIs from synchronous code. That can serialize otherwise independent work and complicate event-loop lifetime. It should not be revived unchanged for a high-volume production connector.

6.2.2 Active Protocols

`RiskSource` describes a callable receiving one Risk date and returning a DataFrame. `MarketSource` describes a callable receiving one Market date, one Underlying, and an exact market status.

Protocols document the boundary without forcing inheritance. Any callable with the right shape can be supplied.

6.2.3 `exact_frame`

This helper requires a DataFrame and an exact ordered column tuple, then returns a copy. The copy prevents downstream normalization from mutating connector-owned memory.

6.2.4 `exact_status`

Only `Live` and `OFFICIAL` are accepted. Case or spelling is not guessed. This is important because status routing can change which market snapshot a private system returns.

6.2.5 `exact_underlying`

The market boundary accepts one nonblank text identity. Batch semantics remain a framework decision rather than an undocumented connector feature.

6.2.6 `market_frame`

This function validates the requested Underlying and status, calls the bound market function, validates the exact schema, confirms that all returned rows belong to the requested Underlying, and optionally appends the selected status.

6.2.7 Why this pattern is good

It makes adapter builders small and uniform. The framework can rely on exact identity without duplicating checks in every product module.

6.2.8 Improvement

The copy and exact-order rule are defensible, but error messages could include the source callable name and a structured error code. Production observability benefits from knowing whether the failure was type, column, identity, timeout, or source transport.

6.3 `adapters/s02_ir.py`: IR Delta and DeltaVega contracts

6.3.1 Recovered connector example

Most of the file is a comment-only record of private MRX/RAMP-style logic, imports, schemas, business transformations, and market access. It demonstrates how the original system grouped currencies, scaled risk, assigned tenor order, and handled product-specific cases.

The retained code is useful integration evidence, but manual uncommenting is fragile. It can reintroduce broad wildcard imports, blanket warning suppression, broad `except` blocks, single-thread async bridging, and file side effects.

6.3.2 Active constants

The active section defines exact schemas for:

- IR Delta risk, Open, and Current;
- IR DeltaVega risk, Open, and Current.

IR Delta is a one-axis curve using `Tenor Swap`. IR DeltaVega is a true two-axis surface using `Tenor Swap` and `Tenor Option`, plus market-owned order columns for both axes.

6.3.3 `build_ir_adapters`

The builder accepts six source callables and returns adapters keyed by `ir/delta` and `ir/deltavega`.

Each closure:

- forwards the authoritative date;
- forwards exactly one Underlying for market calls;
- validates the exact schema;
- attaches status to Current; and
- returns a `ProductConnectorAdapter` with `risk`, `market_open`, and `market_status` legs.

6.3.4 Why closures are used

The closure binds source implementations once during composition. The refresh manager receives a uniform adapter and does not need to know which file or service implements IR.

6.3.5 Limitation

Only the active IR Delta and DeltaVega public adapter examples are implemented here, while other IR ProductSpecs can be served through the generic feed adapter path. A production connector registry should make that distinction explicit rather than relying on file comments.

6.4 `adapters/s03_fx.py`: FX Delta, Gamma, and Vega contracts

6.4.1 Recovered connector example

The comment-only portion shows private data retrieval and currency-group mapping. It derives EUR crosses from available pairs, calculates mid quotes, handles Live rollback behavior for volatility, and generates tenor ranks.

Several historical patterns should be corrected before reuse:

- bare `except` blocks can hide missing identity or source failures;
- fallback zeros can turn missing quotes into apparently valid moves;
- hard-coded currency lists should be governed reference data;
- broad imports and warning suppression obscure dependencies; and
- quote construction should carry explicit provenance and failure status.

6.4.2 Active schemas

FX Delta and Gamma are scalar by Underlying. FX Vega uses one `Tenor` Swap axis. The active schemas require portfolio and connector-owned Group on risk rows.

6.4.3 `_scalar_adapter`

One helper builds a uniform ProductConnectorAdapter for a risk source and two market sources. Current receives the selected market status.

6.4.4 `build_fx_adapters`

The public builder binds nine source functions and returns three source-type adapters: `fx/delta`, `fx/gamma`, and `fx/vega`.

6.4.5 Strength

The active contract does not carry historical private assumptions into the calculation engine. Cross construction, currency grouping, and service-specific parsing remain connector responsibilities.

6.4.6 Improvement

Rename `_scalar_adapter` to `_single_market_key_adapter` or similar. FX Vega is not scalar in the ordinary sense; it has a tenor axis. The current name refers to the common function shape, not the market grain.

6.5 `adapters/s04_credit.py`: Credit Delta and optional measures

6.5.1 Recovered connector example

The comment-only block shows a complex Credit source: issuer mapping, index constituent handling, PMRB calculations, 5Y scaling, Risk/dRisk normalization, Region, and six measure families.

It also writes a temporary mapping CSV and uses broad exception handling. Production should replace that with an explicit cached reference-data service and typed source errors.

6.5.2 Active base schemas

Credit risk can arrive in one of two exact bases:

- without Region; or
- with connector-owned Region.

Optional measure columns may follow the base. The canonical measure set is SP01, PSP01, PM01, PM01P, Theta, and JTD, with a Risk and dRisk column for each.

6.5.3 `build_credit_adapter`

`get_risk` performs more flexible validation than `exact_frame` because optional measure pairs are allowed. It:

1. verifies the object is a DataFrame;
2. chooses the Region or non-Region base;
3. rejects unexpected columns;
4. requires canonical column order;
5. detects which optional measure columns are present; and
6. requires both Risk and dRisk for every included measure.

Open and Current use the standard market helper on a `Tenor Swap` curve.

6.5.4 Why this is better than adding arbitrary columns

Credit measures are a governed extension point. Allowing any extra numeric column would make UI and aggregation behavior unpredictable. Pair-complete canonical measures preserve a stable public contract while allowing connectors that do not support every measure.

6.5.5 Limitation

Column-order strictness can be inconvenient for independently produced DataFrames. That inconvenience is acceptable at the boundary, but a production adapter could reorder a named source schema into canonical order before calling this public contract.

6.6 `adapters/s05_stock.py`: validated Stock boundary

6.6.1 Date normalization

`normalize_stock_date` rejects nulls and booleans, parses one scalar date, removes timezone information, and normalizes to midnight.

6.6.2 `StockConnectorAdapter`

The frozen adapter binds one `StockSource` callable. `get_stock` normalizes the date and calls `core.s07_stock.validate_stoc`
with a date-specific label.

6.6.3 Fake source

The checked-in source returns three visibly fake rows. Values vary deterministically with the date so the comparison page proves it requested two distinct snapshots.

6.6.4 `GetStock`

The module retains the business-facing `GetStock` spelling as an alias. Production composition can replace the source without changing callers.

6.6.5 Strength and limitation

The boundary is small, explicit, and well tested. The fake source lives in the adapter module, whereas the rest of the application centralizes fixtures in `feeds` and `data`; moving it to the fixture feed would make source composition more uniform.

6.7 `adapters/s06_new_positions.py`: raw New Trades blotter contract

6.7.1 Public schema aliases

The module preserves earlier `NEW_POSITION_*` names as aliases while the core New Trades module owns the canonical column order. This is a compatibility bridge.

6.7.2 Date and primitive validators

Private helpers normalize one date, detect blank values, require nonblank text, require blanks where a row type forbids a field, reject booleans in numeric columns, require finite numbers, and parse trade timestamps without accepting ambiguous numeric dates.

6.7.3 `validate_new_positions`

The mixed blotter has two exact row types.

For a `MARKET` row:

- trade ID, position ID, portfolio, product identity, Underlying, trader code/name, and trade time are required;
- Cash Flow is forbidden;
- Risk is required;
- Notional is optional descriptive metadata;
- a known traded level requires a numeric value;
- an unknown traded level must be blank so the later calculator can use Open; and
- the reserved Cash Flow/New identity is forbidden.

For a `CASHFLOW` row:

- Risk Type must be Cash Flow and Risk Greek must be New;
- market identity, trade time, trader, Risk, Notional, and traded level must be blank;
- Cash Flow is required; and
- P&L is immediately equal to Cash Flow.

Trade/position identity must be unique. MARKET P&L remains unavailable at this stage because the core pipeline has not yet joined a MarketBook.

6.7.4 Builder and fake source

`build_new_positions_adapter` binds a site blotter. The fake source produces deterministic market and cash-flow examples.

6.7.5 Strength

The raw boundary makes row semantics explicit instead of allowing one loosely populated table. The later core calculator can trust the row-type invariants.

6.7.6 Improvement

The active schema historically contains a P&L-related field that is recomputed for MARKET rows. Remove redundant source fields when backward compatibility allows, or clearly mark them as ignored to reduce integration confusion.

6.8 `adapters/s07_cross_gamma.py`: Cross Gamma sensitivity boundary

The adapter validates one date and one exact sensitivity matrix. The fake source includes Credit XGamma and XGamma Vega examples with separate input and output product identities.

`build_cross_gamma_adapter` wraps a site source with `validate_cross_gamma_rows`. `GetCrossGamma` preserves the business-facing name.

The adapter does not calculate P&L or output risk. That belongs to `core/s09_cross_gamma.py`, which has access to ProductSpecs and MarketBooks.

6.9 `adapters/s08_commo.py`: Commodity Delta contract

The header states that recovered originals referenced Commodity Delta and Vega builders but did not contain their implementations. The repository does not invent missing private code.

The active module defines a strict Commodity Delta curve contract and binds risk, Open, and Current functions through `build_commo_adapter`.

6.9.1 Good judgment

Explicitly saying “source body not recovered” is better than filling the gap with plausible but unverified financial logic.

6.9.2 Improvement

The ProductSpec catalogue contains Commodity Vega as well. Production composition should provide a documented adapter or generic source mapping for it and should make unsupported source types fail during startup configuration rather than during a refresh.

Chapter 7

`feeds` package

7.1 `feeds/__init__.py`

The initializer is empty. Importing the package has no side effects.

7.2 `feeds/s01_sources.py`: active source composition

This file is the authoritative checked-in runtime composition. It is intentionally fixture-only.

7.2.1 CSV caching

CSV readers cache a frame using file revision information such as path, modification time, and size. Partitioned risk and market views are also cached. Returned frames are copied so a caller cannot mutate the cached source.

7.2.2 Visible provenance

Fixture strings contain `FAKE_REPLACE_ME`, and connector progress/provenance labels identify the fake mode. This is a safety feature: screenshots and exports should visibly reveal that the integration is unfinished.

7.2.3 Market status

The fixture resolver uses London time and a 22:00 cutoff, with weekend rollback, to choose `Live` or `OFFICIAL`. The manager still owns the date; the resolver only owns status for that date.

7.2.4 Source families

The module supplies callables for:

- RiskChecker readiness and inventory;
- product risk;
- product Open;
- product Current;
- portfolio configuration;
- thresholds;
- Concerto mapping;
- reported-underlying mapping;
- Cross Gamma;
- New Trades;
- Stock;
- historical/Colossus reads; and

- sender placeholders.

7.2.5 Adapter composition

Adapter closures are created with bound source-type filters. Care is taken to capture the loop value rather than letting every closure refer to the final loop item, a common Python closure bug.

7.2.6 Sender behavior

External send functions reject delivery because private destinations and credentials are not included. This is preferable to logging a false success.

7.2.7 Comment-only private blocks

Recovered connector code and imports remain commented. The current migration instruction is to uncomment a selected real block and switch the adjacent registration.

7.2.8 Main limitation and recommended redesign

Manual uncommenting does not scale and is hard to review. Replace it with a connector registry:

```
CUBE_SOURCE_MODE=fixture | production

registry["ir/delta"] = SiteIrDeltaAdapter(...)
registry["fx/delta"] = SiteFxDeltaAdapter(...)
```

The registry should validate at startup that every enabled ProductSpec has risk, Open, and Current legs; expose source version and environment; support bounded concurrency and timeouts; and make fixture mode impossible in a production environment unless explicitly overridden.

Chapter 8

Core domain modules

8.1 `core/__init__.py`

The package initializer is empty. Core modules do not require package-import side effects.

8.2 `core/s01_schema.py`: governed dimensions and canonical column names

8.2.1 Canonical tenor constants

The module defines `Tenor Swap`, `Tenor Option`, and their order-column names. Other modules import these symbols rather than spelling columns independently.

8.2.2 `PortfolioField`

A frozen data class describes each governed portfolio dimension:

- internal key used in component IDs and stores;
- external CSV/DataFrame column name;
- display label;
- whether it is a default/signoff field;
- whether it is required; and
- optional allowed values.

The registry includes Product, Activity, Signoff Group, Category, and optional Sub Category.

8.2.3 Derived registries

Maps and tuples are derived once: fields by key, required configuration columns, optional metadata columns, and the authoritative P&L signoff column.

8.2.4 Import-time validation

The module checks for duplicate keys and external names, exactly one default/signoff field, and coherent allowed values. A malformed schema fails during import rather than producing ambiguous UI IDs later.

8.2.5 Why central registry beats repeated constants

The same dimensions drive feed validation, filtering, saved views, P&L governance, UI labels, and tests. A single registry prevents one page from calling a field `SignoffGroup` while another silently expects `Signoff Group`.

8.2.6 Limitation

Import-time custom validation is clear but not as expressive as a declarative schema library. If the registry grows, consider a typed model that can also emit JSON Schema and documentation.

8.3 `core/s02_pipeline.py`: ProductSpec catalogue, validation, refresh manager, and transaction engine

This is the central domain file and the most important module in the repository.

8.3.1 Column constants and error types

The opening section defines canonical names for Risk, dRisk, P&L, dates, source identity, market fields, mapping status, threshold flags, split labels, status text, and provenance.

Dedicated exceptions distinguish production integration gaps, validation failures, refresh contention, and stale revision requests. The UI translates these exceptions into user-facing status without treating every failure as an unexpected crash.

8.3.2 Protocols and source types

Protocols describe governance loaders, product adapter legs, overlays, and related callables. The manager accepts callables rather than importing feed implementations, preserving testability.

8.3.3 `AxisSpec`

An axis describes a tenor value column and its market-owned order column. Freezing the object makes the product catalogue immutable.

8.3.4 `ProductSpec`

A ProductSpec declares:

- internal key;
- source type;
- Risk Type;
- Risk Greek;
- zero, one, or two market axes;
- P&L formula;
- quote and gamma scaling metadata;
- development step for Taylor Gamma;
- and UI/business identity.

Validation ensures unique keys, unique source types, and unique Risk Type/Risk Greek pairs. The calculation engine therefore performs generic operations from metadata.

8.3.5 Product catalogue

The pinned catalogue includes:

- FX Delta, Gamma, and Vega;
- IR Delta, Gamma, DeltaVega, XCCY, XCCYVega, Inflation, InflationVega, Basis, and Bond;
- Credit Delta and Vega; and
- Commodity Delta and Vega.

One-dimensional products use `Tenor Swap`. True surfaces use both `Tenor Swap` and `Tenor Option`. IR Gamma and Credit Vega are intentionally modeled as curves, not arbitrary fixed grids.

8.3.6 Formula metadata

The catalogue selects among identity, absolute move, percentage move, and Taylor Gamma. Gamma metadata includes `gamma_move_scale` and `gamma_risk_step`, so the generic formula does not branch on product names.

This is a good alternative to one function per product when the differences really are contract metadata. A separate strategy object would become preferable if products acquire substantially different join, interpolation, or scenario semantics.

8.3.7 Primitive validation helpers

The module contains reusable checks for:

- scalar dates;
- exact market status;
- finite multipliers;
- exact DataFrame type;
- required columns;
- unexpected columns;
- blanks;
- text;
- booleans in numeric fields;
- finite numeric conversion;
- exact identity;
- duplicate position or quote keys;
- tenor/rank consistency; and
- complete optional Credit measure pairs.

Validation normally copies source data before normalization.

8.3.8 Risk validation

`get_product_risk` resolves a DataFrame or callable, verifies required identity and value columns, accepts only canonical optional metadata, normalizes numeric columns, enforces one row per product position key, and stamps product/source identity.

Connector-owned `Group` is required but not restricted to a hard-coded vocabulary. Optional Credit Region is preserved.

8.3.9 Market Open and Current validation

Open and Current are validated separately. Current must carry or be attached to the exact requested `Live` or `OFFICIAL` status. Market quote keys must be unique and rank columns must map each tenor label consistently.

8.3.10 Market merge

The two quote legs are outer-joined. Rows are classified as available, missing Open, missing Current, or otherwise incomplete. Move is calculated only from the two numeric legs.

The code rejects rank disagreement rather than choosing one leg as authority. This is correct: different ranks for the same quote identity indicate source inconsistency.

8.3.11 Risk-to-market join

Risk positions are many-to-one joined to the MarketBook. A position without a quote remains present with `No matching market row` and unavailable P&L. Results are sorted by Underlying, market-owned ranks, and Portfolio.

8.3.12 P&L calculation

`_pnl_move` selects identity, percentage, or raw absolute movement. `get_product_pl` applies the multiplier and masks unavailable rows.

Taylor Gamma receives special post-processing because it emits both a sourced Gamma row and a derived Delta exposure row. The derived row is omitted when market is unavailable and does not manufacture dRisk.

8.3.13 Dashboard tenor completion

`_with_dashboard_tenors` supplies presentation placeholders for products with no explicit axis and creates nullable order columns when market ranks are absent. It does not pretend that Risk owns those ranks.

8.3.14 Supplemental Credit SP01

Cross Gamma and New Trades Credit Delta rows use generic Risk/dRisk columns. The helper maps their generic sensitivity into SP01 for the Credit UI and leaves unavailable dRisk as NaN rather than zero.

8.3.15 Portfolio configuration

The manager loads exact required portfolio columns and optional metadata. Portfolio identity must be unique. Product values are governed. Rows in financial data receive mapping status after calculation.

Unmapped rows are retained. This is a major correctness strength: totals can explicitly exclude unmapped data while remediation views still show it.

8.3.16 Thresholds

Threshold rows are exact Risk Type/Risk Greek keys with finite P&L, Risk, and dRisk values. Unknown or duplicate keys fail. Thresholds feed promotion and status logic after calculation.

8.3.17 RiskChecker readiness and inventory

Readiness can be partial. Known missing ProductSpec pairs are completed with Age zero and an `Age Defaulted` marker. Unknown identities, invalid ages, and ambiguous booleans fail.

Checker inventory is separate and uses an exact MMM file and Product schema. Turning RiskChecker off changes the committed setting and related inventory behavior without discarding the product catalogue.

8.3.18 Date helpers

`market_date_for`, `checker_date_for`, and `risk_date_for` are pure and tested. They normalize timezone/date-like values and reject ambiguous Age values.

The use of pandas `BDay` is deterministic and easy to test. It is not holiday-aware.

8.3.19 Snapshot data classes

The module defines immutable control, refresh, compact, health, and progress snapshots. The full financial snapshot contains committed DataFrames and metadata. Compact/control readers return only what a UI operation needs, reducing accidental coupling and serialization.

8.3.20 Locks and state ownership

`RiskRefreshManager` owns three separate synchronization concerns:

- `_refresh_lock`: long-lived exclusive writer ownership;
- `_state_lock`: short critical sections around committed state and metadata; and

- `_progress_lock`: progress updates readable while source calls are running.

Separating these locks is excellent. A reader does not block for the entire connector transaction, and progress polling does not need the state lock.

8.3.21 Lazy constructor

The constructor validates callable configuration and delay values but does not execute sources. This property supports fast imports, first-paint startup, and test app construction.

8.3.22 Progress model

The manager records stage, source, product, Underlying, current/total counts, start time, attempt IDs, revision, status, and failure text. Stage delays can be injected in tests and demos, but invalid booleans or non-finite delays are rejected.

8.3.23 Serial connector loops

For each source type, Risk is normally loaded once. Market Open and Current are called once per required Underlying.

The code deliberately keeps calls simple and traceable. However, independent Underlyings are serial, so startup time grows with product count and source latency.

Recommended improvement: introduce a bounded connector executor with:

- per-source concurrency caps;
- deterministic result ordering;
- hard per-call deadlines implemented in a killable process or source-native timeout;
- circuit breakers;
- cancellation when a refresh becomes stale; and
- preserved progress/provenance for each task.

Do not simply add an unbounded thread pool. Private clients may not be thread-safe, and a Python timeout cannot terminate a blocked C extension or network call.

8.3.24 Refresh request normalization

The manager validates forced dates, expected revision, setting changes, and explicit refresh scope before work begins. Stale browser requests are rejected early.

8.3.25 Refresh planner

The planner compares requested state with the committed snapshot and decides what can be reused. Examples:

- portfolio-only change can remap committed calculated rows;
- P&L refresh can recompute from committed Risk and MarketBook;
- a changed Risk date reloads only affected products;
- a changed market date reloads required quote legs;
- setting toggles can produce metadata-only commits when financial frames are unchanged; and
- a full reload discards all reusable product caches.

8.3.26 Overlay planning

Cross Gamma and New Trades can require market identities absent from ordinary Risk. The planner loads and validates overlays before final quote scope, then combines requested identities with the normal set.

This ordering is subtle and correct. Loading quotes first would make supplemental rows fail or silently remain unavailable even though a quote exists.

8.3.27 Candidate construction

The manager builds per-product validated Risk and MarketBook frames, calculates P&L, appends supplemental rows, attaches portfolio metadata, applies thresholds, attaches reported underlyings, and creates dashboard/search data.

Reusable committed frames are copied into the candidate where the plan permits.

8.3.28 Final invariants and commit

The candidate is checked for schema, source, date, status, and revision coherence. Search indexes are built before publication. Under the state lock, the manager checks that the base revision is still current and replaces the committed snapshot and search catalog together.

8.3.29 Failure semantics

Any source or validation failure leaves the old snapshot in place. The error appears in progress and health. This is safer than updating tables product by product.

8.3.30 Strengths

- metadata-driven product catalogue;
- exact schemas and identity;
- explicit missing-market status;
- fail-closed financial calculations;
- immutable revisioned snapshots;
- single writer with concurrent readers;
- optimistic stale protection;
- selective refresh planning;
- overlays included before market scope;
- search catalog committed with data; and
- rich tests.

8.3.31 Main limitations

- the file is too large;
- refresh-plan rules are difficult to reason about locally;
- connector calls are serial;
- no governed holiday calendar;
- watchdog cannot cancel blocked calls;
- in-memory authority requires one process;
- extensive copying can be expensive on large frames;
- local provenance is rich but not exported to centralized telemetry; and
- some helper boundaries are private methods on one manager rather than named services.

8.3.32 Refactoring path

Preserve behavior and extract in this order:

1. `ProductValidator` pure functions remain unchanged.
2. `RefreshRequest` and `RefreshPlan` become explicit frozen data classes.
3. A pure `plan_refresh(previous, request)` function receives no manager locks.
4. `ConnectorExecutor` owns source calls, deadlines, concurrency, and progress.
5. `CandidateBuilder` owns reuse, calculation, governance, and index construction.
6. `SnapshotStore` owns revision compare-and-swap and readers.
7. `RiskRefreshManager` becomes a thin orchestrator.

This path is safer than a rewrite because every extracted stage can be checked against the existing tests.

8.4 `core/s03_search.py`: immutable Quick Search indexes and pivots

8.4.1 Search identity

A combined identity is normalized from Risk Type, Risk Greek, and Underlying. Labels have bounded lengths and a controlled separator. Exact identity maps are distinct from searchable normalized labels.

8.4.2 `SearchCatalog`

The immutable catalogue stores compact frames and indexes for a specific committed revision. It includes exact identity-to-row-position mappings and pre-normalized option labels.

8.4.3 Option search

Typed text is case-folded and tokenized. A candidate option matches when the normalized label contains the requested tokens. Results are bounded. This scan operates over option labels, not every risk row.

8.4.4 Exact selection

Dropdown search is permissive; pivot selection is strict. Passing typed text directly to an exact pivot returns empty instead of guessing which identity the user meant.

8.4.5 Risk pivot

Risk values are additive and can be grouped by selected hierarchy columns. Reported or raw Underlying mode can be selected.

8.4.6 Market quote handling

Portfolio positions can repeat one quote many times. The search module therefore builds a separate quote identity and maps risk rows to unique market rows. Market averages are calculated from unique quotes, not from position-expanded rows.

When rolling up a market parent, only complete Open/Current pairs contribute. An Open-only or Current-only child remains visible at the leaf but does not distort the parent average.

8.4.7 Tenor rank authority

The module resolves market-owned rank maps and rejects ambiguous mappings. Pivots sort by those ranks and only use a documented fallback when authority is unavailable.

8.4.8 Compact arrays

Risk-to-quote mappings and row positions use compact integer arrays. This reduces memory and speeds exact slicing.

8.4.9 Strengths

- no live connector calls;
- exact selection separated from fuzzy option search;
- no row-level posting index explosion;
- market quote deduplication;
- complete-pair rollups;
- immutable revision coherence; and
- bounded dropdown output.

8.4.10 Limitations

Option search is still O(number of labels). At the current scale that is simpler and likely faster than a complex index. For hundreds of thousands of identities, use a prefix/token index or a dedicated search service.

The label separator and formatting are part of an implicit public identity. A structured value object would avoid parsing or collision concerns.

8.5 `core/s04_pl.py`: governed P&L, history, overlays, and period analysis

8.5.1 Canonical columns and labels

The module defines send keys, historical schemas, type labels, period labels, and mapping status. Compatibility aliases normalize older labels such as Histo/Predicted to user-facing Colossus/Predict.

8.5.2 Strict input loaders

Historical P&L and legacy history leaves require exact column order, numeric P&L, unique daily keys, valid dates, and known type labels.

8.5.3 Directory-signature cache

Historical readers cache parsed CSVs using file and directory signatures. Unchanged files are reused; changed files invalidate the relevant cache.

8.5.4 `build_pl_send_base`

This is the governance core. It:

1. validates calculated dashboard rows;
2. excludes supplemental source P&L that is intentionally zero/non-sendable where required;
3. maps each Risk Type/Risk Greek pair to one Concerto field;
4. joins Portfolio to Signoff Group authority;
5. rejects ambiguous or missing governed mappings for sendable rows;
6. collapses rows to the exact send grain; and
7. returns a canonical frame.

8.5.5 Adjustment overlay

`apply_adjustment_overlay` aligns adjustment rows to the same key and replaces the base P&L. It validates that the adjustment cannot introduce an unknown key or inconsistent governance.

8.5.6 Historical series and periods

The module can select a hierarchy path and aggregate once per observed day/type. It does not fill missing dates with zero. Daily uses the global or explicit as-of date; WTD begins Monday, MTD begins calendar month, and YTD begins calendar year.

Missing types remain missing. This is important: zero would assert “the system observed a value of zero,” while NaN/absence says “no observation.”

8.5.7 Strengths

- send governance is independent of Dash;
- one canonical send grain;
- replacement semantics are explicit;
- strict history types and keys;

- no fabricated missing daily values;
- cache invalidation is tested; and
- legacy and official archive readers can be combined behind one boundary.

8.5.8 Limitations

The directory signature can be expensive for a very large historical tree. Use a manifest or catalog database at scale.

The local history model assumes file-based dates and exact CSVs. A production warehouse adapter should implement the same canonical output contract.

8.6 `core/s05_storage.py`: local CSV adjustment repository

8.6.1 Path naming

Adjustment files are derived from a normalized scope name plus a hash. This avoids unsafe path characters and reduces collisions.

8.6.2 Exact persisted schema

Files use one exact column order. Reads validate dates, text, numeric values, booleans, and duplicate keys.

8.6.3 Revision and stale writes

The repository calculates a revision/fingerprint. Saves can require an expected revision so a browser cannot overwrite a file changed since it was loaded.

8.6.4 Atomic replacement

Writes use a temporary file, flush and fsync, optional backup/rescue behavior, and atomic replacement under an in-process reentrant lock.

8.6.5 Strength

A crash should not expose a partially written CSV, and stale UI edits are detected.

8.6.6 Important limitation

Unlike saved views, this repository does not use an OS-level file lock. Multiple processes writing the same adjustment file can race. The current Gunicorn policy uses one process, so the limitation is aligned with deployment. It must be fixed before multi-process or shared filesystem deployment.

Also fsync the containing directory after rename when strong local durability is required.

8.7 `core/s06_reporting.py`: raw-to-reported Underlying mapping

The module validates an exact mapping from `(Risk Type, Risk Greek, Underlying)` to one or more reported labels. It then attaches reporting rows after calculation.

One raw identity may map to several reported identities, which intentionally duplicates reporting rows for alternate views. The mapping is not recursive; a reported label is not looked up again.

8.7.1 Why post-calculation attachment is correct

Market identity and P&L should be calculated once on raw canonical Underlying. Reporting labels are aggregation/display governance, not quote keys.

8.7.2 Limitation

One-to-many reporting can make users double-count if exported without a clear reporting-mode flag. Exports should include raw identity and mapping status.

8.8 `core/s07_stock.py`: Stock validation, comparison, filtering, and hierarchy

8.8.1 Source schema

The Stock contract defines exact text and numeric columns such as position/security identity, counterparty, portfolio/book, description, currency, notional, and market value.

8.8.2 Validation

The module rejects schema drift, blanks in required identities, booleans and non-finite numbers, and duplicate source grain.

8.8.3 Current/prior comparison

Two validated snapshots are outer-joined on exact position identity. Current and prior values remain separately visible, with movement calculated after the join. New, closed, and continuing positions can therefore be distinguished.

8.8.4 Governance mapping

Portfolio metadata is attached after source comparison. Unmapped rows remain explicit.

8.8.5 Filtering and thresholding

Filters use the same include/exclude semantics as other pages. A default absolute current market-value threshold controls promoted hierarchy rows.

8.8.6 Lazy hierarchy

Path tokens represent open nodes. The renderer emits only visible children and returns the effective open set.

8.8.7 Provisional grouping

Currency supplies a temporary grouping layer. The UI labels it as provisional rather than claiming it is an authoritative taxonomy.

8.8.8 Improvement

Inject a governed Stock hierarchy schema and keep the current currency layer as a documented fallback only in fixture mode.

8.9 `core/s08_saved_views.py`: durable small JSON view documents

8.9.1 Name normalization

View names are trimmed, length-bounded, slugged, and checked for duplicates. Filter values are normalized to bounded lists of nonblank strings.

8.9.2 Versioned document

Each JSON document has a schema version, name, saved values, exclude mode, and related metadata. It stores control state, not financial data or a snapshot revision.

8.9.3 Locking

The module combines:

- an in-process lock;
- an OS-level file lock; and
- atomic file replacement.

This is stronger than the adjustment repository for multi-process file safety.

8.9.4 CRUD behavior

List, get, save, update, and delete all validate the entire document. Corrupt files fail closed. Temporary files do not become visible as saved views.

8.9.5 Limitations

There is no per-user ownership, authentication, authorization, audit trail, or migration beyond the current schema version. Names are global within a scope. Production should store views in an authenticated shared database with owner and visibility fields.

8.10 `core/s09_cross_gamma.py`: matrix validation and supplemental calculation

8.10.1 Raw matrix contract

Each row identifies:

- Portfolio and Group;
- input Risk Type/Greek/Underlying/tenors;
- Cross Gamma kind (`XGamma` or `XGamma Vega`);
- output Risk Type/Greek/Underlying/tenors; and
- sensitivity.

Text, product pairs, axes, blanks, and finite sensitivity are validated.

8.10.2 Market scope

Both input and output product identities can be added to quote scope. This ensures the calculation has the required market movement and can present output context.

8.10.3 Calculation

Input market is mandatory. The contribution is sensitivity times the appropriate input market move convention. Source rows preserve audit metadata and have P&L zero. Contributions are aggregated to canonical output Risk rows.

Output market can be absent; calculated output risk is retained with unavailable market/P&L rather than discarded.

8.10.4 Why source rows remain

A pure aggregate would be smaller but would lose explainability. Retaining source matrix rows allows a detail view to show which input cells contributed to an output exposure.

8.10.5 Limitations

The output risk is calculated, but output P&L is not automatically modeled as an ordinary aged position. That is a defensible boundary but should be documented to users.

Product lookups are deferred in places to avoid import cycles. Extracting a small product registry interface would make dependencies clearer.

8.11 `core/s10_new_trades.py`: intraday overlay calculation

8.11.1 Canonical row types

The module consumes the validated raw adapter output. CASHFLOW rows already carry identity P&L. MARKET rows require a declared product identity, Risk, and traded-level semantics.

8.11.2 Market identity and quote join

A MARKET row is mapped to a ProductSpec, normalized to its exact axes, and many-to-one joined to the MarketBook. If `Traded True` is false, Open becomes the trade level. Otherwise the supplied traded level is used.

8.11.3 Formula application

The same percentage, absolute, or Taylor metadata used by aged risk is applied from traded level to Current. This prevents New Trades from implementing a second, subtly different P&L engine.

8.11.4 Trace metadata

Trade ID, position ID, trade time, trader, notional, traded-level source, and calculation status remain attached for detail views.

8.11.5 Cash flows

Cash-flow P&L bypasses market calculation and uses the supplied amount exactly.

8.11.6 Strengths

- one formula authority;
- exact row-type semantics;
- market scope expanded before calculation;
- complete audit metadata; and
- failed/unsupported rows remain explicit.

8.11.7 Improvement

Separate adapter validation DTOs from dashboard output DTOs. The current wide DataFrame carries fields that are relevant at different stages and can be difficult for an integrator to understand.

8.12 `core/s11_risk_archive.py`: official immutable daily archive

8.12.1 Schema versions and file names

The current archive version uses exact `risk.csv`, `colossus.csv`, `market.csv`, and `_SUCCESS` files. Legacy version 1 remains readable where supported.

8.12.2 Risk projection

The archive stores canonical calculated risk/P&L rows and governed metadata required for later validation. It rejects partial financial arithmetic and invalid identities.

8.12.3 Market projection

Market rows retain unique exact quote keys, Open, Current, Move, status, availability, axes, and market-owned ranks.

8.12.4 Portfolio authority

Archived portfolio metadata supports later mapping of Colossus rows into hierarchy. Ambiguous authority is not guessed; those rows become Unmapped in validation views.

8.12.5 Manifest

The success document records schema version, date, status, exact files, row counts, hashes, and other metadata. Only OFFICIAL completed leaves are eligible.

8.12.6 Write protocol

A hidden pending directory is written and validated. `_SUCCESS` is written last. An atomic directory rename publishes the leaf. Existing completed dates are idempotent and not overwritten.

8.12.7 Read protocol

A completed leaf must pass:

- exact manifest schema;
- exact allowed file set;
- SHA-256 for each file;
- row-count agreement;
- exact CSV schemas;
- natural date and OFFICIAL status;
- unique market quote identity;
- rank consistency;
- Market Move arithmetic;
- P&L arithmetic where applicable; and
- governed mapping invariants.

8.12.8 Caching

Readers use file fingerprints so unchanged official leaves can be reused.

8.12.9 Strengths

This is a robust local immutable-data protocol and one of the best-engineered parts of the repository.

8.12.10 Limitations and production path

- local rename semantics vary across filesystems;
- no object-store generation/ETag protocol;
- no retention, compaction, or catalog service;
- no encryption or access control in the module;
- re-reading files to hash them adds I/O;
- weekday eligibility is not holiday-aware; and

- concurrent schedulers still rely on filesystem behavior.

For object storage, write versioned objects, validate hashes, then atomically publish a small catalog pointer or transaction record. Readers should list only catalog-approved versions.

Chapter 9

User-interface architecture

The `ui/` package converts committed domain snapshots into Dash components and converts browser actions back into small, validated state transitions. This distinction is important. The UI does **not** own the financial truth. It owns presentation state such as which hierarchy rows are open, which metric is expanded, which saved view is selected, or which date a user has drafted.

A beginner-friendly way to read the package is:

1. `s01_contracts.py` describes what services the UI is allowed to call.
2. `s02_constants.py` gives stable names and ordering rules to the interface.
3. `s03_aggregate.py` turns canonical rows into summaries.
4. `s04_components.py`, `s05_staticdata.py`, `s06_plview.py`, `s10_stock.py`, `s12_plhistory.py`, and `s13_validate_pl.py` build visual trees.
5. `s07_events.py`, `s08_plevents.py`, `s11_saved_views.py`, and parts of `s09_factory.py` register the callbacks that mutate presentation state.
6. `s09_factory.py` composes the Dash application and binds it to Flask.

9.1 `ui/__init__.py`

The package initializer is intentionally empty. Importing `ui` alone performs no registration and creates no Dash application. That avoids an important class of test pollution: callbacks and pages are process-global inside parts of Dash, so registration must happen only when a particular app is being built.

9.2 `ui/s01_contracts.py`: dependency boundaries expressed as protocols

This module defines runtime-checkable `Protocol` types for the refresh manager, committed snapshots, adjustment storage, saved-view storage, and history readers. A protocol says, in effect, "the UI needs an object with these methods and attributes"; it does not require that object to inherit from one particular class.

9.2.1 Why this is useful

- Unit tests can supply small fakes rather than constructing the full financial pipeline.
- The UI cannot casually reach into private manager implementation details without the type contract changing.
- Storage can later move from local files to a database while preserving the callback-facing interface.
- Page builders can be recreated repeatedly because they receive services rather than importing one global singleton.

9.2.2 Limitation

Python protocols provide design-time and optional runtime shape checks, not a security boundary. A structurally compatible object may still have different semantics. High-value operations therefore still validate returned DataFrames and revision numbers.

9.2.3 Improvement

Keep these protocols narrow. Where one protocol has accumulated unrelated methods, split it into read, refresh, progress, and administrative capabilities. That makes accidental coupling visible and lets permissions follow capability.

9.3 `ui/s02_constants.py`: the UI vocabulary

This file centralizes values that would otherwise be copied through component and callback modules:

- Portfolio filter definitions and aliases.
- Risk hierarchy dimensions.
- Metric names and display order.
- Risk-type and Greek order.
- Credit measures such as SP01, PSP01, PM01, PM01P, Theta, and JTD.
- Row-toggle glyphs and pattern-matching component types.
- Interest-rate family groupings.
- Canonical display and validation maps.

Centralizing these values is preferable to scattering string literals because Dash component IDs and DataFrame column names are contracts. One misspelling can silently disconnect a callback from its component or create an empty group.

The trade-off is that this module can become a "miscellaneous constants" drawer. A cleaner next step would group constants into frozen namespaces or dataclasses: `RiskIds`, `PnlIds`, `HierarchyRules`, and `DisplayLabels`. That would retain one source of truth while making ownership clearer.

9.4 `ui/s03_aggregate.py`: canonical rows to visible hierarchy

This module contains mostly pure functions. It prepares a committed snapshot for display, applies risk-page filters, recomputes promotion flags, aggregates metrics, emits deterministic row keys, and formats numbers.

9.4.1 Preparation

The preparation path:

1. Requires the canonical financial columns expected by the UI.
2. Normalizes text used for grouping without changing the financial values.
3. Preserves source identity, market availability, and mapping status.
4. Attaches display-only fields where required.
5. Keeps missing values missing. In particular, unavailable `dRisk` or P&L is not silently converted to zero.

9.4.2 Filtering

Within one selector, selected values are an OR: selecting Books A and B means "A or B." Across populated selectors, include mode is an AND: "Book A or B, and Activity X." Exclude mode instead removes a row when it matches any populated selector. This asymmetry is deliberate because an exclusion control is usually read as a blacklist.

Filters operate on normalized case-insensitive values but return copies of the original rows. That means display spelling remains intact.

9.4.3 Promotion and thresholds

Promotion is recalculated after filtering rather than trusted as a static input. This is necessary because threshold visibility depends on the scope the user is looking at. The module distinguishes threshold-based prominence from financial calculation; it never changes Risk, `dRisk`, or P&L to make a row prominent.

9.4.4 Credit measures

Credit Delta can expose several complete Risk/`dRisk` measure pairs. The aggregation code only presents measures actually supplied by the canonical source contract. Supplemental Cross Gamma and New Trades rows may populate the generic Credit Delta sensitivity as SP01, but missing companion measures remain unavailable.

9.4.5 Hierarchy

A hierarchy row key encodes its dimension path rather than relying on the visible label. This prevents ambiguity when two branches contain the same label. Parent rows aggregate children with `min_count` semantics so all-missing input remains missing instead of becoming zero. Sorting uses business orders for Risk Type, Greek, market tenor ranks, and deterministic case-insensitive fallbacks.

9.4.6 Why pure aggregation is the right design

The same functions can be tested without a browser, reused in multiple views, and reasoned about independently of callback timing. The alternative - putting groupby logic directly inside each callback - would duplicate business rules and make stale-state defects much harder to isolate.

9.4.7 Improvement opportunities

- Cache expensive grouping by `(revision, filter fingerprint, metric set)`.
- Introduce an explicit `PreparedRiskView` dataclass instead of passing several loosely related frames and maps.
- Separate row-key encoding from HTML concerns into a small versioned codec.
- Add property-based tests asserting that totals equal visible leaves under arbitrary filters and missing-value patterns.

9.5 `ui/s04_components.py`: the main component library

This is the largest presentation module. It is best read as a collection of component families rather than one algorithm.

9.5.1 Risk table builders

The table builders produce semantic HTML tables rather than a callback per cell. Rows carry stable data attributes for hierarchy keys, metrics, source, and the current view-generation token. Buttons are used for actions so keyboard users receive native focus and activation behavior. The browser JavaScript delegates clicks from these buttons to a few `dcc.Store` action envelopes.

The component tree distinguishes:

- Index cells from numeric metric cells.
- Total rows from group rows and leaves.
- Main Risk from alternate/detail views.
- Ordinary metrics from Credit measures.
- Available market data from unavailable values.
- Promoted rows from ordinary rows.

A major benefit is scale: a table with thousands of actionable cells does not register thousands of Dash callbacks. The cost is that HTML data attributes, JavaScript envelopes, and Python reducers must remain synchronized.

9.5.2 Quick Risk and Quick Market

Quick Risk uses the committed search catalog's exact Risk identity index. Quick Market uses the full MarketBook, including market-only tenors not present in portfolio Risk. Their components therefore look similar but intentionally query different grains.

Quick Market surface controls expose one- and two-axis products without assuming a fixed grid size. Market-owned rank columns determine display order.

9.5.3 Top Book, alternate views, and detail disclosures

Secondary panels reuse the same canonical preparation and formatting rules. Details are generally native `<details>` elements or lazily materialized children. This avoids loading large audit tables until a user asks for them.

9.5.4 Unmapped rows

The unmapped-book disclosure is not hidden. It presents rows excluded from mapped totals, explains why, caps the first display at 2,000 rows, and adds native DataTable filtering, sorting, and pagination. This is a strong governance choice: bad mappings remain visible without contaminating approved totals.

9.5.5 Control strip and operating dates

The shared control strip includes refresh actions, source toggles, automatic P&L state, theme control, and the committed Market/Risk date cards. The date cards read the current snapshot; they do not display a user's uncommitted Force Date draft as though it were live.

9.5.6 Refresh hero and cold-start shell

The cold shell can render with `snapshot=None`. Its controls are disabled, but the header, status, progress stages, retry affordance, and accessible loader are present. This lets the browser paint before connectors run.

The progress hero has stable IDs for:

- Current function.
- Source and Underlying.
- Product count.
- Deliberate hold information.
- Overall progress bar.
- Readiness, Risk, market, P&L, and commit stage rows.

JavaScript updates these nodes from `/progressz` while Dash owns the authoritative refresh request and completion signal.

9.5.7 Accessibility

The component library uses captions, `scope`, `role`, `aria-expanded`, `aria-pressed`, live regions, focusable buttons, keyboard shortcut metadata, and screen-reader-only labels. Reduced-motion behavior is completed in JavaScript and CSS.

9.5.8 Why the module is designed this way

A shared component library keeps markup, data attributes, and accessibility semantics consistent across callbacks. The alternative - constructing ad-hoc HTML in each event function - would make browser delegation and styling brittle.

9.5.9 Main limitation and refactor

At more than two hundred kilobytes, the module has become a second composition root. Split it by stable domain:

- `risk_tables.py`
- `quick_search.py`
- `refresh_shell.py`
- `filters.py`
- `unmapped.py`
- `shared_controls.py`

The public builders can be re-exported temporarily so the refactor does not force a flag day.

9.6 `ui/s05_staticdata.py`: safe fixture inspection

The Static Data page exposes only an allowlisted set of the nine root fixture CSVs. It does not accept an arbitrary path from the browser. The loader resolves the path, verifies membership in the allowlist, reads values as text for faithful inspection, and returns a DataTable with native filtering, sorting, paging, and a fixed header.

This is safer than joining a user-supplied filename to `data/`, because path normalization alone is easy to misuse. The remaining risk is that the allowlist must stay synchronized with the fixture catalogue. A generated allowlist from the fixture manifest would remove that duplication.

9.7 `ui/s06_plview.py`: P&L page components

This module builds the P&L Explorer page and editor controls. It keeps the display hierarchy, editable adjustment tables, send controls, history explorer, and validation panel visually related while their domain rules remain in `core/s04_pl.py`.

9.7.1 Filters

The page has one authoritative set of Activity, Signoff Group, Portfolio, Category, and Sub Category filters. Saved views and every P&L subsection refer to these same controls. That prevents one panel from appearing filtered while another silently sends an unfiltered scope.

9.7.2 Aggregate hierarchy

Aggregate rows use stable IDs and explicit selection metadata. A user can choose a Signoff Group or Portfolio scope, then materialize the corresponding editor. The editor does not become the source of truth; it is a view over a revisioned base plus local changes.

9.7.3 Editor tables

Editable columns are governed. Identity columns and calculated base rows are read-only; approved adjustment values can be changed. Stable row IDs allow Dash `Patch` updates and let late browser events be reconciled to the correct row.

9.7.4 Send panels

The page distinguishes sending all governed rows, a Signoff Group, or one Portfolio. Each action has a status area and uses the same stale-revision, filter, and market-date checks in the event layer.

9.7.5 History and Validate P&L

Historical Colossus/Predict analysis and archive validation are separate sections. The former examines time series; the latter compares one completed official archive at its governed hierarchy.

9.8 `ui/s07_events.py`: Risk and shared callbacks

This module is the main Risk-page event controller. It registers both small callbacks and one deliberately large reducer/render callback. The code generally follows a state-machine pattern:

1. Read the component that triggered the request through `dash.ctx`.
2. Validate the action and its view-generation token.
3. Normalize browser-owned state.
4. Read the current immutable snapshot.
5. Produce new presentation state and rendered children together.
6. Return `no_update` or raise `PreventUpdate` for irrelevant transitions.

9.8.1 Startup coordinator

`StartupCoordinator` owns the process-wide cold-start attempt. A browser posts to `/startz`; one daemon thread claims the attempt and calls the manager. A watchdog may report that the writer appears stalled, but it deliberately never starts another financial writer. Attempt IDs, boot IDs, and progress timestamps let a replacement browser or worker distinguish a current attempt from stale progress.

This is preferable to running source I/O during module import because:

- The HTTP worker can start and return a shell.
- Health probes can succeed independently of connector latency.
- Importing the app in tests performs no financial work.
- Only an explicit coordinator owns the first write.

9.8.2 Shared-shell hydration

Once revision 1 exists, a callback replaces cold-shell content with the committed dashboard controls. It carries the revision into the page-local `data-revision-store`, allowing downstream callbacks to render the exact snapshot.

9.8.3 Filter option synchronization

Dimension options are derived from the current committed data and the other active filters. Existing selections are pruned only when no longer valid. Saved-view application arrives as a request envelope and is acknowledged only after the target filter values have actually appeared in the controls.

9.8.4 Main Risk reducer and renderer

`reduce_and_render_risk_view` is the most important UI callback. It combines several transitions that were previously separate:

- Open or close a hierarchy branch.
- Select or clear a metric cell.
- Expand or collapse a metric group.
- Change Risk Type/family/dimension.
- Apply filter or display settings.
- Reconcile state after a new data revision.
- Render the main and alternate tables.
- Return pruned open-row and selection stores.
- Update view tokens and visible-state metadata.

The browser action contains a sequence number, source, hierarchy key or metric, and a `view_token`. The callback rejects an action generated by an older render. This prevents a common Dash race: a user clicks a node just as another callback replaces it, and the late request mutates the new view.

Combining state reduction and rendering saves a full request-response cycle. A separate “update Store” callback followed by a “render from Store” callback would be conceptually smaller but slower and more exposed to intermediate stale states.

The cost is maintainability. A single callback with many Inputs, States, and Outputs is difficult to review. A better internal structure is:

```
parse_trigger -> validate_action -> reduce_view_state
              -> load_snapshot -> build_view_model -> render_outputs
```

Each arrow should be a pure typed function. The Dash-decorated wrapper can remain one network round trip while the code becomes testable in small units.

9.8.5 Quick Search callbacks

Quick Risk and Quick Market each separate option search from exact pivot rendering. Text entered into the drop-down scans pre-normalized labels. A selected value must be an exact catalog identity; free text does not become a data query.

Quick Market also manages axis controls, current quotes, and historical market series. Historical reads are scoped by the selected exact identity and current market axes.

9.8.6 Refresh callback

`refresh_pipeline` is the authoritative user-triggered write callback. Inputs include Refresh Risk, Refresh Portfolios, Refresh P&L, commodity and checker toggles, date actions, automatic refresh ticks, and retry. The function:

1. Determines the actual trigger and requested refresh mode.
2. Reads the current revision expected by the browser.
3. Rejects malformed or stale date drafts.
4. Calls the manager's selective refresh operation.
5. Publishes a new revision only after validation succeeds.
6. Returns status, errors, committed settings, and date information.
7. Uses Dash's `running` outputs to lock refresh controls for the request.

When another browser already owns the manager's refresh lock, the callback does not start a duplicate. It reports that it is following the live writer. The browser progress controller then polls the same process state.

9.8.7 Force-date callbacks

Draft Force Market and Force Risk values are not immediately committed. Callbacks manage draft editing, validation, rebase after revision changes, apply, and cancel. This separation is essential: merely opening a date picker must never change financial data.

9.8.8 Automatic P&L

The AutoPL toggle controls periodic P&L refresh requests. The UI interval is only a trigger; the manager planner decides whether a P&L-only refresh is valid and the manager lock still serializes the transaction.

9.8.9 Lazy and client-side callbacks

Unmapped data and checker inventory are loaded only when their disclosures open. Several pure visibility changes - such as IR family controls, Credit controls, and main-versus-alternate grid visibility - are client-side callbacks because they do not need server data.

9.8.10 Error handling

Expected stale or invalid UI transitions use `PreventUpdate`/`no_update`. Connector and validation failures become user-visible status while the last committed snapshot remains available. Catching broad exceptions at the outer callback boundary is reasonable for availability, but the logged error should include a typed incident code and full server traceback while the browser gets a sanitized message.

9.9 `ui/s08_plevents.py`: P&L editor, history, save, and send callbacks

This module owns P&L-page event behavior. `PLSendConfig` packages IDs, labels, scope rules, and service dependencies so the Signoff Group and Portfolio editor families can share one implementation.

9.9.1 Aggregate selection

The aggregate hierarchy callback applies the page's authoritative filters, normalizes open paths, identifies the clicked aggregate row, and writes the selected governed scope. A new data revision rebuilds the hierarchy and prunes selection that no longer exists.

9.9.2 Effective editor stores

Each editor has an effective-data Store representing:

```
current calculated base
+ persisted adjustments
+ unsaved browser edits
```

The merge is keyed by governed P&L identity, not by table row position. That is why resorting or filtering a DataTable cannot attach an edit to the wrong book.

9.9.3 Editor callback factory

A factory registers parallel Signoff Group and Portfolio callbacks. It handles:

- First materialization of rows for the selected scope.
- Filter/sort/page changes.
- Cell edits.
- Selection changes.
- Late edit events from a table that has just been replaced.
- `Patch` updates where a full table replacement is unnecessary.
- Stable row IDs and revision/date metadata.
- Pruning invalid selected rows.

This is an appropriate use of a callback factory because both scopes share the same state machine but have different IDs and governance predicates. The danger is opaque generated callback names; the report's callback atlas therefore lists the concrete semantic instances.

9.9.4 Save

Saving validates the browser's loaded revision, loaded Market Date, selected scope, and exact row schema. It writes only adjustment rows through `LocalCsvAdjustmentRepository`; calculated rows are rebuilt from the snapshot. A stale write is rejected rather than silently overwriting a newer adjustment file.

9.9.5 Send

All, Signoff Group, and Portfolio send actions use the governed effective frame. Before invoking the external sender they re-check:

- Current committed revision.
- Current Market Date.
- Page filters and selected scope.
- Mapping completeness.
- Exact Concerto identities.
- Absence of duplicate governed keys.
- Finite values where required.

The checked-in sender deliberately refuses delivery in fixture mode. That is a safe default.

9.9.6 P&L history

History callbacks build the expandable hierarchy, period comparisons, selected series, and Plotly chart. The hierarchy and chart share normalized path/period state so a stale cell selection cannot plot a branch no longer visible.

9.9.7 Client-side selection summaries

Small client-side callbacks summarize selected DataTable cells and clear selection Stores. Spreadsheet-like drag behavior itself is implemented in `assets/s03_select.js`.

9.9.8 Improvement

The editor state machine deserves its own typed module independent of Dash. Represent commands such as `EditCell`, `ChangeScope`, `ReloadRevision`, `SaveAdjustments`, and `SendScope` explicitly. A pure reducer can return state plus effects, leaving the callback wrapper to perform storage or sending.

9.10 `ui/s09_factory.py`: app construction and service binding

`build_app` is the presentation composition root.

9.10.1 Dash construction

It creates a Dash app with pages enabled, disables automatic page-folder discovery, enables callback exception suppression for dynamic page layouts, and uses the configured request prefix. Before registration it resets relevant Dash page-registry state so repeated app construction in tests does not inherit a prior instance.

9.10.2 Flask routes

The underlying Flask server exposes:

- `/healthz` for process health.
- `/progressz` for current backend refresh/startup progress.
- `POST /startz` to request the process-wide cold-start attempt.

Routes are prefix-aware. Responses use no-store/cache-control and security headers where appropriate. `/progressz` reports state; it does not perform a financial refresh.

9.10.3 Prepared dashboard cache

Rendered dashboard preparation is cached by committed revision. Since snapshots are immutable, revision is a safe cache key. A new revision creates a new prepared view rather than mutating the old one.

9.10.4 Router and page services

The factory builds the common header/navigation/refresh strip and stores page-builder callables on the active Flask app. Thin modules in `pages/` resolve those builders only when Dash renders a page.

9.10.5 Callback registration

The factory calls the Risk, P&L, saved-view, validation, and Stock registration functions with concrete services and IDs. This is where abstract protocols meet the real manager and repositories.

9.10.6 Stock page-local cache

Stock does not live in the Risk snapshot. A page cache is keyed by committed revision and selected current/prior dates. A nonblocking lock and intent sequence prevent an older, slower date request from replacing a newer user selection.

Stock callbacks:

1. Load/refresh the selected date pair.
2. Build/prune filter options.
3. Filter and render the lazy hierarchy.
4. Load source comparison rows only after an explicit request.

9.10.7 Main limitation

The factory now knows routes, page composition, caching, callback registration, and Stock orchestration. Split it into an app factory, route registrar, page service container, and Stock controller. Preserve `build_app` as the public facade.

9.11 `ui/s10_stock.py`: Stock page model and components

`StockPageData` holds the validated comparison and the dates that produced it. Helpers normalize browser dates, choose business-day defaults, build case-insensitive filters, calculate mapped/unmapped summaries, and encode hierarchy paths as bounded JSON tokens.

The hierarchy is lazy. Only children of explicitly open paths are materialized. Closing a parent removes its descendants from open state. A promotion threshold controls visual prominence, not financial calculation.

Source rows are separated from the summary hierarchy so opening the page does not immediately serialize a potentially large raw table.

The provisional grouping by currency is clearly labeled as temporary. This is better than presenting an inferred taxonomy as official, but production should supply a governed hierarchy field from the Stock connector or mapping table.

9.12 `ui/s11_saved_views.py`: reusable saved-filter controller

A `SavedFilterViewControls` value packages one scope's selector ID, name field, buttons, filter IDs, exclude control, and request Stores. The same callbacks are registered for Risk, P&L, and Stock.

9.12.1 Base view

The base view is synthetic and not persisted. It represents the current un-named filter state. Selecting it restores the request envelope's recorded base filters rather than assuming that "base" means all-empty.

9.12.2 Mutation callback

Save, update, rename, and delete actions call the repository, then refresh the catalog options and status. Named views are immutable values in storage; an update replaces one normalized document atomically.

9.12.3 Apply request

Selecting a view writes an application request containing:

- A unique request ID.
- Target filter values.
- Target exclude state.

- The filter state from which the request began.

Separate control callbacks update the individual dropdowns. An acknowledgement callback marks the request complete only when either the target has been reached or the user has manually superseded the original base state. This prevents a pending request from repeatedly overwriting manual changes.

9.12.4 Action-state callback

The button label becomes “Save New” for the base view and “Update View” for a named view. Delete and name editing are enabled only for named views.

9.12.5 Improvement

Include creator, updater, timestamps, and an optional shared/private scope in the stored schema. Introduce versioned migrations rather than rejecting all future versions.

9.13 `ui/s12_plhistory.py`: expandable historical P&L visualization

Hierarchy path tokens are bounded JSON arrays in governed dimension order. Open-path normalization removes malformed or duplicate browser state and sorts it deterministically. Closing a branch also closes descendants.

The table always uses one global latest valuation date. A stale identity can remain discoverable, but it does not pretend its last observation is today’s Daily Predict value.

Visible periods are:

- Daily Predict.
- MTD Colossus, with an optional adjacent MTD Predict column.
- YTD Colossus, with an optional adjacent YTD Predict column.

MTD/YTD headers toggle comparison disclosure across the table. Clicking a metric cell selects a path, period, and series for the chart. The renderer prunes selection if its path is no longer visible.

Missing observations render as unavailable, not zero. That is financially important because zero means a real measured result.

9.14 `ui/s13_validate_pl.py`: official archive comparison

This module compares one completed official Risk archive with Colossus P&L.

9.14.1 Comparison grain

Predict is aggregated to:

```
Signoff Group
-> Risk Type
-> Risk Greek
-> Underlying
-> Product
-> Portfolio
```

Colossus arrives at a smaller identity. Portfolio authority from the archived Risk snapshot attaches Signoff Group, Product, Activity, Category, and Sub Category only when the Portfolio maps uniquely. Missing or ambiguous authority is retained once as `Unmapped`; it is never copied across Products or guessed.

An outer one-to-one join labels rows as Matched, Predict only, or Colossus only. Unmapped Colossus rows appear in a separate audit disclosure and are excluded from mapped totals.

9.14.2 Callbacks

The first callback discovers completed dates only when the Validate P&L disclosure opens. The second loads/caches the selected archive, applies the authoritative P&L-page filters, updates open hierarchy paths, renders the tree, and reports mapped/matched/unmapped counts.

The cache is process-local and keyed by Market Date. Since completed leaves are immutable by protocol, this is a safe key.

9.14.3 Improvement

Cap or evict the date cache in a long-lived process, and include the archive manifest digest in the key if administrative tooling can replace a completed leaf.

9.15 `ui/s14_pl_filters.py`: pure P&L filtering

This small module owns the five P&L filter IDs and two pure helpers.

`pl_external_filter_map` converts UI-keyed dropdown values into canonical external column names. `apply_pl_filters` uses OR within one selector, include-AND across selectors, or exclude-OR when exclusion is active. It rejects missing source columns instead of treating them as no matches.

Keeping this rule in one pure module is the correct design because aggregate P&L, editors, history, and validation must all show the same scope.

Chapter 10

Dash page modules

The `pages/` package is deliberately thin. Dash page registration is process global, while the refresh manager and repositories belong to one app instance. The files therefore register route metadata but resolve layout builders from the currently active Flask application.

10.1 `pages/__init__.py`

This module contains the service-resolution helper. It reads a known builder from `flask.current_app.config` while a request/app context is active. A missing builder produces an explicit configuration error rather than importing a global manager.

This is a subtle but valuable design choice. It allows tests to create more than one app in the same Python process, and it avoids a page imported for metadata binding itself permanently to the first manager constructed.

10.2 `pages/risk.py`

Registers the Risk Explorer route and delegates `layout()` to the active app's Risk-page builder. The route owns no calculation or callback registration.

10.3 `pages/pnl.py`

Registers P&L Explorer and delegates its layout. P&L callbacks are registered once by the factory; this file only makes the page discoverable.

10.4 `pages/stock.py`

Registers Stock and resolves the Stock builder. Because Stock has page-local dates and cache state, keeping the page module thin is especially important.

10.5 `pages/static_data.py`

Registers the fixture-inspection page and builds its static layout. It has no financial snapshot dependency.

10.6 `pages/not_found_404.py`

Provides the fallback page. Navigation targets respect the configured request prefix so a deployment under JupyterHub or another reverse proxy does not send users to the server root.

10.7 Improvement

Keep page modules at this size. If a page needs configuration, add a typed page service to the factory rather than importing concrete global objects.

Chapter 11

Browser assets

Dash automatically serves files in `assets/`. These files are active code, not decorative afterthoughts: they implement spreadsheet gestures, progress recovery, theme synchronization, and the delegated action path used by the largest tables.

11.1 `assets/s01_style.css`: the visual system

The stylesheet is organized into numbered conceptual sections. Its first section defines design tokens such as canvas, surface, text, borders, selection, positive/negative semantics, radii, and font stacks. Dark mode changes neutral surfaces and text but deliberately keeps semantic table colors recognizable.

Subsequent sections style:

- Base elements, focus rings, hidden and screen-reader-only utilities.
- Sticky application header and navigation.
- Refresh controls and operating-date cards.
- Backend progress hero, stages, and cube loader.
- Shared page shells and sections.
- Filter panels and dropdowns.
- Risk, Quick Search, Top Book, and alternate tables.
- Spreadsheet selection and column-resize affordances.
- P&L aggregate tables, editors, history, send, and validation sections.
- Stock hierarchy and source rows.
- Static Data tables.
- Empty/error states.
- Dark-theme overrides.
- Responsive layouts and print/narrow-screen behavior.
- `prefers-reduced-motion` behavior.

11.1.1 Good design details

- Focus-visible styles are explicit.
- Buttons communicate disabled state.
- Numeric cells use tabular numerals.
- Negative values have a stable semantic color.
- Selection has a border as well as a background.
- `[hidden]` is authoritative.
- Controls wrap at narrow widths instead of forcing a fixed desktop canvas.
- Motion is suppressible by user preference.

11.1.2 Limitations

At roughly 76 KB, selector ownership is hard to audit. Component-specific styles and global utilities are mixed in one cascade, and several component IDs appear directly in selectors.

11.1.3 Improvement

Split by cascade layers while retaining one delivered bundle:

```
@layer reset, tokens, layout, components, pages, utilities, overrides;
```

Build the final asset from smaller source files and run a visual-regression suite at representative widths and both themes. Prefer classes over IDs except where an element is truly unique.

11.2 `assets/s02_app.js`: delegated interaction and refresh lifecycle

This IIFE is the browser controller. It intentionally uses ordinary DOM APIs and `window.dash_clientside.set_props` so high-volume interactions do not require a React component or one Dash callback per element.

11.2.1 Theme state

The active theme is loaded from `localStorage` when possible, otherwise from the system color-scheme preference. `applyTheme` updates the root data attribute, button label/ARIA state, and optionally persists the choice.

Plotly figures need more than CSS. A scheduled theme synchronizer calls `Plotly.relayout` for mounted graphs, retries briefly when Dash has inserted a graph node before Plotly has initialized it, and updates axes, legend, and backgrounds without rebuilding data.

11.2.2 Cube animation

The loader uses quaternion multiplication and axis-angle rotations to roll a six-faced cube around a hexagonal path. Each visible root has state for elapsed time, size, orientation, shadow, and paint timestamps. A shared animation frame updates visible loaders at a bounded frame rate.

The controller:

- Pauses disconnected or hidden roots.
- Resets elapsed state when a loader reappears.
- Applies a static cube for reduced-motion users.
- Recomputes size on window resize.
- Stops timers and observers on a non-persisted page hide.

The mathematics is more sophisticated than a CSS spinner, but it is isolated from financial behavior. A simpler CSS animation would be easier to maintain; the present code is justified only if the branded rolling motion is a deliberate product requirement.

11.2.3 Spreadsheet-style selection

Risk and other semantic tables support:

- Ctrl/Cmd or Shift selection.
- Dragging a rectangular range.
- Selecting a visible metric column from its header.
- Count, sum, average, minimum, and maximum summary.
- TSV copy preserving visible row/column geometry.
- Escape to clear.
- Numeric normalization for copy.
- Exclusion of hidden descendants.
- Keyboard selection with Enter/Space plus a modifier.

Selections are a `Set` of live DOM cells rather than server state. That is the right performance choice because highlighting and copying do not change financial data.

11.2.4 Column resizing

Resize handles are attached to table headers after render. Mouse drag or keyboard arrows apply explicit width to all cells in a column; double-click clears the override. Scoped MutationObservers watch stable Risk grid roots instead of the whole document, reducing observer churn when a large hierarchy mounts.

11.2.5 Quick Search hierarchy

Quick Search disclosure can be toggled entirely in the browser by changing row data attributes and `hidden`. Collapsing clears selected hidden cells so copy state never includes invisible descendants.

11.2.6 Delegated Risk action envelopes

For row, metric-header, and metric-cell actions, `publishRiskAction`:

1. Identifies the action kind and destination Store.
2. Reads the nearest `data-risk-view-token`.
3. Reads the stable row key, metric, measure, and source.
4. Increments a browser sequence.
5. Maintains optimistic pending open-row state for rapid repeated clicks.
6. Calls `dash_clientside.set_props(storeId, {data: action})`.

Python validates the token against the rendered view before reducing state. Top Book has separate action Stores where its semantics differ.

This design minimizes callback count and network payload, but the action schema is an implicit cross-language API. Define and test a versioned JSON schema for it. Add an `action_version` field so Python can reject an incompatible older asset after a deployment.

11.2.7 Refresh lifecycle controller

The browser progress system combines three signals:

- Dash's `running` class on the refresh status node.
- JSON progress from `/progressz`.
- The committed revision signal exposed in the rendered shell.

It posts `/startz` during cold startup, polls progress once per second, applies timeouts with `AbortController`, uses exponential retry delays, and records the server boot ID, refresh attempt ID, startup attempt ID, baseline revision, and attempt start time.

These identifiers solve several races:

- A poll may initially return progress from an earlier attempt.
- A fast Dash callback can begin and finish between one-second polls.
- Another browser may own the writer.
- The worker can be replaced during startup.
- Revision 1 can commit while the browser still displays the cold shell.
- A transport failure must not be presented as a confirmed financial failure.

The controller never invents success from elapsed time. It finishes normally when Dash's running state closes or when a matching backend attempt becomes non-running. During cold-start handoff it may perform at most one session-guarded reload for a specific boot/revision. Repeated reload loops are prevented through `sessionStorage`.

When a new committed revision is observed and a financial page can consume it, the controller uses `set_props` to update `data-revision-store`. This decouples slow revision-driven rendering from the refresh transaction: the backend can publish, the progress hero can complete, and page callbacks can then render the new immutable revision.

11.2.8 Keyboard shortcuts

Shift+F8 clicks Refresh Risk and Shift+F9 clicks Refresh P&L when enabled. Readonly radio controls receive manual keyboard behavior because their visual implementation still needs native-like navigation.

11.2.9 Mutation and lifecycle cleanup

A general UI observer detects theme buttons, cube loaders, progress nodes, and new Plotly graphs. Dedicated Risk grid observers add resize affordances. Timers, animation frames, observers, and intervals are disconnected on page hide; BFCache restoration reconnects relevant hooks.

11.2.10 Limitations and improvements

- At roughly 95 KB, one IIFE contains several independent controllers. Split it into ES modules bundled into one asset: theme, animation, selection, risk actions, and refresh progress.
- Add browser unit tests with a DOM implementation and end-to-end tests for attempt/revision races.
- Replace stringly typed `dataset` contracts with generated constants/schema.
- Use one logging helper with severity and sampled diagnostics.
- Review the 45-second bootstrap reload threshold against actual platform startup behavior; keep the one-reload guard.

11.3 `assets/s03_select.js`: P&L DataTable range gesture

Dash DataTable already understands click and Shift-click selection. This script adds drag selection without reimplementing DataTable internals.

On primary-button down it remembers the start cell and table. Once movement exceeds six pixels it marks a drag. On release it dispatches one synthetic native click at the start cell, followed by a Shift-click at the end cell. DataTable therefore computes the rectangle using its own selection model.

The script suppresses the trusted click that follows the drag, clears selection in other editor tables, prevents native dragstart, and cleans up on copy, Escape, blur, or invalid targets.

This is an elegant adapter around an existing component contract. Its main risk is dependence on Dash DataTable's internal `.dash-cell` and `.cell--selected` classes. Pinning Dash reduces but does not remove that risk. An end-to-end compatibility test should run before every dependency upgrade.

Chapter 12

Operational tools, scheduled job, generated documentation, and data

12.1 `tools/__init__.py`

The package initializer is empty. Tooling runs only through an explicit module or script invocation.

12.2 `tools/s01_fixtures.py`: deterministic synthetic source generation

This script is the canonical producer and checker for the visible fake connector data. It can be executed from any working directory because it resolves the project root from its own path and adds that root to `sys.path` before importing the product registry.

12.2.1 Fixture catalogue

`SOURCE_FIXTURES` enumerates every canonical Source Type with three illustrative Underlyings, connector-owned Groups, and a market-kind hint. The expected Source Types are cross-checked against `PRODUCT_SPECS`; fixture generation therefore fails if the product catalogue changes without a matching fixture definition.

12.2.2 Axes

`FULL_AXIS_VALUES` gives each product's complete market-owned tenor structure. `_risk_axis_values` deliberately removes the last market tenor so tests prove that:

- MarketBook retains market-only points.
- Risk/P&L uses only keys carried by positions.
- Quick Market and Quick Risk are not accidentally the same dataset.

Two-axis products form a Cartesian surface from declared axes. Nothing assumes a fixed M-by-N size.

12.2.3 Deterministic values

Stable integers are derived from SHA-256 of identity parts. Risk, `dRisk`, Open, Current, signs, and small movements are consequently reproducible across machines without using process-randomized Python hashes.

Every business-facing fake identity contains `FAKE_REPLACE_ME`. This is an excellent deployment safeguard: an unfinished integration is visible in the UI, archive, and any accidental output.

12.2.4 Governed examples

The fixtures include:

- Mapped and unmapped Portfolios.
- Age-zero and deliberate Age-one readiness.
- All Credit measure pairs.
- Market-only tenors.
- Completed historical market dates.
- Reported-underlying mappings.
- Thresholds and Concerto identities.

12.2.5 Generate versus check

Normal mode writes canonical CSV bytes. `--check` regenerates in memory and compares schema/content with checked-in files. CI should always run check mode; otherwise hand-edited fixtures can diverge from the generator.

12.2.6 Improvement

Emit a machine-readable manifest containing each generated path, schema version, row count, and digest. Static Data, the report inventory, and CI can consume the same manifest rather than maintaining parallel lists.

12.3 `tools/s02_manual.py`: README diagrams and printable manual

This script creates the four PNG diagrams under `docs/` with Pillow and renders the README into a ReportLab PDF. It includes font fallback, text wrapping, drawing helpers, a small Markdown renderer, code blocks, tables, page breaks, and deterministic output paths.

The useful architectural idea is that diagrams are code, not screenshots. Text, boxes, and arrows can be reproduced after a change.

The limitation is maintaining a private Markdown renderer. Pandoc already handles more Markdown constructs and accessibility metadata. The script should either focus only on diagram generation and invoke Pandoc for the document, or use the same report-generation pipeline as this audit.

12.4 `tools/s03_archive_official_risk.py`: scheduler entry point

This module makes the archive workflow callable from Jupyter Scheduler, a shell, or a test.

- `resolve_archive_root` reads `PL_HISTORICAL_PATH`, resolves relative paths against the repository rather than the scheduler's working directory, and returns an absolute path.
- `resolve_colossus_loader` requires a `module:function` reference and verifies the resolved attribute is callable.
- `_default_manager_factory` imports the production manager lazily and disables illustrative stage delays.
- `run_scheduled_archive` creates the manager and calls `archive_from_manager` with refresh enabled.
- `run_from_env` is the notebook-friendly zero-argument API.
- `main` prints a compact status line.

The dynamic import is appropriate at a composition boundary, but configuration errors occur at runtime. A deployment preflight command should resolve the loader, validate write permissions, and perform a connector dry run before a job is scheduled.

12.5 `jobs/archive_official_risk.ipynb`

The notebook contains:

1. A Markdown explanation of the idempotent official archive job.

2. One parameter cell for an optional project root.
3. A code cell that finds the project tree, adds it to `sys.path`, imports `run_from_env`, runs the job, and prints the result.

The notebook intentionally delegates all business behavior to a tested Python module. That is better than embedding an untestable copy of archive logic in cells.

The search for the project root is convenient, but production scheduling should set `RISK_CUBE_PROJECT_ROOT` explicitly and pin the environment. Ambiguous ancestor discovery can otherwise select the wrong checkout.

Chapter 13

Checked-in data contracts

The root `data/` CSVs are executable examples. They are not a substitute for production sources.

13.1 `data/s01_readiness.csv`

Exact columns: `Risk Type`, `Risk Greek`, `Age`.

Each supplied row governs the suggested Risk Date for one known product pair. Missing known pairs default to Age 0 and are marked as defaulted by the pipeline. Unknown pairs, booleans, negative ages, non-integers, and duplicate pairs fail validation.

13.2 `data/s02_checker.csv`

Exact columns: `Risk Type`, `Risk Greek`, `MMMFile`, `Product`.

This is the partial RiskChecker inventory. `MMMFile` must use the `.mmm` extension and Product must be one of the governed portfolio Product values. Absence from this file is meaningful when RiskChecker is enabled.

13.3 `data/s03_risk.csv`

The fixture Risk source includes Source Type, exact market identity, canonical tenor columns, Portfolio, connector-owned Group, Risk, `dRisk`, and optional Credit measure pairs. Empty tenor columns remain present so a single fixture file can be partitioned into product-specific exact schemas.

The active feed partitions by Source Type and projects only the columns required by the chosen adapter. The pipeline validates every resulting product frame again.

13.4 `data/s04_open.csv`

Contains the complete opening MarketBook grain:

```
Source Type + Underlying + declared tenors
+ declared tenor ranks + Open
```

Ranks are market authority. They are not recomputed from labels.

13.5 data/s05_current.csv

Matches the opening identity/rank contract and supplies `Current`. The manager passes the selected `Live` or `OFFICIAL` status to the adapter; the fixture feed attaches that status so the pipeline can prove routing consistency.

13.6 data/s06_portfolios.csv

Contains required Portfolio governance:

```
Portfolio, Product, Activity, SignoffGroup, Category
```

`Sub Category` is optional in the canonical schema and omitted by this fixture. Only uniquely mapped Portfolios contribute to mapped dashboard and send totals. Unmapped rows remain available for remediation.

13.7 data/s07_thresholds.csv

Provides PL, Risk, and `dRisk` thresholds by Risk Type/Greek, including Cash Flow/New and Cross Gamma classifications. Thresholds control presentation promotion only; they do not alter financial values.

13.8 data/s08_concerto.csv

Maps every sendable Risk Type/Greek pair to one exact Concerto field. Cash Flow/New maps to `cashfloweffect`. The P&L send builder requires one unique mapping and does not invent an identifier from labels.

13.9 data/s09_reported.csv

Maps one raw Risk Type/Greek/Underlying identity to a reported Underlying. Mappings are applied after calculation, can expand one raw row to several reported labels, and are not recursive.

Chapter 14

Historical data leaves

14.1 `data/histo/2026-08-10/`

This is an explicitly marked synthetic, completed archive-protocol example.

- `_SUCCESS` is the publication manifest. It records schema version, Market Date/status, exact filenames, row counts, and SHA-256 digests.
- `risk.csv` is the canonical archived financial Risk/P&L frame.
- `market.csv` is the deduplicated official MarketBook with ranks and quotes.
- `colossus.csv` is the exact governed comparison input.

The loader accepts the leaf only when every manifest statement and financial invariant verifies.

14.2 `data/histo/2026-08-11/` through `2026-08-14/`

These are legacy P&L-history demonstration leaves.

Each date contains:

- `histo.csv`: historical/actual rows.
- `predicted.csv`: Predict rows.
- `market.csv`: a checked-in historical MarketBook.

The history loader treats the two P&L files as legacy Colossus/Predict inputs, while the market-history path supplies Quick Market time series. These leaves do not become v2 official archives merely because a `market.csv` exists; completion is determined by the archive protocol expected by the relevant reader.

Chapter 15

Generated documentation images

15.1 `docs/s01_flow.png`

Architecture overview: Browser -> Dash UI -> Refresh Manager -> personal connectors and committed readers.

15.2 `docs/s02_dates.png`

Authoritative date chain: Market Date, checker date, suggested Risk Dates, and forced overrides.

15.3 `docs/s03_market.png`

Per-Underlying Open/Current loading, full MarketBook construction, Quick Market, and the Risk-left join.

15.4 `docs/s04_startup.png`

Nonblocking import/first paint, startup coordinator, revision 1, and recovery.

These PNGs are generated by `tools/s02_manual.py`. A generated-binary check in CI should fail when their source logic changes without regeneration.

Chapter 16

Test suite: the executable specification

`pytest.ini` discovers `tests/s*.py`, and the numbered modules follow the same reading order as the implementation. The suite is unusually valuable because it tests not only happy-path arithmetic but also negative contracts: malformed schemas, duplicate identities, stale revisions, concurrent writers, partial archives, browser progress wording, and import-time I/O.

The explanations below describe what each file proves and why that proof matters.

16.1 `tests/s01_schema.py`

This module establishes uniqueness and catalogue invariants:

- Portfolio external names and UI keys are one-to-one.
- Required/optional configuration columns are exact.
- Product values are governed.
- Source Types and Risk Type/Greek pairs are unique.
- One-axis and two-axis products declare the correct canonical tenors.
- IR Gamma and Credit Vega are not misclassified as surfaces.
- Credit measures contain complete Risk/`dRisk` pairs in settled order.

These tests catch configuration drift at the smallest and cheapest layer.

16.2 `tests/s02_checker.py`

This file tests business dates, readiness, checker inventory, and stage-delay validation.

It explicitly covers Friday, Saturday, Sunday, and Monday; verifies that the checker date is the prior business day; then applies Age **after** that step. Ambiguous Age values such as booleans, negatives, fractions, and non-finite numbers fail.

Readiness can be partial: missing known products default to Age 0 and carry a defaulted flag. The checker inventory can also be partial but must use exact columns, approved Product values, and `.mmm` files. A combined loader receives the single authoritative checker date. Stage-delay test values must be real, finite, nonnegative numbers.

16.3 `tests/s03_adapters.py`

These are executable examples of every active adapter family.

They prove that:

- Adapter module numbering stays sequential.
- Dates, Underlying, and dynamic market status reach the personal source unchanged.

- Arbitrary two-dimensional IR DeltaVega surfaces are accepted.
- Commodity Delta is a one-axis curve and sources are called one Underlying at a time.
- Credit preserves every supplied measure and optional Region.
- Partial Credit measure pairs fail.
- Schema drift and guessed market-status spelling fail closed.

This is the right place for production connector teams to add contract tests before changing a source.

16.4 tests/s04_market.py

This module proves MarketBook semantics:

- Group is required but connector-owned.
- Open and Current outer-join, retaining market-only tenors.
- The caller chooses `Live` versus `OFFICIAL`.
- A status mismatch fails.
- Open and Current must agree on tenor ranks.
- Risk left-joins to market and therefore excludes market-only points from Risk/P&L.
- Quick Risk and Quick Market read different grains.
- A 30-by-30 surface retains all 900 ranked quotes.
- Dropdown search is tokenized/case-insensitive, while pivot selection is exact.
- The search catalogue stores compact indexes rather than row-level postings.
- Parent quote averages use only complete Open/Current pairs.

These tests defend several financial subtleties that a generic join or pivot could easily violate.

16.5 tests/s05_pl.py

This file covers P&L governance, history, overlays, and local adjustment storage.

It tests exact historical schemas and duplicate rejection; canonical Colossus/Predict labels; path-level series aggregation; Daily/WTD/MTD/YTD calendar bounds; global latest-date behavior; missing-date/type semantics; directory-signature caching; P&L base construction; Signoff authority; Concerto mapping; adjustment replacement; stale revisions; and storage recovery.

The recurring principle is “missing is not zero” and “an adjustment replaces the governed key.” Those are appropriate financial controls.

16.6 tests/s06_ui.py

This module tests pure UI preparation and component contracts. It walks rendered component trees, verifies IDs and semantic structures, checks aggregate totals, sorting, row keys, disclosure behavior, accessible labels, and table variants. It provides a fast alternative to browser tests for most markup invariants.

16.7 tests/s07_integration.py

These are end-to-end manager tests with controlled sources. They exercise the complete transaction from checker/date selection through product Risk, market, P&L, governance enrichment, search publication, and revision commit.

They also verify that a failed attempt retains the last good snapshot, that selective refreshes reuse eligible immutable data, and that concurrent/stale requests cannot publish over a newer revision.

16.8 tests/s08_feeds.py

This file verifies the checked-in fixture feed boundary:

- The fixture files are clearly fake.
- Each source partitions into the exact adapter schema.
- File-revision caches invalidate when content changes.
- Underlying selection is exact.
- Market status is passed rather than guessed.
- Production manager composition includes every product and supplemental loader.
- External sender placeholders refuse delivery.

16.9 tests/s09_plui.py

This is the largest P&L UI contract suite. It covers filter controls, aggregate selection, editor materialization, editable columns, stable row IDs, late-edit recovery, `Patch` behavior, selection summaries, save validation, and all three send scopes.

Tests call unwrapped callbacks where useful, supplying controlled context and services. They verify that stale revision/date/filter combinations are rejected and that a sender receives only the validated governed effective frame.

16.10 tests/s10_reads.py

This module focuses on read-path efficiency and immutability. It proves that normal page reads do not call connectors, that snapshots return safe copies or immutable structures as designed, and that repeated reads share prepared revision state without permitting a caller to mutate the manager's authority.

16.11 tests/s11_fixtures.py

These tests execute fixture generation in check mode, validate exact schemas, confirm deterministic output, require the fake marker, ensure ProductSpec axes match fixture axes, and verify the historical MarketBook examples.

The suite makes generated CSVs reviewable artifacts rather than hand-maintained test data.

16.12 tests/s12_startup.py

This module deeply tests cold startup and process ownership.

It proves that:

- The base Dash layout and Risk cold shell mount before any source I/O.
- Component IDs are not duplicated between shared and page-local shells.
- Static Data does not trigger financial startup.
- Only one visitor starts the writer.
- Only one delayed start can be pending.
- A watchdog reports the active function but never starts a second writer.
- Failure is visible and retryable.
- `/startz` is idempotent.
- `/progressz` carries attempt and boot identity.
- Public endpoint URLs correctly distinguish request and internal route prefixes.
- A warm manager retains recovery callbacks.
- Browser progress copy never claims an unconfirmed result.
- Session reload guards and revision handoff logic remain present.

Some tests inspect the JavaScript source text. Such tests are brittle, but here they act as high-signal guards for race-handling statements that are difficult to cover in a Python-only runner. They should eventually be complemented by browser integration tests.

16.13 `tests/s13_publish.py`

This file tests publishing staging. It verifies the allowlist, renamed entry point, inclusion of required assets/packages/data, exclusion of runtime adjustments and private material, archive filtering, and cleanup of the temporary directory. It also checks command construction and failure/timeout reporting.

16.14 `tests/s14_reporting.py`

These tests cover reported-underlying mapping. They assert exact input schema, nonrecursive mapping, one-to-many expansion, preservation of unmapped rows, stable totals, and deterministic order. Mapping is proven to occur after calculation rather than changing raw market identity.

16.15 `tests/s15_overlays.py`

This module tests the adjustment overlay and related supplemental behavior against the canonical dashboard. It verifies that overlays replace matching base values, preserve unmatched base rows, reject duplicate/ambiguous keys, and do not manufacture missing financial fields.

16.16 `tests/s16_refresh_shell.py`

The refresh-shell suite inspects loading, success, failure, and retained-snapshot component states. It checks disabled controls, live regions, stage labels, operating dates, retry behavior, progress endpoint payloads, callback `running` outputs, and the distinction between a cold-start failure and a later failed refresh that still has usable data.

16.17 `tests/s17_stock.py`

This file covers the Stock contract end to end:

- Exact source schema and numeric validation.
- Current/prior outer comparison.
- Portfolio governance mapping.
- Include/exclude filters.
- Mapped/unmapped counts.
- Provisional hierarchy construction.
- Open-path normalization and pruning.
- Promotion threshold.
- Date defaults.
- Page cache and stale intent sequencing.
- Explicit lazy loading of source rows.

16.18 `tests/s18_new_positions_adapter.py`

These tests target the raw New Trades adapter. They cover exact mixed-blotter columns, MARKET and CASHFLOW discriminator values, duplicate trade/position identity, required and forbidden text/numeric fields, boolean traded-level flags, timezone normalization, known/unknown traded levels, reserved Cash Flow classification, and the rule that only CASHFLOW receives P&L at the adapter stage.

16.19 tests/s19_risk_filters.py

This module exercises Risk filter semantics and reducer state. It verifies OR within a selector, include-AND/exclude-OR across selectors, option pruning, promotion recalculation, open-row pruning, stable view tokens, and rejection of stale delegated actions. It also checks that filter changes do not mutate the committed snapshot.

16.20 tests/s20_connector_provenance.py

These tests ensure source provenance stays visible through the adapter and pipeline boundaries. Fixture and placeholder sources are labeled as fake, while active adapters retain Source Type and connector-owned Group/Region. The tests guard against an integration silently passing fixture data as production data.

16.21 tests/s21_pipeline_provenance.py

This file follows provenance through normalization, joins, supplemental overlays, and dashboard publication. It verifies source classification, Market/Risk dates, status, mapping flags, split/origin metadata, and preservation of missingness. It is a defense against transformations that produce correct numbers but lose the evidence needed to audit them.

16.22 tests/s22_refresh_dates.py

These tests focus on the selective refresh planner and date overrides. They cover suggested versus forced dates, drafts versus committed settings, revision-rebase behavior, which products must reload after a Risk Date change, which quote legs can be reused, metadata-only and P&L-only plans, and the final compare-and-swap check.

16.23 tests/s23_saved_views.py

This module treats saved views as both storage and UI state.

It proves that Risk, Stock, and P&L share the same five explicit filter keys and the same stored named views; names and values normalize deterministically; documents live in one shared scope; writes are atomic; concurrent thread writers serialize; path traversal, duplicate names, invalid IDs, and unknown filter keys fail; update preserves identity; and no temporary files remain.

UI tests verify the always-present Base view, collapsed disclosure, current label, action states, application request validation, manual-supersede detection, and the important ownership rule that generic saved-view callbacks never write individual filter dropdown values directly.

16.24 tests/s24_plhistory.py

This file tests the expandable history component and callbacks: path-token validation, descendant pruning, global latest date, period headers, comparison toggle, missing observations, cell selection, chart series, filter interaction, and cache behavior. It verifies that a stale leaf is not promoted to the latest Daily result.

16.25 tests/s25_cross_gamma.py

These tests cover both the raw sensitivity contract and pipeline contribution:

- Input/output product identities must be known and supported.
- Input and output market keys are exact.
- Duplicate matrix cells fail.
- Missing input quote prevents contribution.

- Sensitivity times input move produces the output exposure.
- Contributions aggregate to output identity.
- Both legs expand quote scope.
- Source rows and aggregated output rows keep distinct origin/split metadata.
- Optional output quotes never change the contribution calculation.

16.26 tests/s26_new_trades.py

This module tests the pipeline stage after the raw blotter adapter. It verifies MarketBook joins, known and Open-fallback traded levels, absolute/percentage/ Taylor formulas, Cash Flow bypass, source metadata, missing quote behavior, MarketBook scope expansion, supplemental Credit SP01, and publication into dashboard/P&L/search outputs.

16.27 tests/s27_expandable_visuals.py

These tests cover reusable open-path and selection rules across Risk, Stock, P&L history, and validation trees. They ensure parent closure removes descendants, hidden rows are not copied or aggregated, pattern-matching toggle IDs encode stable paths, and accessible expanded state matches visual state.

16.28 tests/s28_validate_pl.py

This module tests official archive comparison. It constructs Predict and Colossus examples, derives unique Portfolio authority from archived Risk, labels matched/Predict-only/Colossus-only rows, retains ambiguous/unmapped Colossus exactly once, applies page filters, builds the six-level hierarchy, and exercises lazy date discovery and row toggles.

16.29 tests/s29_risk_archive.py

The final test module is a full archive-protocol specification.

It verifies:

- Atomic complete publication and idempotent rerun.
- Weekend eligibility against the resolved natural Market Date.
- v2 manifest coverage of `market.csv`.
- Read compatibility for a valid v1 leaf without market data.
- Rejection of v1 mislabeled with v2 market metadata.
- Source Type/Risk Type/Greek consistency.
- Exact Market Date and OFFICIAL status.
- Unique quote identity and nonnegative integer ranks.
- Preservation of unavailable quotes as missing.
- Legacy optional MarketBook history and daily rank order.
- Identity-specific market reads that avoid loading Risk/Colossus and cache the selected leaf.
- Skipping Live, stale-date, or error-bearing snapshots before Colossus I/O.
- Cleanup after loader or validation failure.
- Digest, row-count, filename, and arithmetic tamper detection.
- Scheduler path resolution and manager composition.

The archive tests are especially strong because they attack partial and corrupt states rather than merely confirming that files can be written.

Chapter 17

Complete callback and browser-event atlas

The application has three kinds of callback:

1. **Dash server callbacks**: Python functions registered with `@app.callback` or a callback factory.
2. **Dash client-side callbacks**: JavaScript functions registered through Dash for pure presentation transitions.
3. **Delegated browser event handlers**: capture/bubble listeners, timers, observers, and animation frames in `assets/`.

The atlas below accounts for the authored semantic callback families. Dash also creates its own internal page-router callback and renderer plumbing; those are framework-generated and are not repository business logic.

17.1 How the callback graph stays coherent

Several Stores have distinct ownership:

- `data-revision-store` identifies the committed financial snapshot the browser should render.
- Filter Stores normalize authoritative dropdown values.
- Open-path Stores contain only presentation hierarchy paths.
- Action Stores are write-once envelopes from delegated browser events.
- View tokens identify the exact render that produced an action.
- P&L effective Stores carry a calculated base, persisted adjustments, unsaved edits, loaded revision, Market Date, and adjustment revision.
- Saved-view request Stores coordinate a target across several dropdown-owning callbacks without letting the generic controller own those dropdowns.
- Startup/refresh attempt IDs live in progress responses and browser state, not in financial rows.

The central rule is that a callback may optimistically update presentation, but only `RiskRefreshManager` can publish financial revision `N+1`.

17.2 Atlas entries

17.2.1 ui/s07_events.py - Dash server

17.2.1.1 R01 - Cold Risk page bootstrap request

Trigger. The page-local initial-load trigger appears only on a cold Risk page.

State read. Startup coordinator status and current manager revision.

Outputs/effect. Requests `/startz`/startup participation and keeps the loading shell visible until a committed revision exists.

Race and validation guard. The process coordinator deduplicates callers; a mounted Static Data/P&L/Stock page does not start financial I/O.

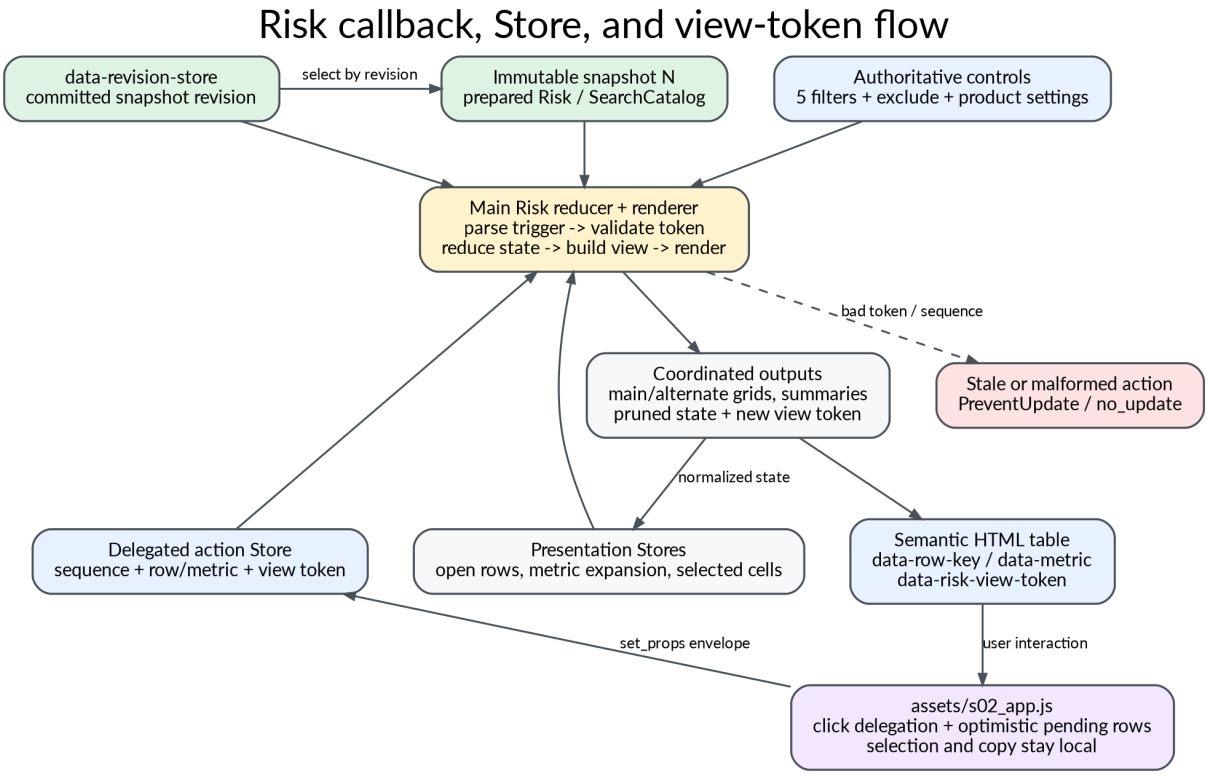

Figure 17.1: Callback and Store flow

Interaction with the rest of the app. Browser progress polling follows the same startup attempt and later writes `data-revision-store`.

17.2.1.2 R02 - Shared refresh-shell hydration

Trigger. Committed data revision changes or a warm page mounts.

State read. Manager health, committed snapshot settings, dates, errors, and stage delays.

Outputs/effect. Refresh controls, operating-date cards, status class/text, hidden commit-revision signal, and shell visibility.

Race and validation guard. Uses only a committed revision; cold state remains a separate component tree.

Interaction with the rest of the app. The hidden revision signal is consumed by browser refresh recovery and page-local rendering.

17.2.1.3 R03 - Risk dimension-filter state

Trigger. Activity, Signoff Group, Portfolio, Category, Sub Category, or exclude control changes.

State read. Current five dropdown values and exclude mode.

Outputs/effect. One normalized filter Store used by aggregate, Top Book, Risk hierarchy, and saved-view matching.

Race and validation guard. Normalizes missing/duplicate text; no financial snapshot mutation.

Interaction with the rest of the app. A filter fingerprint contributes to the Risk view-generation token.

17.2.1.4 R04 - Aggregate P&L panel

Trigger. Revision, filters, aggregate hierarchy actions, or dimension choice changes.

State read. Committed dashboard frame, normalized filters, open paths, selected aggregate scope.

Outputs/effect. Aggregate P&L table, open-path Store, selected-scope Store, summary/status.

Race and validation guard. Prunes paths and selection against the filtered revision; totals use missing-aware aggregation.

Interaction with the rest of the app. Selection can seed related P&L or detail views without changing the underlying snapshot.

17.2.1.5 R05 - Top Book reducer and renderer

Trigger. Revision, filters, Top Book row action Store, Top Book cell action Store, metric/measure settings.

State read. Prepared Risk data and current Top Book open/selection state.

Outputs/effect. Top Book grid and its normalized open-row/cell state.

Race and validation guard. Delegated actions carry stable row keys; invalid keys are ignored.

Interaction with the rest of the app. Browser JavaScript publishes Top Book actions to dedicated Stores.

17.2.1.6 R06 - Risk Type tab selection

Trigger. Risk Type tab button clicks or a new revision.

State read. Available Risk Types and previous active tab.

Outputs/effect. Active Risk Type Store and tab classes/ARIA state.

Race and validation guard. Falls back to an available canonical type when prior choice disappears.

Interaction with the rest of the app. Feeds family controls, hierarchy preparation, and view token construction.

17.2.1.7 R07 - Risk filter option and saved-view application

Trigger. Revision, another filter, exclude mode, or a pending Risk saved-view request.

State read. Committed mapped dimensions, current controls, request target/base values.

Outputs/effect. Dropdown options and values, exclude control, request-resolution metadata.

Race and validation guard. A pending request applies only while controls still match its recorded base; manual supersession wins.

Interaction with the rest of the app. The generic saved-view acknowledgement callback later verifies the target.

17.2.2 ui/s07_events.py - Dash clientside

17.2.2.1 R08 - IR-family control visibility

Trigger. Active Risk Type/Greek/family state.

State read. Client-side control values only.

Outputs/effect. Styles/hidden state for family-specific controls.

Race and validation guard. No server data and no financial state.

Interaction with the rest of the app. Reduces a server round trip for a pure presentation transition.

17.2.3 ui/s07_events.py - Dash server

17.2.3.1 R09 - Main Risk state reducer and renderer

Trigger. Revision; filters; Risk Type/family/dimension; row, cell, or metric action Stores; measure/region/reported/promotion controls; saved-view effects.

State read. Current open rows, expanded metrics, selected cells, previous view token, committed prepared snapshot.

Outputs/effect. Approximately fifteen coordinated outputs: main/alternate grids, normalized open/metric/cell state, summaries, counts, view token, visible-state metadata, and related controls.

Race and validation guard. Rejects stale generation tokens, malformed action envelopes, duplicate/old action sequences, and state absent from the new view.

Interaction with the rest of the app. Combines reduction and render in one request to avoid the former Store-then-render second round trip.

17.2.3.2 R10 - Lazy unmapped-book audit

Trigger. Unmapped disclosure opens or revision/filters change while open.

State read. Unmapped rows from the committed prepared view.

Outputs/effect. Audit note and paged/filterable DataTable children.

Race and validation guard. Does nothing while closed and caps the initial rendered rows.

Interaction with the rest of the app. Makes excluded data inspectable without burdening ordinary page load.

17.2.3.3 R11 - Quick Risk option search

Trigger. Dropdown `search_value` or revision.

State read. SearchCatalog's pre-normalized exact Risk labels.

Outputs/effect. Bounded dropdown options.

Race and validation guard. Typing only searches labels; it never becomes a data query.

Interaction with the rest of the app. The selected exact label triggers R12.

17.2.3.4 R12 - Quick Risk exact pivot

Trigger. Exact identity selection, index dimensions, hierarchy actions, or revision.

State read. Risk row-position index and additive metrics from SearchCatalog.

Outputs/effect. Quick Risk pivot table, total/count text, open hierarchy state.

Race and validation guard. Unknown/free-text identity returns empty; hidden descendants are pruned.

Interaction with the rest of the app. Browser hierarchy expansion may be instantaneous for already-rendered rows.

17.2.3.5 R13 - Quick Market option search

Trigger. Market dropdown `search_value` or revision.

State read. SearchCatalog's exact MarketBook identities.

Outputs/effect. Bounded market identity options.

Race and validation guard. Case-insensitive token search, exact subsequent selection.

Interaction with the rest of the app. Selected identity drives axis, current, and history callbacks.

17.2.3.6 R14 - Quick Market axis controls

Trigger. Selected exact market identity or revision.

State read. ProductSpec axes and ranked MarketBook values.

Outputs/effect. Swap/option selectors, defaults, enabled/hidden state.

Race and validation guard. Does not assume a fixed surface size; one-axis products hide the second control.

Interaction with the rest of the app. Axis choices scope R15 and R16.

17.2.3.7 R15 - Quick Market current curve/surface

Trigger. Exact identity, axis choices, index layout, hierarchy state, or revision.

State read. Full MarketBook, not portfolio Risk.

Outputs/effect. Exact current/open/move table or surface plus status/count.

Race and validation guard. Averages only complete quote pairs at rollups; preserves missing quotes.

Interaction with the rest of the app. Shares identity and axis state with historical market rendering.

17.2.3.8 R16 - Quick Market history

Trigger. Exact identity, axis choices, history disclosure/settings.

State read. Archive market-history reader for the selected identity.

Outputs/effect. Historical table/chart and status.

Race and validation guard. Reads only market files; missing dates/quotes are not filled with zero.

Interaction with the rest of the app. Uses archive ranks to preserve daily tenor order.

17.2.3.9 R17 - AutoPL toggle

Trigger. AutoPL button click or committed settings hydration.

State read. Current on/off state.

Outputs/effect. Toggle Store, label, class, ARIA pressed state.

Race and validation guard. A presentation setting only; does not directly refresh.

Interaction with the rest of the app. R18 converts enabled state into interval behavior.

17.2.3.10 R18 - Automatic P&L interval control

Trigger. AutoPL state.

State read. Configured 15-minute interval and current disabled state.

Outputs/effect. Interval disabled/enabled property.

Race and validation guard. Manager planner and writer lock still govern every tick.

Interaction with the rest of the app. Enabled interval ticks enter R23 as P&L-only refresh requests.

17.2.3.11 R19 - Commodity market-data toggle

Trigger. Commodity toggle click.

State read. Committed commodity setting and browser revision.

Outputs/effect. Requested setting through the refresh transaction; label/class update after commit.

Race and validation guard. A stale browser cannot overwrite a newer setting; controls lock while refreshing.

Interaction with the rest of the app. Changing this setting can force selective Commodity quote reload.

17.2.3.12 R20 - Reported/promoted Risk display settings

Trigger. Reported Underlying and promotion controls.

State read. Current display settings and filtered data.

Outputs/effect. Settings Stores and affected rendered view.

Race and validation guard. Reported mapping is display aggregation after financial calculation.

Interaction with the rest of the app. Contributes to R09's view token and hierarchy.

17.2.3.13 R21 - Region display toggle

Trigger. Region control.

State read. Whether connector-owned Region exists for the active view.

Outputs/effect. Region dimension state and rerender.

Race and validation guard. Does not infer Region when absent.

Interaction with the rest of the app. Most relevant to Credit rows.

17.2.3.14 R22 - RiskChecker toggle

Trigger. RiskChecker button.

State read. Committed checker setting and expected revision.

Outputs/effect. Refresh request/settings status and post-commit toggle UI.

Race and validation guard. The manager decides whether metadata, dates, Risk, and market must reload.

Interaction with the rest of the app. R23 performs the transaction; checker inventory panel reflects committed state.

17.2.3.15 R23 - Authoritative refresh transaction

Trigger. Refresh Risk, Refresh Portfolios, Refresh P&L, AutoPL tick, Commodity/Checker change, Force Date apply, or bootstrap retry.

State read. Trigger identity, expected revision, committed settings, date drafts, manager progress/health.

Outputs/effect. New revision signal, status/error, committed dates/settings, refresh control state; Dash `running` locks buttons.

Race and validation guard. One writer lock, stale expected-revision rejection, validated date drafts, final compare-and-swap, atomic manager commit.

Interaction with the rest of the app. Browser starts/follows progress; revision publication fans out to every revision-driven read callback.

17.2.3.16 R24 - Operating-date banner

Trigger. Committed data revision.

State read. Snapshot Market Date/status and grouped Risk Dates.

Outputs/effect. Market and Risk date cards.

Race and validation guard. Displays committed dates only, never an unapplied draft.

Interaction with the rest of the app. Shared shell and page-local date editors use the same revision.

17.2.3.17 R25 - Committed dashboard setting synchronization

Trigger. Revision or shell hydration.

State read. Commodity, checker, forced-date, and other committed flags.

Outputs/effect. Control Stores, labels, classes, ARIA state.

Race and validation guard. Server snapshot wins after commit/failure; browser drafts remain separate.

Interaction with the rest of the app. Prevents optimistic button labels from becoming false authority.

17.2.3.18 R26 - Force-date draft reducer

Trigger. Date picker changes, Force Risk toggles, clear/cancel, or revision rebase.

State read. Current draft, committed dates, ProductSpec/readiness rows.

Outputs/effect. Normalized draft Store, validation messages, dirty state.

Race and validation guard. Rejects null/boolean/invalid dates and unknown Source Types; revision rebase distinguishes user edits.

Interaction with the rest of the app. R28 enables apply only for a valid draft.

17.2.3.19 R27 - Force-date editor renderer

Trigger. Draft, revision, checker/readiness state, editor disclosure.

State read. Suggested and effective dates per source.

Outputs/effect. Pattern-matching date rows/pickers and explanatory status.

Race and validation guard. Date rows carry exact Source Type identity.

Interaction with the rest of the app. MATCH callbacks R30 control individual pickers.

17.2.3.20 R28 - Force-date action state and apply

Trigger. Draft validity/dirty state and Apply/Cancel actions.

State read. Draft base revision and current manager revision.

Outputs/effect. Button disabled state/status or a refresh transaction request.

Race and validation guard. Apply requires a valid draft based on the current revision.

Interaction with the rest of the app. Successful apply enters R23 and later R25 replaces draft-facing controls with committed state.

17.2.3.21 R29 - Lazy RiskChecker inventory

Trigger. Inventory disclosure opens or committed checker revision changes.

State read. Validated checker inventory.

Outputs/effect. Inventory table and status.

Race and validation guard. No work while closed; exact `.mmm`/Product contract already validated.

Interaction with the rest of the app. Explains checker-driven product eligibility/date behavior.

17.2.4 ui/s07_events.py - Dash server with MATCH

17.2.4.1 R30 - Per-source date-picker enable/value

Trigger. Matching Force Risk checkbox/draft row and revision.

State read. One Source Type's suggested/effective/draft date.

Outputs/effect. That matching date picker's disabled/value/min/max state.

Race and validation guard. Pattern ID binds the output to the same Source Type.

Interaction with the rest of the app. Keeps a large dynamic editor from requiring one handwritten callback per product.

17.2.5 ui/s07_events.py - Dash clientside

17.2.5.1 R31 - Main versus alternate Risk grid visibility

Trigger. View-mode state.

State read. Mode only.

Outputs/effect. Container style/hidden state.

Race and validation guard. No data mutation.

Interaction with the rest of the app. Both rendered grids can share server-prepared state without a visibility round trip.

17.2.5.2 R32 - Credit control visibility

Trigger. Active Risk Type/Greek and available measures.

State read. Control state only.

Outputs/effect. Credit measure selector visibility.

Race and validation guard. No measure is invented.

Interaction with the rest of the app. Selected measure later enters R09.

17.2.6 ui/s07_events.py - Dash server

17.2.6.1 R33 - Static Data file table

Trigger. Allowlisted file dropdown.

State read. Selected allowlist key.

Outputs/effect. DataTable or safe error state.

Race and validation guard. No arbitrary path and no financial refresh.

Interaction with the rest of the app. Independent of the revision pipeline.

17.2.7 ui/s09_factory.py - Dash client/server

17.2.7.1 F01 - Active navigation state

Trigger. Current pathname.

State read. Registered page paths and request prefix.

Outputs/effect. Navigation classes/ARIA current state.

Race and validation guard. Prefix-aware path comparison.

Interaction with the rest of the app. Does not load page data.

17.2.8 ui/s09_factory.py - Dash server

17.2.8.1 F02 - Stock date-pair loader

Trigger. Current/prior date, revision, refresh intent, or pending Stock saved view.

State read. Stock adapter, Portfolio mapping, cache, intent sequence.

Outputs/effect. Loaded-date/revision Stores, filter options/values, counts, hierarchy, status.

Race and validation guard. Nonblocking lock and intent sequence prevent an older slow request from winning.

Interaction with the rest of the app. Populates the page cache consumed by F03/F04.

17.2.8.2 F03 - Stock filter and hierarchy reducer

Trigger. Five filters, exclude, promotion threshold, loaded dates, or pattern row toggle.

State read. Cached StockPageData and current open paths.

Outputs/effect. Counts, normalized filter Store, open paths, hierarchy children.

Race and validation guard. If cache key is absent returns `no_update`; hierarchy-only action avoids rewriting unrelated outputs.

Interaction with the rest of the app. Saved-view application resolves through the loader/filter controls.

17.2.8.3 F04 - Stock source-row lazy renderer

Trigger. Load/Hide source rows, loaded date pair, or filters.

State read. Cached mapped comparison and requested-state Store.

Outputs/effect. Source DataTable panel, requested state, button label, disclosure open state.

Race and validation guard. Rows are not serialized until explicitly requested; request resets across snapshot/date keys.

Interaction with the rest of the app. Uses the same filter semantics as F03.

17.2.9 ui/s08_plevents.py - Dash server

17.2.9.1 P01 - P&L filter options and saved-view target

Trigger. Revision, current P&L filters, or pending P&L saved-view request.

State read. Governed P&L base dimensions and request base/target.

Outputs/effect. Five dropdown options/values and exclude control.

Race and validation guard. Manual supersession wins; invalid target values are normalized/pruned.

Interaction with the rest of the app. All P&L subsections read these authoritative controls.

17.2.9.2 P02 - P&L aggregate hierarchy

Trigger. Revision, P&L filters, hierarchy row actions, dimension setting.

State read. Governed effective P&L frame and open/selected scope state.

Outputs/effect. Aggregate table, open paths, selected Signoff/Portfolio scope, summaries.

Race and validation guard. Prunes stale scope/path and preserves unavailable values.

Interaction with the rest of the app. Selected scope materializes P03/P04 and editor callback families.

17.2.9.3 P03 - Effective Signoff editor Store

Trigger. Revision, selected Signoff scope, persisted adjustments, or unsaved editor data.

State read. Calculated base plus repository adjustment revision.

Outputs/effect. Revisioned effective Signoff rows and metadata.

Race and validation guard. Merge by governed key; stale browser edits are reconciled only to matching stable row IDs.

Interaction with the rest of the app. P07 renders/edits it; P12 sends it.

17.2.9.4 P04 - Effective Portfolio editor Store

Trigger. Revision, selected Portfolio scope, persisted adjustments, or unsaved editor data.

State read. Calculated base plus repository adjustment revision.

Outputs/effect. Revisioned effective Portfolio rows and metadata.

Race and validation guard. Exact Portfolio/Concerto identity; no row-position merge.

Interaction with the rest of the app. P08 renders/edits it; P13 sends it.

17.2.10 ui/s08_plevents.py / ui/s12_plhistory.py - Dash server

17.2.10.1 P05 - P&L history hierarchy

Trigger. Page filters, open/toggle actions, period comparison state, history signature.

State read. Cached normalized historical frame.

Outputs/effect. Expandable history table, normalized open paths, selected metric state, status.

Race and validation guard. Global latest date, descendant pruning, missing remains missing.

Interaction with the rest of the app. Metric selection feeds P06.

17.2.11 ui/s08_plevents.py - Dash server

17.2.11.1 P06 - P&L history chart

Trigger. Selected history cell/path, type choices, range, filters.

State read. Exact historical series for selected identity.

Outputs/effect. Plotly figure and chart status.

Race and validation guard. No series for stale/hidden path; no fabricated dates or zeros.

Interaction with the rest of the app. Browser theme controller relayouts the mounted figure.

17.2.12 ui/s08_plevents.py - Dash server (factory instance)

17.2.12.1 P07 - Signoff editor reducer/render

Trigger. Effective Store, DataTable edit/filter/sort/page/selection, selected Signoff scope.

State read. Stable row IDs, loaded revision/date, prior browser edit state.

Outputs/effect. DataTable data/columns/styles, editor state, selection, summary/status; uses `Patch` where possible.

Race and validation guard. Late edit recovery and stable IDs prevent edits attaching to replaced rows.

Interaction with the rest of the app. P09 summarizes selection; P10 saves; P12 sends.

17.2.12.2 P08 - Portfolio editor reducer/render

Trigger. Effective Store, DataTable edit/filter/sort/page/selection, selected Portfolio scope.

State read. Stable row IDs, loaded revision/date, prior browser edit state.

Outputs/effect. Portfolio DataTable and state/status outputs.

Race and validation guard. Same reducer contract as P07 with exact Portfolio scope.

Interaction with the rest of the app. P09/P10/P13 and `s03_select.js`.

17.2.13 ui/s08_plevents.py - Dash clientside

17.2.13.1 P09 - P&L DataTable selected-cell summaries

Trigger. Selected cells in each editor.

State read. Visible table data and selection coordinates.

Outputs/effect. Count/sum/average/min/max summary text.

Race and validation guard. Pure browser presentation; ignores nonnumeric cells.

Interaction with the rest of the app. `assets/s03_select.js` creates range selection using native DataTable behavior.

17.2.14 ui/s08_plevents.py - Dash clientside/server

17.2.14.1 P10 - Clear editor selections

Trigger. Clear button, copy cleanup, scope/table replacement.

State read. Current DataTable selection.

Outputs/effect. Empty selected-cells state.

Race and validation guard. Does not change editor values.

Interaction with the rest of the app. JavaScript clicks generated clear controls when switching tables.

17.2.15 ui/s08_plevents.py - Dash server

17.2.15.1 P11 - Save adjustments

Trigger. Save button.

State read. Effective edited rows, loaded adjustment revision, committed data revision/date, selected scopes and filters.

Outputs/effect. Atomic repository revision, rebuilt effective Stores, status.

Race and validation guard. Exact schema/key/governance, stale revision rejection, finite values, no duplicate identities.

Interaction with the rest of the app. Repository replacement overlay becomes input to future P03/P04.

17.2.15.2 P12 - Send all governed P&L

Trigger. Send All button.

State read. Current committed governed effective frame and authoritative P&L filters.

Outputs/effect. Sender invocation result/status.

Race and validation guard. Revalidates revision, date, filters, mapping, Concerto grain, duplicates, and values immediately before send.

Interaction with the rest of the app. Fixture sender refuses external delivery.

17.2.15.3 P13 - Send selected Signoff Group

Trigger. Signoff Send button.

State read. Selected Signoff scope and its effective editor frame.

Outputs/effect. Scoped sender result/status.

Race and validation guard. Scope must still exist under current filters/revision/date.

Interaction with the rest of the app. Uses same domain validator/sender as P12.

17.2.15.4 P14 - Send selected Portfolio

Trigger. Portfolio Send button.

State read. Selected Portfolio and its effective editor frame.

Outputs/effect. Scoped sender result/status.

Race and validation guard. Exact selected Portfolio and current governed identity.

Interaction with the rest of the app. Uses same domain validator/sender as P12.

17.2.16 ui/s11_saved_views.py - Dash server (generic registration)

17.2.16.1 SVR1 - Risk saved-view current label

Trigger. Saved-view selector or catalog options change.

State read. Selected identifier and normalized option list.

Outputs/effect. Human-readable current-view label.

Race and validation guard. Deleted/unknown identifier falls back to Base.

Interaction with the rest of the app. Keeps collapsed disclosure understandable without opening it.

17.2.16.2 SVR2 - Risk saved-view mutation

Trigger. Save New/Update, Rename, or Delete button.

State read. Name, selected ID, five authoritative filter values, exclude mode, repository.

Outputs/effect. Updated catalog options/selection/name and status.

Race and validation guard. Repository validates paths, duplicate names, IDs, schema, and atomic write; Base cannot be renamed/deleted.

Interaction with the rest of the app. Named documents are shared across Risk, Stock, and P&L.

17.2.16.3 SVR3 - Risk saved-view application request

Trigger. Saved-view selector changes.

State read. Selected stored document and current control state as request base.

Outputs/effect. Versioned apply-request Store with request ID, target, and base.

Race and validation guard. Generic callback does not write individual dropdowns.

Interaction with the rest of the app. The Risk-specific option callback applies target values.

17.2.16.4 SVR4 - Risk saved-view acknowledgement

Trigger. Apply request, authoritative controls, or applied-request Store change.

State read. Target values, base values, current values, request ID.

Outputs/effect. Applied-request ID/status and clears/settles pending state.

Race and validation guard. Acknowledges only target reached or a genuine manual supersession of base.

Interaction with the rest of the app. Prevents a pending view from repeatedly overwriting user edits.

17.2.16.5 SVR5 - Risk saved-view action labels/state

Trigger. Selected ID, name, catalog.

State read. Whether Base or named view is selected.

Outputs/effect. Save New/Update label and disabled state for rename/delete/name controls.

Race and validation guard. Base remains synthetic and immutable.

Interaction with the rest of the app. Pure control state; no repository write.

17.2.16.6 SVP1 - P&L saved-view current label

Trigger. Saved-view selector or catalog options change.

State read. Selected identifier and normalized option list.

Outputs/effect. Human-readable current-view label.

Race and validation guard. Deleted/unknown identifier falls back to Base.

Interaction with the rest of the app. Keeps collapsed disclosure understandable without opening it.

17.2.16.7 SVP2 - P&L saved-view mutation

Trigger. Save New/Update, Rename, or Delete button.

State read. Name, selected ID, five authoritative filter values, exclude mode, repository.

Outputs/effect. Updated catalog options/selection/name and status.

Race and validation guard. Repository validates paths, duplicate names, IDs, schema, and atomic write; Base cannot be renamed/deleted.

Interaction with the rest of the app. Named documents are shared across Risk, Stock, and P&L.

17.2.16.8 SVP3 - P&L saved-view application request

Trigger. Saved-view selector changes.

State read. Selected stored document and current control state as request base.

Outputs/effect. Versioned apply-request Store with request ID, target, and base.

Race and validation guard. Generic callback does not write individual dropdowns.

Interaction with the rest of the app. The P&L-specific option callback applies target values.

17.2.16.9 SVP4 - P&L saved-view acknowledgement

Trigger. Apply request, authoritative controls, or applied-request Store change.

State read. Target values, base values, current values, request ID.

Outputs/effect. Applied-request ID/status and clears/settles pending state.

Race and validation guard. Acknowledges only target reached or a genuine manual supersession of base.

Interaction with the rest of the app. Prevents a pending view from repeatedly overwriting user edits.

17.2.16.10 SVP5 - P&L saved-view action labels/state

Trigger. Selected ID, name, catalog.

State read. Whether Base or named view is selected.

Outputs/effect. Save New/Update label and disabled state for rename/delete/name controls.

Race and validation guard. Base remains synthetic and immutable.

Interaction with the rest of the app. Pure control state; no repository write.

17.2.16.11 SVS1 - Stock saved-view current label

Trigger. Saved-view selector or catalog options change.

State read. Selected identifier and normalized option list.

Outputs/effect. Human-readable current-view label.

Race and validation guard. Deleted/unknown identifier falls back to Base.

Interaction with the rest of the app. Keeps collapsed disclosure understandable without opening it.

17.2.16.12 SVS2 - Stock saved-view mutation

Trigger. Save New/Update, Rename, or Delete button.

State read. Name, selected ID, five authoritative filter values, exclude mode, repository.

Outputs/effect. Updated catalog options/selection/name and status.

Race and validation guard. Repository validates paths, duplicate names, IDs, schema, and atomic write; Base cannot be renamed/deleted.

Interaction with the rest of the app. Named documents are shared across Risk, Stock, and P&L.

17.2.16.13 SVS3 - Stock saved-view application request

Trigger. Saved-view selector changes.

State read. Selected stored document and current control state as request base.

Outputs/effect. Versioned apply-request Store with request ID, target, and base.

Race and validation guard. Generic callback does not write individual dropdowns.

Interaction with the rest of the app. The Stock-specific option callback applies target values.

17.2.16.14 SVS4 - Stock saved-view acknowledgement

Trigger. Apply request, authoritative controls, or applied-request Store change.

State read. Target values, base values, current values, request ID.

Outputs/effect. Applied-request ID/status and clears/settles pending state.

Race and validation guard. Acknowledges only target reached or a genuine manual supersession of base.

Interaction with the rest of the app. Prevents a pending view from repeatedly overwriting user edits.

17.2.16.15 SVS5 - Stock saved-view action labels/state

Trigger. Selected ID, name, catalog.

State read. Whether Base or named view is selected.

Outputs/effect. Save New/Update label and disabled state for rename/delete/name controls.

Race and validation guard. Base remains synthetic and immutable.

Interaction with the rest of the app. Pure control state; no repository write.

17.2.17 ui/s13_validate_pl.py - Dash server

17.2.17.1 V01 - Discover completed official dates

Trigger. Validate P&L disclosure summary click.

State read. Archive root completion manifests and prior selected date.

Outputs/effect. Date options/value/disabled state and catalog status.

Race and validation guard. Runs only while disclosure is open; lists only fully validated completed leaves.

Interaction with the rest of the app. Selected date triggers V02.

17.2.17.2 V02 - Render official Predict versus Colossus validation

Trigger. Official date, five P&L filters, exclude mode, or pattern hierarchy row click.

State read. Cached archive comparison and current open paths.

Outputs/effect. Mapped hierarchy, unmapped audit disclosure, status counts, normalized open paths.

Race and validation guard. Unique archived Portfolio authority only; no guessed hierarchy; date/filter changes reset stale open state.

Interaction with the rest of the app. Uses the same authoritative P&L filters as aggregate/edit/send/history.

17.2.18 assets/s02_app.js - Browser DOM event

17.2.18.1 J01 - Theme toggle click

Trigger. Capture-phase click on `#theme-toggle`.

State read. Current theme, localStorage, system preference.

Outputs/effect. Root `data-theme`, button ARIA/title, localStorage, scheduled Plotly relayout.

Race and validation guard. Storage failures are tolerated; selected spreadsheet cells clear first.

Interaction with the rest of the app. CSS tokens and mounted charts update together.

17.2.18.2 J02 - Delegated Risk/Top Book action click

Trigger. Capture-phase click on row-toggle, metric-header-button, or metric-cell-button.

State read. Nearest view token and data attributes; pending open-row state.

Outputs/effect. Version-like action envelope to the correct Dash Store via `set_props`.

Race and validation guard. Requires token for ordinary Risk actions; disabled/malformed nodes ignored.

Interaction with the rest of the app. R05 or R09 validates and reduces the action.

17.2.18.3 J03 - Refresh-trigger click

Trigger. Refresh/Toggle/Force Apply/Retry button click.

State read. Button ID.

Outputs/effect. Starts local progress UI immediately with the matching mode.

Race and validation guard. Financial truth still comes from R23 and `/progressz`.

Interaction with the rest of the app. Makes perceived response immediate while Dash request begins.

17.2.19 assets/s02_app.js - Browser DOM events

17.2.19.1 J04 - Spreadsheet range selection

Trigger. mousedown, mouseover/move, mouseup on semantic metric cells.

State read. Live table geometry and modifier keys.

Outputs/effect. Selected-cell Set, cell classes/ARIA, aggregate summaries.

Race and validation guard. Selection stays within one table; hidden rows and toggle controls excluded.

Interaction with the rest of the app. Copy handler J06 serializes the current rectangle.

17.2.20 assets/s02_app.js - Browser DOM event

17.2.20.1 J05 - Metric/header selection click

Trigger. Modified click on one cell or column header.

State read. Metric data attributes and current selected Set.

Outputs/effect. Toggle/append visible cell selection.

Race and validation guard. Suppresses the ordinary action click when selection modifiers mean spreadsheet selection.

Interaction with the rest of the app. Prevents a Ctrl-click from both selecting and expanding a Risk cell.

17.2.20.2 J06 - TSV copy

Trigger. Document copy event outside editable controls.

State read. Selected visible cells, row/column geometry, `data-copy-value`.

Outputs/effect. `text/plain` and `text/tab-separated-values` clipboard data.

Race and validation guard. No interception for inputs/editors; hidden descendants excluded.

Interaction with the rest of the app. Selection summary hides after successful copy.

17.2.20.3 J07 - Keyboard controller

Trigger. keydown/keyup.

State read. Focused control/cell, modifiers, key code.

Outputs/effect. Readonly-radio navigation, Shift+F8/F9 button clicks, selection toggle, Escape clear.

Race and validation guard. Ignores repeats/disabled controls and suppresses duplicate synthetic clicks.

Interaction with the rest of the app. Keyboard actions enter the same Dash or DOM paths as pointer actions.

17.2.21 assets/s02_app.js - Browser timer/network

17.2.21.1 J08 - Backend progress poll

Trigger. One-second interval while page lives.

State read. Dash running state, progress nodes, `/progressz`, attempt IDs, boot ID, revision baseline.

Outputs/effect. Progress stage/card/bar, global loader, recovery messages, finish state, optional revision handoff.

Race and validation guard. One poll in flight; abort timeout; attempt/revision matching; no unconfirmed success; one session-guarded reload.

Interaction with the rest of the app. May POST `/startz` during bootstrap and call `set_props` on revision Store after commit.

17.2.22 assets/s02_app.js - Browser observers

17.2.22.1 J09 - UI mutation synchronization

Trigger. DOMContentLoaded/load and MutationObserver changes.

State read. New theme buttons, cube roots, Plotly graphs, progress/status nodes.

Outputs/effect. Registers animation, observers, theme/chart sync, and refresh lifecycle hooks.

Race and validation guard. Deduplicates registered roots/nodes.

Interaction with the rest of the app. Adapts to Dash replacing component subtrees.

17.2.23 assets/s02_app.js - Browser pointer/keyboard events

17.2.23.1 J10 - Column resize

Trigger. Resize-handle drag, keyboard arrows, or double-click.

State read. Header geometry and table column cells.

Outputs/effect. Explicit column widths or cleared override.

Race and validation guard. Scoped to mounted risk grids; bounded minimum width.

Interaction with the rest of the app. MutationObservers attach handles to newly rendered headers.

17.2.24 assets/s02_app.js - Browser animation/lifecycle

17.2.24.1 J11 - Cube animation frame and page lifecycle

Trigger. requestAnimationFrame, visibility, resize, blur, pagehide/pageshow.

State read. Visible cube roots, reduced-motion preference, timestamps.

Outputs/effect. Transforms, shadow, sizing; cleanup/reconnect of timers and observers.

Race and validation guard. Stops on nonpersisted page hide and pauses hidden/disconnected roots.

Interaction with the rest of the app. Progress visibility controls which loader is animated.

17.2.25 assets/s02_app.js - Browser DOM event

17.2.25.1 J12 - Quick Search hierarchy toggle

Trigger. Click on `.quick-search-hierarchy-toggle`.

State read. Row path/depth/parent/open data attributes.

Outputs/effect. Expanded state, hidden descendants, glyph/ARIA/title.

Race and validation guard. Clears selection so collapsed hidden rows cannot remain selected.

Interaction with the rest of the app. Avoids a Dash round trip for already-rendered hierarchy.

17.2.26 assets/s03_select.js - Browser DOM events

17.2.26.1 J13 - P&L DataTable drag-to-range adapter

Trigger. mousedown/move/up, trusted click, dragstart, copy, Escape, blur.

State read. Start/end `.dash-cell`, movement distance, table identity.

Outputs/effect. Synthetic native click then Shift-click; clears other table selections.

Race and validation guard. Six-pixel drag threshold; suppresses the following trusted click; cancels invalid/disconnected targets.

Interaction with the rest of the app. Dash DataTable remains the owner of actual selected-cell state consumed by P07/P08/P09.

Chapter 18

Design assessment: what is strong, what is constrained, and why

I did not author this repository, so statements about "thought process" below are inferences from code structure, comments, tests, and repeated invariants. Where the repository explicitly states a reason, this report treats it as evidence. Where it does not, the wording is "the design appears intended to" rather than a claim about the author's private reasoning.

18.1 Strongest design choices

18.1.1 1. Immutable, atomic snapshot publication

Readers never observe a partially refreshed set of products. The manager builds candidate Risk, market, P&L, search, and settings state away from the committed snapshot, validates the complete candidate, then swaps one reference and increments the revision.

This is materially better than updating product DataFrames in place because a Dash page may issue several callbacks concurrently. In-place mutation could show IR from the new attempt and FX from the old attempt in one table.

18.1.2 2. Fail-closed external contracts

Adapters and core validators require exact ordered schemas, unique identities, finite numeric values, consistent statuses, complete Credit measure pairs, and authoritative tenor ranks. The pipeline refuses to "helpfully" guess renamed or missing columns.

This is the correct default for financial calculation. A permissive mapper can make a dashboard look available while silently changing meaning. The trade-off is operational brittleness; schema changes must be coordinated and versioned.

18.1.3 3. One authoritative date chain

Market Date, checker date, suggested Risk Dates, forced dates, and status are computed centrally. Connectors receive dates; they should not independently subtract another business day.

This avoids the classic double-offset defect in which both orchestration and a source wrapper apply T-1.

18.1.4 4. Full MarketBook separated from portfolio Risk

The application retains all market tenors, including points with no position. Risk/P&L then left-joins the subset carried by positions. Quick Market reads the full book; Quick Risk reads the portfolio intersection.

This is better than deriving market views from Risk because a zero-position tenor is still meaningful market information and may be needed by New Trades or Cross Gamma.

18.1.5 5. Missing is not zero

Unavailable quotes, supplemental `dRisk`, absent history dates, and unmapped governance values remain unavailable. Parent aggregation uses minimum-count semantics.

That distinction prevents a display convenience from becoming a false financial statement.

18.1.6 6. Governance is outside display code

Portfolio mapping, Concerto identities, thresholds, reported labels, adjustment overlays, and archive authority are explicit inputs. The UI cannot infer an approved send identity from a visible label.

This supports auditability and lets business governance change without editing calculation formulas.

18.1.7 7. Nonblocking cold start with one writer

The application imports and paints before connector I/O. A process coordinator, attempt IDs, boot IDs, and the manager lock ensure many visitors do not create many first-refresh writers.

The alternative - loading data during import - is simpler but causes health probe timeouts, test side effects, and duplicate work when workers preload.

18.1.8 8. Delegated browser events for large semantic tables

Stable data attributes and a small number of action Stores replace thousands of individual Dash callback inputs. Spreadsheet selection and copy remain local to the browser.

For large tables this is substantially more scalable than registering a callback per cell. The cross-language action contract is the price paid for that scale.

18.1.9 9. Archive completion as a verifiable protocol

A date leaf is visible only after exact files are staged, validated, hashed, and renamed with `_SUCCESS` written as a manifest. Readers distrust filenames and verify the contents.

Directly writing final CSVs would be shorter, but a crash could leave a date that appears complete. The manifest protocol makes completeness testable.

18.1.10 10. Tests as design documentation

The 29 numbered test modules exercise negative contracts, races, and tampering, not just representative numbers. They make subtle intent - exact selection, global latest date, no duplicate writer, no hidden zero - executable.

18.2 Principal limitations

18.3 1. Horizontal scalability is intentionally absent

The authoritative snapshot, refresh lock, progress, startup coordinator, caches, and some repository locks are process-local. Gunicorn therefore uses one process with threads.

This gives consistent reads inside one process, but it creates:

- A single process failure domain.
- No rolling multi-worker deployment without separate warmup behavior.
- No horizontal scale for CPU-heavy pandas work.

- A cold snapshot after process replacement.
- Process-local history/search caches that cannot be shared.

18.3.1 Recommended direction

Introduce a durable immutable snapshot service:

```
writer builds candidate
-> stores revisioned frames/metadata
-> conditional publish of "current revision"
-> readers cache by revision
```

This can be implemented with object storage plus a small metadata database, Redis plus Parquet/object storage, or a dedicated calculation service. Only after authority is external should Gunicorn worker count increase.

18.4 2. The active runtime is still fixture-only

Every checked-in source is visibly fake and external senders refuse delivery. Large comment-only recovered private connectors show migration history but are not production code.

18.4.1 Risk

A manual "uncomment this block and edit registration" process is hard to review, easy to merge incorrectly, and incompatible with automated environment validation.

18.4.2 Recommended direction

Create a connector registry selected by configuration:

```
registry = ConnectorRegistry(
    mode="fixture" | "production",
    products={source_type: adapter},
    governance=...,
    overlays=...,
)
```

Each production adapter should live in a private installable package with its own contract test suite. Startup should fail visibly if `mode=production` resolves any fixture provenance.

18.5 3. Very large modules concentrate change risk

`core/s02_pipeline.py`, `ui/s04_components.py`, `ui/s07_events.py`, `ui/s08_plevents.py`, `ui/s09_factory.py`, and `assets/s02_app.js` each combine several stable responsibilities.

Large size is not itself a defect, but these modules have many reasons to change: new products, new controls, callback races, rendering, storage, and deployment.

18.5.1 Recommended decomposition

- Pipeline: schema validators, product calculations, refresh planner, transaction manager, snapshot model.
- Components: shared shell, Risk table, Quick Search, filter controls, audit panels.
- Events: startup/refresh, filter options, Risk reducer, Quick Search, date settings.
- P&L: pure editor reducer, Dash adapter, save/send effects, history controller.
- Browser: theme, animation, table selection, delegated Risk actions, progress.

Keep one public facade in each original module during migration.

18.6 4. Connector calls are mostly serial

The manager deliberately calls product Risk and per-Underlying market legs in stable order. Serial execution simplifies progress, source safety, and deterministic failure.

At scale it is the dominant cold-start cost.

18.6.1 Recommended direction

Add bounded concurrency only after each connector declares:

- Thread/process safety.
- Maximum in-flight calls.
- Timeout.
- Rate limit.
- Retry classification.
- Whether calls for one product must remain ordered.

Use a bounded executor and collect results into deterministic order before validation. One failed required call should still fail the candidate transaction; optional sources need an explicit policy rather than a blanket exception.

18.7 5. Business-day logic is weekday-based

`pandas` business-day offsets correctly handle weekends but do not, by themselves, know jurisdiction/product holidays.

18.7.1 Recommended direction

Inject a governed calendar service. Store the calendar name/version in snapshot and archive metadata. Add tests for year-end, region-specific holidays, and unexpected market closure.

18.8 6. Local file persistence has deployment limits

Adjustment, saved-view, archive, and history data live on local paths.

- Saved views have process and OS locks.
- Adjustments have a process lock and atomic replacement, but not the same cross-process lock strategy.
- Archive publication relies on local filesystem rename semantics.
- Local storage may disappear on platform redeploy.
- Multiple hosts do not share authority.

18.8.1 Recommended direction

Move user/governed writes to a shared transactional store. Retain local repositories as development implementations behind existing protocols. For object storage, use immutable objects and conditional metadata updates rather than assuming directory rename is atomic.

18.9 7. Application-level identity and authorization are not visible

This snapshot contains no application-owned user authentication, role authorization, per-user saved-view ownership, or attributable audit log for adjustment/save/send actions. The deployment platform may provide authentication, but that is outside the repository.

18.9.1 Recommended direction

Before real sending:

- Require an authenticated principal from trusted middleware.
- Apply role/Portfolio/Signoff authorization on the server, not by hiding controls.
- Include actor, timestamp, revision, Market Date, filter fingerprint, payload digest, and sender result in an append-only audit record.
- Protect write endpoints/actions against cross-site request forgery according to the hosting model.
- Rate-limit startup/refresh/send actions.

18.10 8. Broad UI exception boundaries need structured observability

Outer callbacks sensibly preserve availability and show a safe message. However, a string status is not enough to operate a financial service.

18.10.1 Recommended direction

Emit structured events for every attempt and source call:

```
boot_id, attempt_id, revision_before, requested_mode,
source_type, underlying, function, duration_ms,
result, exception_type, incident_id
```

Add counters, latency histograms, active writer gauge, cache metrics, and archive verification metrics. Return the incident ID to the user; retain full traceback server-side.

18.11 9. Some caches have no explicit capacity policy

Process-local caches keyed by revision, file signatures, identities, or dates are safe for immutable input but can grow throughout a long process.

18.11.1 Recommended direction

Document maximum cardinality and use bounded LRU/TTL policies. Clear revision-specific caches when the old revision falls outside the supported reader window. Expose cache size/hit metrics.

18.12 10. Browser contracts depend on DOM and internal component classes

Delegated Risk actions use repository-owned data attributes, which is controllable. P&L range selection also relies on Dash DataTable internal class names.

18.12.1 Recommended direction

Add Playwright tests pinned to the exact Dash version for:

- Range drag.
- Copy.
- Rapid expand during rerender.
- Progress recovery after worker replacement.
- Theme relayout.
- Keyboard-only operation.
- Narrow/mobile layout.

Run them before dependency updates. Consider a small custom table component if DataTable internals become unstable.

18.13 11. Documentation can drift

The README, two implementation/performance guides, generated manual, source comments, and tests overlap. Some describe prior designs or migration stages.

18.13.1 Recommended direction

Mark documents with:

- Applicable commit or architecture version.
- “Current contract” versus “historical decision record.”
- Generated sections linked to code registries.
- A CI link/check that validates examples and diagram generation.

Move historical rationale into numbered Architecture Decision Records.

18.14 12. No checked-in CI workflow is present

The snapshot includes strong tests and tooling but no `.github/workflows` files.

18.14.1 Recommended direction

A baseline pipeline should run:

1. Ruff format/lint.
2. Fixture `--check`.
3. Full pytest suite with coverage thresholds.
4. Static type checking.
5. Pandoc/manual generation and binary drift check.
6. Browser smoke/accessibility tests.
7. Dependency vulnerability and secret scanning.
8. Publish-staging allowlist test.
9. Archive corruption/tamper test sample.

Chapter 19

Alternatives and why the present choice works - until it does not

19.1 One giant Risk callback versus several chained callbacks

Present choice: one reducer renders all coordinated Risk outputs.

Why it is good now: one network round trip, no externally visible intermediate Store state, and one place to reject a stale view token.

Alternative: small callbacks chained through Stores.

When the alternative wins: when independent panels can update without sharing state and latency is less important. For the current tightly coupled hierarchy it would likely reintroduce races unless an explicit state machine coordinates the chain.

Best improvement: keep one decorated callback but move parsing, reduction, view-model creation, and rendering into pure typed functions.

19.2 In-memory immutable snapshot versus querying a database on every callback

Present choice: read one committed pandas snapshot.

Why it is good now: fast consistent reads, simple revision semantics, and no database round trip for every hierarchy action.

Alternative: callbacks query a shared database.

When the alternative wins: data exceeds memory, many processes must serve the same authority, or row-level authorization must be enforced centrally.

Best improvement: publish immutable columnar revisions to shared storage and retain a bounded in-process cache, rather than returning to ad-hoc live queries.

19.3 Exact schemas versus tolerant input normalization

Present choice: exact columns/order and strict identities.

Why it is good now: semantic drift is visible and calculation is auditable.

Alternative: accept aliases, extra columns, coercible values, and defaults.

When the alternative wins: at an ingestion boundary that explicitly versions and maps several known source schemas.

Best improvement: keep the canonical core strict. Put tolerant, versioned translation in site-owned adapters, record source schema version, and test each translation.

19.4 Delegated JavaScript versus Dash callback per table control

Present choice: one action envelope Store per interaction family.

Why it is good now: bounded callback graph and responsive local selection.

Alternative: pattern-matching callback for every button/cell.

When the alternative wins: small tables or interactions that require server authority before any visual response.

Best improvement: preserve delegation, but version the envelope and generate Python/JavaScript constants from one schema.

19.5 One process with threads versus multiple Gunicorn workers

Present choice: one process protects one in-memory truth.

Why it is good now: all callbacks see the same manager, lock, revision, and progress state.

Alternative: several worker processes.

Why it is unsafe now: each process would build and publish its own snapshot; a user's successive requests could observe different revisions/settings.

When the alternative wins: after snapshot, refresh ownership, progress, and user writes move to shared authority.

19.6 Local atomic files versus a database

Present choice: simple, inspectable, testable files with atomic replacement.

Why it is good now: minimal infrastructure and strong single-host semantics.

Alternative: relational database or object store.

When the alternative wins: multiple users/hosts, durable redeploys, authorization, audit history, conflict resolution, or large archives.

Best improvement: keep repository protocols and add a production implementation; do not leak database calls into callbacks.

19.7 Serial connector calls versus parallel loading

Present choice: deterministic serial source order and progress.

Why it is good now: easiest to validate and safest for unknown private source thread-safety/rate limits.

Alternative: unconstrained parallel tasks.

Why that is not automatically better: it can overload source systems, reorder errors, and create cancellation/resource leaks.

Best improvement: declarative per-connector concurrency limits and a bounded orchestrator.

Chapter 20

Prioritized improvement roadmap

20.1 Priority 0 - required before real money-moving use

1. Replace every fixture source and refusal sender through an explicit production connector registry.
2. Add production-mode provenance enforcement: startup fails if any active source contains fixture provenance.
3. Add authenticated identity, server-side authorization, and append-only audit for adjustment/save/send/force-date actions.
4. Move adjustments, saved views, and send audit to shared durable storage.
5. Introduce governed holiday calendars and record their version.
6. Add connector/send/archive deployment preflight and dry-run modes.
7. Add source timeouts so one private connector cannot hold the writer forever.

20.2 Priority 1 - reliability and performance

1. Externalize immutable snapshot authority and current revision.
2. Add bounded per-connector concurrency where declared safe.
3. Add structured logs, metrics, traces, and incident IDs.
4. Bound every cache and document invalidation.
5. Add graceful shutdown/cancellation rules for active refresh.
6. Add an operational dashboard for attempt duration by source/Underlying and archive completion.

20.3 Priority 2 - maintainability

1. Split the six largest modules by stable responsibility.
2. Extract pure typed reducers/view models from Dash wrappers.
3. Generate action IDs/data-attribute constants from one schema.
4. Move comment-only recovered private code to versioned ADR/reference files or a private connector package.
5. Add static type checking and stricter public API exports.
6. Generate product/data/callback documentation from registries where possible.

20.4 Priority 3 - UX and verification

1. Add Playwright race, keyboard, copy, theme, and progress tests.
2. Add visual regression at desktop/tablet/mobile and both themes.
3. Run automated accessibility checks plus manual keyboard/screen-reader review.
4. Add virtualized rendering or server-side paging for very large raw audit tables.
5. Surface exact snapshot revision, archive digest, data age, and incident ID in a compact diagnostics panel.

Chapter 21

A safer development and release process

A practical change workflow for this repository is:

1. **Write or update the contract first.** Change ProductSpec, schema, protocol, or action schema explicitly.
2. **Update deterministic fixtures.** Include both valid and deliberately invalid examples.
3. **Add negative tests.** Test duplicate identity, missing quote, stale revision, partial write, or conflicting user action - not only the new happy path.
4. **Implement in a pure function first.** Keep Dash/Flask wrappers small.
5. **Run targeted tests, then full tests.**
6. **Run fixture check and documentation/diagram generation.**
7. **Run browser race/accessibility tests.**
8. **Load test a full refresh with realistic source cardinality.**
9. **Canary in read-only mode.** Compare snapshot totals and archives with the incumbent process.
10. **Enable writes/sending behind an explicit feature flag and audit.**
11. **Verify the first official archive by reloading it through the same reader.**
12. **Retain rollback by revision/configuration, not by editing live files.**

This process matches the strongest idea already present in the code: no state is authoritative merely because it was produced; it becomes authoritative only after its contract is validated and it is published atomically.

Chapter 22

Appendix A - Active product catalogue and formulas

The table below is the active fixture-validated catalogue at the pinned commit. The comment-only recovered catalogue contains unsupported historical formula names and is deliberately not active.

Key	Source Type	Risk identity	Axes	Market unit	Formula	Gamma scale / step
`fxdelta`	`fx/delta`	FX / Delta	none (Spot)	pips	percentage	1 / 1
`fxgamma`	`fx/gamma`	FX / Gamma	none (Spot)	outright	Taylor Gamma	1 / 1
`fxvega`	`fx/vega`	FX / Vega	Tenor Swap	vol points	absolute	1 / 1
`irdelta`	`ir/delta`	IR / Delta	Tenor Swap	bp	absolute	1 / 1
`irgamma`	`ir/gamma`	IR / Gamma	Tenor Swap	bp	Taylor Gamma	10,000 / 10
`irdeltavega`	`ir/deltavega`	IR / DeltaVega	Tenor Swap x Tenor Option	bp	percentage	1 / 1
`xccy`	`ir/xccy`	IR / XCCY	Tenor Swap	bp	absolute	1 / 1
`xccyvega`	`ir/xccyvega`	IR / XCCYVega	Tenor Swap x Tenor Option	bp	percentage	1 / 1
`inflation`	`ir/inflation`	IR / Inflation	Tenor Swap	bp	absolute	1 / 1
`inflationvega`	`ir/inflationvega`	IR / InflationVega	Tenor Swap x Tenor Option	bp	percentage	1 / 1
`basis`	`ir/basis`	IR / Basis	Tenor Swap	bp	absolute	1 / 1
`bond`	`ir/bond`	IR / Bond	Tenor Swap	bp	absolute	1 / 1
`creditdelta`	`credit/delta`	Credit / Delta	Tenor Swap	bp	absolute	1 / 1
`creditvega`	`credit/vega`	Credit / Vega	Tenor Swap	bp	absolute	1 / 1
`commodelta`	`commo/delta`	Commo / Delta	Tenor Swap	outright	percentage	1 / 1
`commovega`	`commo/vega`	Commo / Vega	Tenor Swap	vol points	absolute	1 / 1
`cashflownew`	`new-position/cash-flow`	Cash Flow / New	none; no MarketBook	outright	identity	1 / 1

22.1 Formula legend

Let `move = Current - Open` and let `m` be the validated product multiplier.

- **Identity:** P&L is the supplied Risk-like amount times 1. This is used for Cash Flow/New and has no MarketBook.
- **Absolute:** `P&L = Risk * move * m`.
- **Percentage:** `P&L = Risk * (move / Open) * m`; an Open of zero makes P&L unavailable.
- **Taylor Gamma:** `scaled_move = move * gamma_move_scale`; `developed_delta = GammaRisk * scaled_move / gamma_risk_step`; sourced Gamma P&L is `0.5 * developed_delta * scaled_move * m`. A derived Delta exposure row receives `Risk = developed_delta`, `dRisk = missing`, and `P&L = 0`.

All formulas are masked when required market data is unavailable. The table describes metadata, not a recommendation that every market convention should use these formulas in production; production owners must validate units and multipliers.

Chapter 23

Appendix B - Complete tracked-file inventory

The pinned tree contains **123 tracked files**. Every path is listed below. Source files are explained in the preceding chapters; generated CSVs/PNGs and the notebook are described by their contract and role rather than as Python execution.

A machine-readable copy accompanies this report as `Rebirth_File_Inventory.csv`.

Path	Mode	Purpose in the snapshot
`.gitattributes`	Configuration	Git text/binary and line-ending attributes.
`.gitignore`	Configuration	Excludes environments, caches, secrets, runtime adjustments, and unapproved archive leaves; explicitly permits synthetic demo history.
`README.md`	Documentation	Primary project manual and architecture/deployment guidance.
`plotly-cloud.toml`	Deployment configuration	Plotly Cloud application identity used by publishing tooling.
`pytest.ini`	Test configuration	Discovers numbered tests under ‘tests/‘ and enables summary reporting.
`rebirth_cohesive_implementation_guide.md`	Documentation; may lag code	Earlier cohesive implementation guide retained as project documentation.
`rebirth_full_cold_start_and_performance_guide.md`	Documentation; may lag code	Earlier cold-start/performance analysis and recommendations.
`requirements-dev.txt`	Dependency lock input	Development dependencies layered over runtime requirements.
`requirements.txt`	Dependency lock input	Pinned runtime Python dependencies.
`s01_app.py`	Active runtime	Creates the Dash/Flask application and exports ‘app‘/‘server‘; direct-run entry.
`s02_config.py`	Active runtime	Parses environment, prefixes, paths, and immutable runtime settings.
`s03_publish.py`	Active deployment tool	Stages an allowlisted temporary deployment bundle and invokes Plotly Cloud.
`s04_server.py`	Active deployment config	Gunicorn configuration: one process with threaded concurrency.
`adapters/__init__.py`	Active source	Side-effect-free package marker.
`adapters/s01_common.py`	Active + recovered comments	Exact DataFrame/status/Underlying helper contract; also preserves comment-only recovered helper code.
`adapters/s02_ir.py`	Active + recovered comments	IR Delta and DeltaVega active adapters; extensive comment-only recovered private connectors.
`adapters/s03_fx.py`	Active + recovered comments	FX Delta/Gamma/Vega active adapters; comment-only recovered private connectors.
`adapters/s04_credit.py`	Active + recovered comments	Credit Delta active curve adapter with optional Region/measure pairs; comment-only recovered connector.
`adapters/s05_stock.py`	Active fixture boundary	Validated Stock adapter and deterministic date-sensitive fake source.
`adapters/s06_new_positions.py`	Active fixture boundary	Strict mixed MARKET/CASHFLOW new-trade blotter adapter and fake source.
`adapters/s07_cross_gamma.py`	Active fixture boundary	Strict Cross Gamma sensitivity adapter and deterministic fake source.
`adapters/s08_commo.py`	Active contract; missing private implementation	Commodity Delta curve adapter; marks unrecovered real connector body.
`assets/s01_style.css`	Active browser asset	Authoritative tokens, layouts, components, dark mode, responsive and reduced-motion styles.

Continued on next page

Path	Mode	Purpose in the snapshot
assets/s02_app.js	Active browser asset	Theme, animation, spreadsheet selection, delegated Risk actions, refresh progress, recovery, keyboard, resize.
assets/s03_select.js	Active browser asset	P&L Dash DataTable drag-to-range gesture adapter.
core/__init__.py	Active source	Empty side-effect-free package marker.
core/s01_schema.py	Active domain code	Canonical tenor and Portfolio governance schema.
core/s02_pipeline.py	Active domain code	Product catalogue, validation, financial calculations, refresh planner/manager, atomic snapshot publication.
core/s03_search.py	Active domain code	Immutable exact-identity Quick Risk/Market indexes and pivots.
core/s04_pl.py	Active domain code	Governed P&L send, overlay, and historical analysis domain logic.
core/s05_storage.py	Active persistence code	Local revisioned CSV adjustment persistence with atomic replacement/recovery.
core/s06_reporting.py	Active domain code	Post-calculation reported-Underlying mapping.
core/s07_stock.py	Active domain code	Stock validation, prior/current comparison, governance mapping, filters, hierarchy data.
core/s08_saved_views.py	Active persistence code	Versioned JSON saved-filter-view repository with process/OS locks.
core/s09_cross_gamma.py	Active domain code	Cross Gamma validation, market-scope expansion, contribution and output-risk construction.
core/s10_new_trades.py	Active domain code	New Trades MARKET/CASHFLOW market join and P&L publication.
core/s11_risk_archive.py	Active persistence/domain code	Official Risk/Market/Colossus archive protocol, validation, projection, and history reads.
data/s01_readiness.csv	Generated fixture	Synthetic readiness Age rows by Risk Type/Greek.
data/s02_checker.csv	Generated fixture	Synthetic RiskChecker '.mmm' inventory and Product classification.
data/s03_risk.csv	Generated fixture	Synthetic partitioned raw Risk including Credit measures.
data/s04_open.csv	Generated fixture	Synthetic opening MarketBook with authoritative tenor ranks.
data/s05_current.csv	Generated fixture	Synthetic current MarketBook leg.
data/s06_portfolios.csv	Generated fixture	Synthetic governed Portfolio mapping.
data/s07_thresholds.csv	Generated fixture	Synthetic display thresholds by Risk Type/Greek.
data/s08_concerto.csv	Governance fixture	Concerto send identity mapping.
data/s09_reported.csv	Governance fixture	Reported Underlying mapping examples.
data/histo/2026-08-10/_SUCCESS	Generated archive manifest	v2 synthetic completed archive manifest with hashes/counts/schema/status.
data/histo/2026-08-10/colossus.csv	Generated archive data	v2 synthetic Colossus comparison rows.
data/histo/2026-08-10/market.csv	Generated archive data	v2 synthetic official MarketBook.
data/histo/2026-08-10/risk.csv	Generated archive data	v2 synthetic canonical Risk/P&L archive.
data/histo/2026-08-11/histo.csv	Generated legacy history	Legacy synthetic Colossus/actual P&L rows for 2026-08-11.
data/histo/2026-08-11/market.csv	Generated market history	Synthetic historical MarketBook for 2026-08-11.
data/histo/2026-08-11/predicted.csv	Generated legacy history	Legacy synthetic Predict P&L rows for 2026-08-11.
data/histo/2026-08-12/histo.csv	Generated legacy history	Legacy synthetic Colossus/actual P&L rows for 2026-08-12.
data/histo/2026-08-12/market.csv	Generated market history	Synthetic historical MarketBook for 2026-08-12.
data/histo/2026-08-12/predicted.csv	Generated legacy history	Legacy synthetic Predict P&L rows for 2026-08-12.
data/histo/2026-08-13/histo.csv	Generated legacy history	Legacy synthetic Colossus/actual P&L rows for 2026-08-13.
data/histo/2026-08-13/market.csv	Generated market history	Synthetic historical MarketBook for 2026-08-13.
data/histo/2026-08-13/predicted.csv	Generated legacy history	Legacy synthetic Predict P&L rows for 2026-08-13.
data/histo/2026-08-14/histo.csv	Generated legacy history	Legacy synthetic Colossus/actual P&L rows for 2026-08-14.
data/histo/2026-08-14/market.csv	Generated market history	Synthetic historical MarketBook for 2026-08-14.
data/histo/2026-08-14/predicted.csv	Generated legacy history	Legacy synthetic Predict P&L rows for 2026-08-14.
docs/s01_flow.png	Generated binary	Generated architecture diagram.
docs/s02_dates.png	Generated binary	Generated authoritative date-chain diagram.

Continued on next page

Path	Mode	Purpose in the snapshot
docs/s03_market.png	Generated binary	Generated per-Underlying market-flow diagram.
docs/s04_startup.png	Generated binary	Generated cold-start/recovery diagram.
feeds/__init__.py	Active source	Package marker.
feeds/s01_sources.py	Active fixture composition + recovered comments	Fixture CSV source registry, caches, manager composition, placeholders for site senders/history.
jobs/archive_official_risk.ipynb	Active scheduler wrapper	Minimal scheduled notebook delegating official archive work to a tested Python module.
pages/__init__.py	Active page infrastructure	Resolves page builders from the active Flask app service container.
pages/not_found_404.py	Active page	Prefix-safe 404 layout.
pages/pnl.py	Active page	Thin P&L page registration/layout resolver.
pages/risk.py	Active page	Thin Risk page registration/layout resolver.
pages/static_data.py	Active page	Static Data page registration/layout.
pages/stock.py	Active page	Thin Stock page registration/layout resolver.
tests/s01_schema.py	Executable specification	Governed schema and ProductSpec catalogue invariants.
tests/s02_checker.py	Executable specification	Business dates, readiness, checker inventory, and delay validation.
tests/s03_adapters.py	Executable specification	Executable adapter examples and fail-closed source contracts.
tests/s04_market.py	Executable specification	MarketBook joins, rank authority, search grain, and quote aggregation.
tests/s05_pl.py	Executable specification	P&L governance, overlays, history, and adjustment storage.
tests/s06_ui.py	Executable specification	UI aggregation, components, row keys, markup, and accessibility contracts.
tests/s07_integration.py	Executable specification	End-to-end refresh transaction and snapshot lifecycle.
tests/s08_feeds.py	Executable specification	Fixture feed composition, caches, fake provenance, and sender placeholders.
tests/s09_plui.py	Executable specification	P&L filters, editors, selections, save, and send callbacks.
tests/s10_reads.py	Executable specification	Read-path efficiency and snapshot immutability.
tests/s11_fixtures.py	Executable specification	Deterministic fixture generation/check.
tests/s12_startup.py	Executable specification	Cold shell, coordinator ownership, watchdog, endpoint, and recovery races.
tests/s13_publish.py	Executable specification	Deployment staging allowlist and publish invocation.
tests/s14_reporting.py	Executable specification	Reported-Underlying mapping.
tests/s15_overlays.py	Executable specification	Adjustment/supplemental overlay semantics.
tests/s16_refresh_shell.py	Executable specification	Refresh shell/progress success/failure/retained-snapshot states.
tests/s17_stock.py	Executable specification	Stock source, comparison, mapping, filters, hierarchy, and cache.
tests/s18_new_positions_adapter.py	Executable specification	Raw New Trades MARKET/CASHFLOW adapter contract.
tests/s19_risk_filters.py	Executable specification	Risk filters, reducer, view tokens, and stale delegated actions.
tests/s20_connector_provenance.py	Executable specification	Connector/fake provenance preservation.
tests/s21_pipeline_provenance.py	Executable specification	Transformation provenance through publication.
tests/s22_refresh_dates.py	Executable specification	Selective refresh planner and forced-date revisions.
tests/s23_saved_views.py	Executable specification	Saved-view storage concurrency and UI request protocol.
tests/s24_plhistory.py	Executable specification	P&L history hierarchy, periods, selection, and chart.
tests/s25_cross_gamma.py	Executable specification	Cross Gamma validation/contribution/scope.
tests/s26_new_trades.py	Executable specification	New Trades market join/formulas/publication.
tests/s27_expandable_visuals.py	Executable specification	Open-path, hidden descendant, and accessible expansion behavior.
tests/s28_validate_pl.py	Executable specification	Official Predict/Colossus comparison and hierarchy.
tests/s29_risk_archive.py	Executable specification	Archive atomicity, compatibility, tamper detection, and scheduler.
tools/__init__.py	Active source	Side-effect-free package marker.
tools/s01_fixtures.py	Active developer tool	Generates and checks deterministic connector/history fixtures.
tools/s02_manual.py	Active documentation tool	Generates README diagrams and ReportLab manual.
tools/s03_archive_official_risk.py	Active operational tool	Official archive scheduler/CLI composition boundary.
ui/__init__.py	Active source	Side-effect-free package marker.
ui/s01_contracts.py	Active UI boundary	Protocols for manager, snapshots, repositories, and history services.
ui/s02_constants.py	Active UI configuration	Portfolio fields, hierarchy, metrics, IDs/types, Credit/IR display rules.
ui/s03_aggregate.py	Active UI model code	Pure risk filtering, promotion, aggregation, row-key, and hierarchy helpers.

Continued on next page

Path	Mode	Purpose in the snapshot
ui/s04_components.py	Active component library	Risk/shared components, tables, controls, progress shell, Quick Search, unmapped audit.
ui/s05_staticdata.py	Active page component	Allowlisted fixture CSV inspection page.
ui/s06_plview.py	Active component library	P&L filters, aggregate/editor/send/history/validation components.
ui/s07_events.py	Active callback controller	Risk/shared/startup/refresh/date/filter/Quick Search callbacks.
ui/s08_plevents.py	Active callback controller	P&L editor/history/save/send callbacks and editor callback factory.
ui/s09_factory.py	Active composition root	Dash/Flask factory, routes, service binding, page builders, Stock callbacks/cache.
ui/s10_stock.py	Active UI model/components	Stock page models, filters, path tokens, hierarchy and table components.
ui/s11_saved_views.py	Active callback/component controller	Generic saved-view controls and callback family.
ui/s12_plhistory.py	Active UI model/components	P&L historical hierarchy/table/component logic.
ui/s13_validate_pl.py	Active domain/UI controller	Official archive comparison and validation callbacks/components.
ui/s14_pl_filters.py	Active UI model code	Pure canonical P&L filter mapping/application.

Chapter 24

Appendix C - Beginner glossary

24.1 Adapter

A small boundary that calls a site-specific source and translates or validates its result into the repository's canonical contract. In this project adapters do not perform the whole refresh; they supply one product's Risk/Open/Current legs.

24.2 Aggregate

A summary produced by grouping several leaf rows and combining numeric values, usually with `pandas.groupby`. Aggregation must define how missing values behave.

24.3 Atomic

An operation that becomes visible all at once. The refresh manager publishes one complete snapshot pointer; file repositories replace a complete temp file rather than exposing a half-written file.

24.4 Business day

A day considered open for business-date arithmetic. The current code uses pandas weekday-based business-day offsets; jurisdiction holidays need a separate calendar.

24.5 Callback

A function Dash calls when declared Input component properties change. It reads Inputs and optional State, then returns values for declared Outputs. Client-side callbacks run in the browser; server callbacks run in Python.

24.6 Callback factory

A Python function that registers several callbacks from shared configuration. This repository uses factories for parallel P&L editor scopes and saved-view controllers.

24.7 Candidate snapshot

The private set of frames and metadata being built during a refresh. It is not visible to readers until every required validation passes.

24.8 Compare-and-swap

A write succeeds only if the current revision still equals the revision from which the writer planned. It prevents a stale request from publishing over a newer state.

24.9 Concerto field

The governed external identity used by the P&L sending workflow. It is mapped from exact Risk Type/Greek pairs rather than inferred from labels.

24.10 Connector

A function or service that retrieves source data such as Risk, opening quotes, current quotes, Portfolio mapping, or Colossus P&L.

24.11 Current

The selected Live or OFFICIAL market quote at Market Date. It is paired with Open to calculate Move.

24.12 Dash

A Python web framework built on Flask, React, and Plotly. Python declares components and callbacks; the browser renderer synchronizes component properties with the server.

24.13 `dcc.Store`

A Dash component used to hold JSON-serializable browser/session state. In this repository Stores carry filters, open paths, action envelopes, revisions, saved view requests, and editor state - not the authoritative financial snapshot.

24.14 Delegated event

One browser listener attached high in the DOM handles clicks from many child buttons by inspecting data attributes. This avoids registering one listener or Dash callback per cell.

24.15 `dRisk`

The source-provided change in Risk. It is distinct from Risk and from P&L. A missing `dRisk` is not zero.

24.16 Exact identity

The complete set of columns that uniquely identifies a position, quote, send row, saved view, or archive row. Validators reject duplicate exact identities.

24.17 Fixture

Deterministic fake data used to demonstrate and test a contract. All visible fixture business identities contain `FAKE_REPLACE_ME`.

24.18 Flask

The Python HTTP server framework under Dash. This project adds `/healthz`, `/progressz`, and `/startz` routes to Dash's Flask server.

24.19 Force Market / Force Risk

Explicit user-governed date overrides. A browser draft is not authoritative until validated and committed through a refresh revision.

24.20 Grain

The columns that define one row's meaning and uniqueness. Risk position grain, market quote grain, and P&L send grain are different.

24.21 Group

Connector-owned descriptive metadata on Risk rows. The core requires the column but does not constrain its vocabulary.

24.22 Hierarchy path

A tuple or bounded JSON token identifying a node from root dimensions to a particular branch. It is safer than using a visible label alone.

24.23 Idempotent

Repeating an operation has no additional effect once the intended result exists. Archiving an already completed Market Date returns `already_archived` and does not overwrite it.

24.24 Immutable snapshot

A committed revision whose financial content is not modified in place. A new refresh publishes a new revision.

24.25 Input, State, and Output

Dash callback dependencies. Input changes trigger the callback. State is read without triggering it. Outputs are properties the callback writes.

24.26 Live / OFFICIAL

The two exact market-status routing values accepted by the code. Adapters must not guess or alter their spelling.

24.27 MarketBook

The complete set of opening/current quote identities and market-owned tenor ranks, including points where the current portfolio has no Risk.

24.28 Market move

`Current - Open`. Some ProductSpecs use the raw move; percentage formulas divide it by Open.

24.29 Mapped / unmapped Portfolio

A mapped Portfolio has one exact row in the governance configuration. Unmapped financial rows stay visible for remediation but are excluded from mapped totals and approved sends.

24.30 Market-only tenor

A quote point present in MarketBook but absent from current portfolio Risk. Quick Market retains it; Risk/P&L does not manufacture a position for it.

24.31 MutationObserver

A browser API that reports DOM changes. Dash replaces component subtrees after callbacks; observers let the asset code attach behavior to newly mounted nodes.

24.32 Open

The opening quote leg, loaded using the authoritative checker/open date.

24.33 Overlay

Supplemental rows or adjustments combined with the calculated base. P&L adjustments replace matching governed rows rather than adding a second row.

24.34 Pattern-matching callback

A Dash callback using dictionary component IDs and `MATCH`, `ALL`, or `ALLSMALLER`. It lets one callback definition serve dynamic product/path components.

24.35 P&L

Profit and loss. In this repository it is calculated from Risk and market movement according to ProductSpec metadata, or supplied directly for Cash Flow/New.

24.36 ProductSpec

Frozen metadata describing a product's Source Type, Risk identity, axes, market unit, and calculation formula.

24.37 Promotion

A presentation flag based on thresholds. It makes a row visually prominent but does not change the financial value.

24.38 Protocol

A Python structural interface describing required methods/attributes. It lets the UI depend on capabilities rather than concrete classes.

24.39 Quick Market

An exact pivot over the full committed MarketBook.

24.40 Quick Risk

An exact pivot over committed portfolio Risk/P&L rows.

24.41 Revision

A monotonically increasing number assigned only after an atomic successful financial commit. Browser actions use it and related view tokens to detect stale state.

24.42 Risk

A source-provided sensitivity or exposure. Its unit and interpretation depend on ProductSpec and connector contract.

24.43 RiskChecker

Readiness and inventory governance that influences source availability and suggested Risk Dates.

24.44 SearchCatalog

An immutable revision-scoped index of exact Risk and Market identities, labels, row-position arrays, and pivot helpers.

24.45 Signoff Group

The governed Portfolio ownership dimension used for P&L approval and scoped sending.

24.46 Snapshot

The manager's committed collection of dashboard frame, product Risk/market frames, search catalog, dates, settings, errors, and metadata at one revision.

24.47 Source Type

The canonical connector/product key such as `ir/delta` or `credit/vega`.

24.48 Split / origin

Metadata distinguishing sourced Risk, Gamma-derived Delta, Cross Gamma, New Trades, and other supplemental row origins.

24.49 Stale action

A browser action based on an older revision, view token, loaded date, or adjustment revision. The server rejects it rather than applying it to new state.

24.50 Tenor

A market maturity label. `Tenor Swap` and `Tenor Option` are canonical axes; their corresponding order columns come from market authority.

24.51 Thread versus process

Threads in one process share the manager and snapshot. Separate Gunicorn processes do not. That is why this version uses one process with several threads.

24.52 Transaction

The complete refresh attempt: plan, load, validate, calculate, assemble, final revision check, and atomic commit.

24.53 Underlying

The market object being risked or quoted, such as a currency pair, curve, index, or commodity identity.

24.54 View token

A browser/render generation identifier embedded in a Risk grid. A delegated action must carry the token of the view that created it.

24.55 Writer lock

The manager lock that permits one refresh transaction at a time. Readers use the last committed snapshot and do not need to wait for source I/O.

Chapter 25

Final evaluation

At the pinned commit, Rebirth is a thoughtfully governed **single-process, fixture-safe financial dashboard architecture**. Its correctness posture is stronger than its production-operational posture: validation, missing-value semantics, immutable revision publication, stale-action protection, and archive verification are unusually deliberate, while real connectors, shared authority, identity/authorization, and horizontal scaling remain integration work.

The safest path forward is evolutionary. Preserve the exact contracts, immutable snapshot model, one-writer semantics, visible unmapped data, and negative tests. Externalize authority and persistence, replace fixture composition explicitly, split the largest modules behind their current public facades, and add attributable security/observability before enabling real sends.